

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA

LÊ PHÚC ĐỨC

NGHIÊN CỨU VÀ PHÁT TRIỂN HỆ
THỐNG TỰ ĐỘNG PHÁT HIỆN HIỆN
TƯỢNG TRÔI DẠT VÀ CẬP NHẬT MÔ
HÌNH HỌC MÁY THÍCH ỨNG

Chuyên ngành: Khoa học máy tính
Mã ngành: 8480101

LUẬN VĂN THẠC SĨ

Thành phố Hồ Chí Minh, tháng 5 năm 2026

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA

LÊ PHÚC ĐỨC

NGHIÊN CỨU VÀ PHÁT TRIỂN HỆ
THỐNG TỰ ĐỘNG PHÁT HIỆN HIỆN
TƯỢNG TRÔI DẠT VÀ CẬP NHẬT MÔ
HÌNH HỌC MÁY THÍCH ỨNG

Chuyên ngành: Khoa học máy tính
Mã ngành: 8480101

LUẬN VĂN THẠC SĨ

Thành phố Hồ Chí Minh, tháng 5 năm 2026

**CÔNG TRÌNH ĐƯỢC HOÀN THÀNH TẠI
TRƯỜNG ĐẠI HỌC BÁCH KHOA – ĐHQG-HCM**

Cán bộ hướng dẫn khoa học 1: PGS. TS. Thoại Nam

(Ghi rõ họ, tên, học hàm, học vị và chữ ký)

Cán bộ hướng dẫn khoa học 2: TS. Nguyễn Quang Hùng

(Ghi rõ họ, tên, học hàm, học vị và chữ ký)

Cán bộ chấm nhận xét 1:

(Ghi rõ họ, tên, học hàm, học vị và chữ ký)

Cán bộ chấm nhận xét 2:

(Ghi rõ họ, tên, học hàm, học vị và chữ ký)

Luận văn thạc sĩ được bảo vệ tại Trường Đại học Bách Khoa – ĐHQG-HCM
ngày tháng năm 20.....

Thành phần Hội đồng đánh giá Luận văn thạc sĩ gồm:

(Ghi rõ họ, tên, học hàm, học vị của Hội đồng chấm bảo vệ Luận văn thạc sĩ)

1.
2.
3.
4.
5.

Xác nhận của Chủ tịch Hội đồng đánh giá Luận văn và Trưởng khoa quản lý
ngành sau khi luận văn đã được sửa chữa (nếu có).

CHỦ TỊCH HỘI ĐỒNG

TRƯỞNG KHOA

NHIỆM VỤ LUẬN VĂN THẠC SĨ

Họ tên học viên: Lê Phúc Đức MSHV: 2370116
Ngày, tháng, năm sinh: 29/09/2000 Nơi sinh: Thành phố Hồ Chí Minh
Chuyên ngành: Khoa học máy tính Mã số: 8480101

I. TÊN ĐỀ TÀI:

Tên Tiếng Việt:

NGHIÊN CỨU VÀ PHÁT TRIỂN HỆ THỐNG TỰ ĐỘNG PHÁT HIỆN HIỆN
TƯỢNG TRÔI DẠT VÀ CẬP NHẬT MÔ HÌNH HỌC MÁY THÍCH ỨNG

Tên Tiếng Anh:

Research and Development of an Adaptive Machine Learning System with
Automatic Concept Drift Detection and Model Updating

II. NHIỆM VỤ VÀ NỘI DUNG:

Nhiệm vụ	Nội dung
Khảo sát và phân tích	Khảo sát hiện tượng trôi dạt khái niệm, các thống kê kiểm định, khoảng cách phân phối MMD và phát hiện drift không giám sát.
Đánh giá hiệu suất	Tiến hành đánh giá toàn diện các bộ phát hiện drift trên benchmark gồm 14 tập dữ liệu với 30 lần chạy độc lập.
Phân loại trôi dạt	Xây dựng hệ thống SE-CDT để nhận dạng loại drift dựa trên hình dạng tín hiệu MMD không cần nhãn.
Thích ứng mô hình	Đề xuất và triển khai các chiến lược cập nhật mô hình thích ứng tương ứng với từng loại drift được phát hiện.
Tích hợp thời gian thực	Thiết kế và kiểm thử nguyên mẫu hệ thống xử lý dòng thời gian thực trên nền tảng Apache Kafka.

III. NGÀY GIAO NHIỆM VỤ: 25/08/2025

IV. NGÀY HOÀN THÀNH NHIỆM VỤ: 11/05/2026

V. CÁN BỘ HƯỚNG DẪN:

- **CBHD 1:** PGS. TS. Thoại Nam
- **CBHD 2:** TS. Nguyễn Quang Hùng

Tp. HCM, ngày tháng năm 20.....

CÁN BỘ HƯỚNG DẪN
(Họ tên và chữ ký)

HỘI ĐỒNG NGÀNH
(Họ tên và chữ ký)

TRƯỞNG KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH
(Họ tên và chữ ký)

THE TASK SHEET OF MASTER'S THESIS

Full name: Lê Phúc Đức **Student ID:** 2370116
Date of birth: September 29, 2000 **Place of birth:** Ho Chi Minh City
Major: Computer Science **Major ID:** 8480101

I. THESIS TITLE (In Vietnamese):

NGHIÊN CỨU VÀ PHÁT TRIỂN HỆ THỐNG TỰ ĐỘNG PHÁT HIỆN HỆ
THỐNG TRÔI DẠT VÀ CẬP NHẬT MÔ HÌNH HỌC MÁY THÍCH ỨNG

II. THESIS TITLE (In English):

Research and Development of an Adaptive Machine Learning System with
Automatic Concept Drift Detection and Model Updating

III. TASKS AND CONTENTS:

Task	Content
Literature Review	Studying concept drift, statistical testing, Maximum Mean Discrepancy (MMD), and unsupervised drift detection.
Performance Evaluation	Conducting a comprehensive evaluation of drift detectors on a 14-dataset benchmark under 30 independent runs.
Drift Classification	Developing the SE-CDT framework to classify drift types from MMD signal shapes without labels.
Model Adaptation	Implementing type-specific model adaptation and update strategies driven by drift classification.
Real-time Integration	Designing and testing an event-driven streaming prototype on Apache Kafka for real-time monitoring.

IV. THESIS START DAY: August 25, 2025

V. THESIS COMPLETION DAY: May 11, 2026

VI. SUPERVISORS:

- **Supervisor 1:** PGS. TS. **Thoại Nam**
- **Supervisor 2:** TS. **Nguyễn Quang Hùng**

Ho Chi Minh City,

SUPERVISORS

(Full name and signature)

CHAIR OF PROGRAM

COMMITTEE

(Full name and signature)

DEAN OF FACULTY OF COMPUTER SCIENCE AND ENGINEERING

(Full name and signature)

LỜI CẢM ƠN

Lời đầu tiên, học viên xin được bày tỏ lòng biết ơn sâu sắc đến Ban Giám hiệu nhà trường, khoa Khoa học và Kỹ thuật Máy tính của Đại học Bách Khoa - ĐHQG-HCM trong quãng thời gian học viên được học tập và nghiên cứu tại trường trong chương trình Cao học.

Học viên đặc biệt xin được gửi lời cảm ơn sâu sắc đến thầy PGS.TS. Thoại Nam và thầy TS. Nguyễn Quang Hùng, những người thầy hướng dẫn và giám sát trực tiếp quá trình thực hiện luận văn. Nhờ có những góp ý, chỉ bảo tận tình cùng nhiều kiến thức bổ ích, quý giá của quý thầy mà học viên mới có thể hoàn thành tốt được Luận văn tốt nghiệp này.

Cuối cùng, học viên cũng xin gửi lời cảm ơn đến gia đình, người thân, bạn bè, những người đã quan tâm, động viên để học viên có thể hoàn thành tốt luận văn tốt nghiệp này.

Thành phố Hồ Chí Minh, năm 2026

Học viên thực hiện

TÓM TẮT LUẬN VĂN THẠC SĨ

Concept drift là một nguyên nhân quan trọng làm giảm hiệu quả của hệ thống học máy khi dữ liệu vận hành thay đổi theo thời gian. Luận văn đề xuất **SE-CDT (ShapeDD-Enhanced Concept Drift Type)** — một hệ thống phát hiện và phân loại drift tích hợp, không cần nhãn, kèm khung thích ứng mô hình theo loại drift cho luồng dữ liệu.

Module phát hiện của SE-CDT, đặt tên là **ShapeDD-IDW**, cải tiến ShapeDD theo kiến trúc hai bước: tín hiệu Standard MMD được dùng để tìm ứng viên drift, sau đó các ứng viên được kiểm định bằng IDW-MMD (Inverse Density-Weighted MMD) với xấp xỉ Gamma và hiệu chỉnh Bonferroni khi có nhiều đỉnh ứng viên. Trên 14 tập dữ liệu và 30 lần chạy độc lập, module phát hiện đạt $F_1 = 0.531$ với $\delta = 75$ mẫu và cooldown = 150 mẫu, tương đương với DAWIDD trong kiểm định thứ hạng Nemenyi và cao hơn ShapeDD gốc trong cùng benchmark, đồng thời chạy nhanh hơn ShapeDD gốc khoảng $7\times$ nhờ thay permutation test bằng xấp xỉ Gamma.

Module phân loại của SE-CDT đọc hình dạng tín hiệu MMD để phân biệt sudden, gradual, incremental, recurrent và blip drift mà không cần nhãn. Module này dùng bộ nhớ khái niệm cho recurrent drift và một luật blip được nối để chấp nhận tín hiệu có hai hoặc ba đỉnh. Kết quả đạt 80.1% ở mức phân nhóm TCD/PCD, 55.3% ở mức tiêu loại theo trung bình vi mô và 54.4% theo trung bình từng lớp. Sudden đạt 82.4%, Blip đạt 60.8% và Recurrent đạt 71.5%, trong khi Gradual (27.2%) và Incremental (30.3%) là hai trường hợp khó nhất do tín hiệu chuyển tiếp chậm chùng lẩn về phương sai cửa sổ và các chuỗi drift chậm lặp lại bị bộ nhớ khái niệm gán sang nhánh recurrent.

Trong đánh giá thích ứng, chiến lược cập nhật theo loại drift sử dụng nhãn phân loại của SE-CDT cải thiện rõ trên Stepping và Sudden so với không thích ứng, lần lượt đạt 73.96% và 73.34% prequential accuracy. Trên Mixed, chiến lược này đạt 86.36%, gần với Periodic Retrain nhưng tạo ít cảnh báo dư hơn. Nguyên mẫu hệ thống cũng được kiểm thử trên kiến trúc tương thích Kafka/Redpanda để xác nhận quy trình từ phát hiện-phân loại bằng SE-CDT đến cập nhật và nạp lại mô hình.

ABSTRACT

Concept drift is a major cause of performance degradation in machine learning systems when operating data changes over time. This thesis proposes **SE-CDT (ShapeDD-Enhanced Concept Drift Type)**, a unified detector-classifier system for unsupervised drift handling on streaming data, paired with a type-specific adaptation framework.

The **detection module** of SE-CDT, named **ShapeDD-IDW**, extends ShapeDD with a two-stage design. A Standard MMD signal first locates drift candidates; candidate peaks are then validated by IDW-MMD (Inverse Density-Weighted MMD) using a Gamma-approximated null distribution, with Bonferroni correction when several peaks are tested in the same trace. On a 14-dataset benchmark with 30 independent runs, $\delta = 75$ tolerance, and cooldown = 150, the detection module reaches $F_1 = 0.531$, statistically tied with DAWIDD under the Nemenyi rank test and above the original ShapeDD in the same benchmark, while running approximately $7\times$ faster than ShapeDD because the Gamma null replaces the costly permutation test.

The **classification module** of SE-CDT identifies sudden, gradual, incremental, recurrent, and blip drift from the shape of the MMD signal without labels. It uses concept memory for recurrent drift and a relaxed blip rule that accepts two- or three-peak transient signals. The classifier reaches 80.1% TCD/PCD category accuracy, 55.3% micro-averaged subtype accuracy, and 54.4% macro-averaged subtype accuracy. Sudden, blip, and recurrent drift are identified more reliably than the two slow-transition types; gradual and incremental drift are the hardest cases, as their signals overlap in windowed variance and repeated slow drifts are absorbed into the recurrent branch by concept memory.

For adaptation, type-specific model updates driven by SE-CDT’s classification output improve prequential accuracy on Stepping and Sudden scenarios compared with no adaptation, reaching 73.96% and 73.34%, respectively. On the Mixed scenario, the method reaches 86.36%, close to Periodic Retrain but with fewer redundant alarms. A Kafka/Redpanda-compatible prototype is also tested to validate the end-to-end flow from SE-CDT detection to model update and reload.

LỜI CAM ĐOAN

Học viên xin cam đoan rằng luận văn này là công trình nghiên cứu của riêng học viên, được thực hiện dưới sự hướng dẫn khoa học của PGS.TS. Thoại Nam và TS. Nguyễn Quang Hùng. Các nội dung nghiên cứu và kết quả được trình bày trong luận văn là trung thực và do chính học viên thực hiện trong suốt quá trình theo học chương trình Cao học tại Trường Đại học Bách Khoa – ĐHQG-HCM. Mọi tài liệu và nguồn tham khảo được sử dụng đều đã được trích dẫn và ghi nhận đầy đủ.

Học viên xin hoàn toàn chịu trách nhiệm về nội dung nghiên cứu của mình.

Thành phố Hồ Chí Minh,

.....

Học viên thực hiện

MỤC LỤC

LỜI CẢM ƠN	i
TÓM TẮT LUẬN VĂN THẠC SĨ	ii
ABSTRACT	iii
LỜI CAM ĐOAN	iv
MỤC LỤC	v
DANH SÁCH BẢNG BIỂU	x
DANH SÁCH HÌNH VẼ	xii
DANH MỤC TỪ VIẾT TẮT	xiv
DANH MỤC KÝ HIỆU TOÁN HỌC	xvi
1 Giới thiệu đề tài	1
1.1 Thực trạng chung	1
1.2 Tổng quan về bài toán	1
1.3 Mục tiêu đề tài	2
1.4 Phạm vi đề tài và đối tượng nghiên cứu	3
1.4.1 Phạm vi đề tài	3
1.4.2 Đối tượng nghiên cứu	4
1.5 Cấu trúc luận văn	4
2 Cơ sở lý thuyết	5
2.1 Khái niệm về concept drift và phân loại	5
2.1.1 Khái niệm về Concept Drift	5
2.1.2 Phân loại các dạng Concept Drift	5
Phân loại theo sự thay đổi phân phối	6
Phân loại theo tốc độ và mô hình thay đổi	6

2.1.3	Ảnh hưởng của concept drift	8
2.2	ShapeDD - Phương pháp phát hiện trôi dạt dựa trên hình dạng và khử nhiễu tín hiệu	9
2.2.1	Giới thiệu	9
2.2.2	Quy ước ký hiệu	10
2.2.3	Cơ sở lý thuyết	10
	Đại lượng độ lớn trôi dạt (σ)	10
	Hình dạng đặc trưng của drift đột ngột	10
	Tổng quát hoá cho nhiều sự kiện drift	11
2.2.4	Ứng dụng vào lọc nhiễu và phát hiện drift	12
2.2.5	Maximum Mean Discrepancy (MMD)	12
2.2.6	Cách thức hoạt động của ShapeDD	14
	Giai đoạn 1: Thu thập dữ liệu (Data Collection)	14
	Giai đoạn 2: Xây dựng feature (Feature Construction)	15
	Giai đoạn 3: Tính toán sự khác biệt (Difference Computation)	15
	Giai đoạn 4: Xác thực thống kê (Statistical Validation)	16
	Ưu điểm nổi bật của ShapeDD	18
	Hạn chế và điều kiện áp dụng hiệu quả	18
3	Công trình liên quan	20
3.1	Các phương pháp phát hiện trôi dạt	20
3.1.1	Các phương pháp thống kê	20
3.1.2	Các phương pháp dựa trên hiệu năng mô hình	22
3.1.3	Các phương pháp học máy truyền thống	24
3.1.4	Các phương pháp học sâu	25
3.2	Phương pháp phân loại drift CDT-MSW	29
3.3	Các chiến lược cập nhật mô hình thích ứng	30
3.3.1	Tổng quan	30
3.3.2	Phân loại chiến lược cập nhật	30
3.4	Lý do lựa chọn ShapeDD làm nền tảng	32
4	Hệ thống SE-CDT: Phát hiện, Phân loại và Thích ứng Concept Drift	34
4.1	Động lực và tổng quan cách tiếp cận	34
4.1.1	Tổng quan kiến trúc ba module	35
4.2	Đề xuất cải tiến phương pháp phát hiện drift: IDW-MMD với xấp xỉ Gamma	36

4.2.1 Inverse Density-Weighted MMD (IDW-MMD): Tối ưu hóa hiệu suất và độ chính xác	36
Vấn đề của ước lượng MMD truyền thống	36
Ý tưởng cải tiến bằng cách dùng trọng số thay đổi theo vị trí	37
Giải pháp đề xuất: Trọng số nghịch biến mật độ (Inverse Density Weighting)	37
Ảnh hưởng đến tín hiệu phát hiện	38
Phân tích độ phức tạp tính toán	39
Hạn chế và khi nào nên dùng	40
Kiến trúc kết hợp của Detection Module	41
4.3 Phương pháp phân loại drift SE-CDT	42
4.3.1 Cơ sở lý thuyết	42
4.3.2 Đặc trưng phân loại từ tín hiệu $\sigma(t)$	43
4.3.3 Thuật toán phân loại	47
4.3.4 Ưu điểm và độ phức tạp	49
4.3.5 Giải thích lựa chọn ngưỡng	50
4.3.6 Tự hiệu chỉnh ngưỡng phân loại (Online Self-Calibration)	51
4.3.7 Bộ nhớ khái niệm cho Recurrent Drift (Concept Memory)	52
4.4 Khung chiến lược thích ứng mô hình	53
4.4.1 Ma trận quyết định chiến lược	53
4.4.2 Ảnh hưởng của báo động sai và bỏ sót drift	54
4.4.3 Triển khai các chiến lược	54
Chiến lược cho Sudden Drift (Trọng tâm nghiên cứu)	54
Model Caching theo Concept Memory	54
4.5 Kiến trúc đề xuất cho hệ thống phát hiện trôi dạt và cập nhật mô hình thích ứng	55
4.5.1 Tổng quan kiến trúc	55
4.5.2 Thiết kế chi tiết các thành phần	57
Producer và Data Ingestion	57
Consumer và Cơ chế Windowing	57
Quản lý Topic và Giao tiếp	57
4.6 Cơ chế chịu lỗi và đảm bảo độ tin cậy	57
4.6.1 Xử lý lỗi ở mức ứng dụng	58
4.6.2 Xử lý lỗi ở mức hạ tầng (Kafka)	58
4.7 Kết luận chương	58

5	Thực nghiệm và đánh giá	59
5.1	Tổng quan thực nghiệm	59
5.1.1	Mục tiêu và cấu trúc chương	59
5.1.2	Môi trường và cấu hình	59
5.1.3	Tập dữ liệu thực nghiệm	59
5.1.4	Chiến lược đánh giá	63
5.2	Đánh giá khả năng phát hiện của SE-CDT	64
5.2.1	Thiết lập thực nghiệm phát hiện drift (Detection Setup)	64
5.2.2	Kết quả so sánh hiệu suất phát hiện	65
5.2.3	Phân tích ý nghĩa thống kê	66
5.2.4	Đánh giá chi tiết trên từng loại dữ liệu	67
5.2.5	Kiểm tra tỉ lệ báo động sai trên dữ liệu không có drift	71
5.2.6	Đánh giá hiệu suất tính toán	72
5.3	Đánh giá module phân loại của SE-CDT	73
5.3.1	Thiết lập thực nghiệm phân loại drift	73
5.3.2	Kết quả phân loại tổng hợp của SE-CDT	76
5.3.3	Phân tích chi tiết theo loại drift	78
5.3.4	So sánh với CDT-MSW	80
5.4	Đánh giá khả năng thích ứng mô hình	81
5.4.1	Thiết lập thực nghiệm thích ứng mô hình	81
5.4.2	Phạm vi phát hiện trong thích ứng	82
5.4.3	Kết quả phục hồi (Recovery)	82
5.5	Kiểm thử nguyên mẫu trên kiến trúc tương thích Kafka	86
5.5.1	Kịch bản triển khai	86
5.5.2	Kết quả thực tế	87
5.6	Kết luận chương	88
6	Kết luận và Hướng phát triển	89
6.1	Tổng kết các đóng góp chính	89
6.2	Hạn chế và Phạm vi nghiên cứu	90
6.2.1	Trade-off giữa Recall và False Positives	90
6.2.2	Subcategory Accuracy còn hạn chế	90
6.2.3	Ngưỡng phân loại vẫn còn yếu tố heuristic	91
6.2.4	Giả định joint drift trong phân loại	92
6.2.5	Dữ liệu tổng hợp (Synthetic Data)	92
6.2.6	Phạm vi đánh giá thích ứng	92

6.2.7 Nguyên mẫu Kafka chưa phải benchmark production	92
6.3 Hướng nghiên cứu tương lai	93
6.3.1 Mở rộng sang các loại Drift khác nhau	93
6.3.2 Cải tiến phân loại SE-CDT	93
6.3.3 Hướng dài hạn	93
6.4 Kết luận	94
DANH MỤC CÔNG BỐ KHOA HỌC	95
TÀI LIỆU THAM KHẢO	96
PHỤ LỤC	102

DANH SÁCH BẢNG BIỂU

2.1	Liên hệ giữa loại thay đổi dữ liệu và chiến lược thích ứng mô hình . . .	7
2.2	Bảng ký hiệu thống nhất	10
3.1	Các phương pháp DDM-family	24
3.2	Tóm tắt một số mốc chính trong phát triển phương pháp phát hiện concept drift	27
3.3	Phân loại và đặc tính kỹ thuật của các bộ dò Concept Drift tiêu biểu . .	28
3.4	Tổng hợp hiệu năng phát hiện drift từ các nghiên cứu gốc	28
3.5	Mối quan hệ giữa loại trôi dạt và chiến lược cập nhật mô hình thích ứng	32
4.1	Độ phức tạp tính toán của IDW-MMD và các phương pháp liên quan . .	39
4.2	Tổng hợp các đặc trưng phân loại của SE-CDT	47
4.3	Siêu tham số của SE-CDT và giá trị mặc định	47
4.4	Chiến lược thích ứng theo loại Drift	53
4.5	Danh sách Kafka Topics và Chức năng	57
5.1	Quy ước thiết lập ở các mục đánh giá. Cùng một detector nhưng được đo dưới những cấu hình cửa sổ và dung sai khác nhau tùy câu hỏi mà mục đó trả lời; bảng này là nơi đối chiếu duy nhất cho các giá trị W , δ và cooldown xuất hiện rải rác trong chương.	64
5.2	So sánh tổng hợp hiệu suất các phương pháp phát hiện drift	65
5.3	Thứ hạng trung bình theo F1-score trong kiểm định Friedman/Nemenyi	66
5.4	F1-Score theo dataset (Phần 1: Dữ liệu cơ bản và Mild/Moderate Drift) .	67
5.5	F1-Score theo dataset (Phần 2: Drift mạnh và thay đổi mặt phẳng) . . .	67
5.6	F1-Score theo dataset (Phần 3: RBF Blips, GCS và GRS)	67
5.7	F1-Score theo dataset (Phần 4: Standard SEA và trung bình)	68
5.8	Tỉ lệ Type-I error trên dữ liệu tĩnh	71
5.9	So sánh hiệu suất tính toán của các phương pháp	72
5.10	Sáu kịch bản benchmark module phân loại của SE-CDT	73

5.11	Kết quả phân loại drift type của SE-CDT (tổng hợp 1800 sự kiện, 6 kịch bản $\times$ 30 seeds)	76
5.12	Độ chính xác phân loại theo từng loại drift (oracle mode, SE-CDT Std) .	78
5.13	So sánh phân loại CDT-MSW vs SE-CDT	81
5.14	Kết quả Prequential Accuracy theo loại drift ($N = 30$ runs, $T = 5000$ mẫu). Cột FP: số lần báo động sai trung bình mỗi run, theo thứ tự chiến lược (Type-Specific / Simple Retrain / Periodic Retrain / No Adaptation). .	83
6.1	Tóm tắt số liệu chính của luận văn theo ba trục đóng góp. Chi tiết và phân tích thống kê trong Chương 5.	89

DANH SÁCH HÌNH VẼ

2.1 Phân loại drift theo thay đổi phân phối	8
2.2 Phân loại drift theo tốc độ thay đổi	8
3.1 Tổng quan ba giai đoạn của CDT-MSW: phát hiện vị trí trôi dạt, ước lượng độ dài, theo dõi và phân loại tiểu loại bằng TFR/MTFR.	29
4.1 Kiến trúc tổng quan: Hệ thống SE-CDT (gồm Detection Module chạy ShapeDD-IDW và Classification Module phân tích hình dạng $\sigma(t)$) nhận luồng $\{X_t\}$ và sản xuất nhãn loại drift; Adaptation Module dùng nhãn này để chọn chiến lược cập nhật mô hình phù hợp.	35
4.2 Cơ chế Trọng số Nghịch mật độ IDW-MMD	38
4.3 Cơ chế Cửa sổ trượt tạo tín hiệu MMD	41
4.4 Phân tích tín hiệu SE-CDT	43
4.5 Hình dạng tín hiệu MMD của các loại Drift	44
4.6 Cơ chế hoạt động của Variance Ratio	46
4.7 Lưu đồ phân loại drift của SE-CDT	49
4.8 Sơ đồ kiến trúc hệ thống phát hiện và thích ứng Drift	56
5.1 Biểu đồ Critical Difference (CD)	66
5.2 Minh họa tín hiệu phát hiện trên dữ liệu Sudden Drift	69
5.3 Minh họa tín hiệu trên dữ liệu Gradual Drift	70
5.4 Phân tích số lượng báo động giả trên tập dữ liệu tĩnh	70
5.5 Thời gian chạy trung bình của các phương pháp trên đánh giá phát hiện (10,000 mẫu, 30 lần chạy).	72
5.6 Hình dạng tín hiệu SE-CDT — Sudden Drift	75
5.7 Hình dạng tín hiệu SE-CDT — Incremental Drift	76
5.8 Confusion Matrix — 5 lớp drift	79
5.9 Confusion Matrix — Category Level	79
5.10 Prequential Accuracy — Stepping Drift	84
5.11 Prequential Accuracy — Sudden Drift	85

5.12Prequential Accuracy — Mixed Drift	86
5.13Kết quả kiểm thử nguyên mẫu Kafka	87

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Giải nghĩa
<i>Phương pháp và thuật toán</i>	
MMD	Maximum Mean Discrepancy — Khoảng cách phân phối dựa trên hạt nhân
IDW-MMD	Inverse Density-Weighted MMD — MMD có trọng số nghịch biến mật độ
ShapeDD	Shape-based Drift Detection — Phát hiện drift dựa trên hình dạng tín hiệu
ShapeDD-IDW	ShapeDD kết hợp IDW-MMD với xấp xỉ Gamma
SE-CDT	ShapeDD-Enhanced Concept Drift Type — Hệ thống tích hợp phát hiện và phân loại drift không giám sát
CDT-MSW	Concept Drift Type identification based on Multi-Sliding Windows
ADWIN	Adaptive Windowing — Cơ chế cửa sổ thích nghi
CUSUM	Cumulative Sum — Tổng tích lũy cho kiểm soát chất lượng
DDM	Drift Detection Method
EDDM	Early Drift Detection Method
HDDM	Hoeffding's Drift Detection Method
FHDDM	Fast HDDM — Phiên bản nhanh của HDDM
RDDM	Reactive Drift Detection Method
MDDM	McDiarmid Drift Detection Method
KSWIN	Kolmogorov-Smirnov Windowing
SEA	Streaming Ensemble Algorithm — Bộ tạo dữ liệu drift chuẩn
NSE	Non-Stationary Environment (Learn++.NSE)
DAWIDD	Dynamic Adapting Window Independence Drift Detection
D3	Discriminative Drift Detector
KS	Kolmogorov-Smirnov (test)
<i>Phân loại drift</i>	
TCD	Transient Concept Drift — Drift tạm thời (sudden, blip, recurrent)
PCD	Progressive Concept Drift — Drift tiến triển (gradual, incremental)
<i>Đặc trưng và chỉ số</i>	
FWHM	Full Width at Half Maximum — Độ rộng đỉnh tại nửa chiều cao
WR	Width Ratio — Tỷ số độ rộng đỉnh ($FWHM/2l_1$)

Viết tắt	Giải nghĩa
SNR	Signal-to-Noise Ratio — Tỷ số tín hiệu trên nhiễu
CV	Coefficient of Variation — Hệ số biến thiên
PPR	Peak Proximity Ratio — Tỷ số khoảng cách giữa hai đỉnh
DPAR	Dual-Peak Amplitude Ratio — Tỷ số biên độ hai đỉnh
LTS	Linear Trend Strength — Độ mạnh xu hướng tuyến tính (R^2)
SDS	Step Detection Score — Điểm phát hiện bước nhảy
MS	Monotonicity Score — Điểm đơn điệu
VR	Variance Ratio — Tỷ số phương sai trên dữ liệu thô (Mục 4.3)
EDR	Event Detection Rate — Tỷ lệ phát hiện sự kiện
MDR	Miss Detection Rate — Tỷ lệ bỏ sót ($1 - \text{EDR}$)
FP	False Positive — Báo động giả
CAT	Category Accuracy — Độ chính xác phân nhóm (TCD/PCD)
SUB	Subcategory Accuracy — Độ chính xác phân tiểu loại
<i>Toán học và thống kê</i>	
RKHS	Reproducing Kernel Hilbert Space — Không gian Hilbert hạt nhân tái tạo
RBF	Radial Basis Function — Hàm cơ sở hướng tâm (Gaussian kernel)
HSIC	Hilbert-Schmidt Independence Criterion — Tiêu chí độc lập Hilbert-Schmidt
i.i.d.	Independent and identically distributed — Độc lập cùng phân phối
<i>Hệ thống và hạ tầng</i>	
IoT	Internet of Things — Mạng lưới vạn vật kết nối

DANH MỤC KÝ HIỆU TOÁN HỌC

$P(X, Y)$	Phân phối xác suất đồng thời của đặc trưng và nhãn
$P(X)$	Phân phối xác suất lề của đặc trưng đầu vào
$P(Y X)$	Phân phối xác suất có điều kiện của nhãn
$\sigma(t)$	Tín hiệu độ lớn trôi dạt (drift magnitude signal)
$h_l(t)$	Hàm hình dạng tam giác đặc trưng của ShapeDD
l_1	Kích thước cửa sổ tham chiếu
l_2	Kích thước cửa sổ kiểm tra
s	Bước trượt của cửa sổ
δ	Độ trễ chấp nhận khi đối sánh với ground truth
α	Mức ý nghĩa thống kê
γ	Tham số băng thông của kernel
ϵ	Hằng số ổn định số học
σ_g	Độ lệch chuẩn của bộ lọc Gauss làm mượt tín hiệu
N_{perm}	Số vòng hoán vị trong permutation test
VR	Variance Ratio — tỉ số phương sai trên dữ liệu thô
$\tau_{\text{VR}}^+, \tau_{\text{VR}}^-$	Ngưỡng trên/dưới của Variance Ratio
τ_{WR}	Ngưỡng Peak Width Ratio (WR)
τ_{SNR}	Ngưỡng Signal-to-Noise Ratio (SNR)
τ_{CV}	Ngưỡng Periodicity CV
τ_{match}	Ngưỡng so khớp của Concept Memory
M, n_s	Số snapshot và kích thước mỗi snapshot trong Concept Memory

CHƯƠNG 1

GIỚI THIỆU ĐỀ TÀI

1.1 Thực trạng chung

Hệ thống phát hiện gian lận của một ngân hàng có thể đạt độ chính xác cao trong năm đầu vận hành, nhưng sau một thời gian dài, tỷ lệ cảnh báo sai tăng gấp ba lần. Một mô hình dự báo bảo trì máy móc trong nhà máy thép hoạt động ổn định suốt mùa khô, đến mùa mưa thì hiệu suất giảm mạnh do điều kiện vận hành thay đổi. Đây không phải lỗi của thuật toán hay của kỹ sư xây dựng mô hình, mà là hiện tượng *trôi dạt khái niệm* (concept drift): phân phối dữ liệu thực tế thay đổi theo thời gian trong khi mô hình vẫn “đứng yên” [1].

Về mặt toán học, concept drift xảy ra khi $P_{train}(X, Y) \neq P_{online,t}(X, Y)$, trong đó P_{train} và $P_{online,t}$ lần lượt là phân phối đồng thời của dữ liệu đầu vào và nhãn trong giai đoạn huấn luyện và vận hành tại thời điểm t [1]. Concept drift có thể xuất hiện dưới nhiều dạng: *sudden drift*, *gradual drift*, *incremental drift*, và *recurrent drift* [2].

Các mô hình học máy được huấn luyện với giả định dữ liệu trong tương lai sẽ tuân theo cùng phân phối với dữ liệu huấn luyện (giả định IID). Tuy nhiên, môi trường sản xuất công nghiệp hiếm khi tĩnh: thiết bị lão hóa, quy trình thay đổi, cảm biến xuống cấp, hành vi người dùng biến đổi [1]. Kết quả là hiệu suất mô hình suy giảm dần theo thời gian, đôi khi đột ngột, làm giảm độ tin cậy hệ thống và tăng chi phí vận hành. Để xử lý, cần có cơ chế (1) phát hiện drift càng sớm càng tốt, và (2) thích ứng mô hình theo các điều kiện mới.

1.2 Tổng quan về bài toán

Concept drift có thể xuất hiện ở hai thành phần của phân phối liên hợp $P(X, y) = P(X) \times P(y|X)$. Luận văn tập trung vào hướng không giám sát, tức là theo dõi sự

thay đổi của $P(X)$ khi nhãn chưa có sẵn. Phạm vi này phù hợp với nhiều hệ thống công nghiệp, nơi nhãn thường đến muộn hoặc tốn chi phí kiểm chứng.

Một ví dụ xuyên suốt là **giám sát chất lượng sản xuất** (Manufacturing Quality Control / Process Monitoring), nơi drift thường liên quan đến thay đổi của quy trình, môi trường hoặc cảm biến [1, 3]. Cảm biến trên dây chuyền liên tục sinh luồng đặc trưng X_t như nhiệt độ, áp suất, rung động, độ ẩm, dòng điện hoặc đặc trưng ảnh. Mô hình sản xuất chính dự đoán lỗi, chất lượng sản phẩm hoặc nhu cầu bảo trì từ luồng này; SE-CDT không thay thế mô hình đó. SE-CDT đứng như một tầng giám sát trước bộ thích ứng: nó theo dõi $P(X)$, phát hiện và phân loại drift, sau đó phát sự kiện cho Adaptation Manager quyết định có cập nhật mô hình sản xuất hay không.

Trong các ứng dụng học máy trên dữ liệu luồng, hệ thống giám sát không chỉ cần phát hiện thời điểm drift. Nếu biết thêm dạng drift, hệ thống có thể chọn cách cập nhật mô hình phù hợp hơn: huấn luyện lại nhanh khi thay đổi đột ngột, cập nhật mềm khi thay đổi dần, dùng lại mô hình cũ khi khái niệm lặp lại, hoặc bỏ qua các nhiễu ngắn hạn.

Đề tài xây dựng hệ thống SE-CDT (ShapeDD-Enhanced Concept Drift Type) — một hệ thống tích hợp phát hiện, phân loại và thích ứng với concept drift trên nền tảng ShapeDD [4]. Module phát hiện của SE-CDT (ShapeDD-IDW) dùng IDW-MMD với xấp xỉ Gamma để kiểm định ứng viên drift, cải thiện tốc độ so với permutation test gốc; module phân loại đọc hình dạng tín hiệu MMD để xác định loại drift mà không cần nhãn.

1.3 Mục tiêu đề tài

Luận văn hướng tới bốn mục tiêu chính. Trước hết, đề tài khảo sát các nhóm phương pháp phát hiện drift hiện có, từ kiểm định thống kê, theo dõi hiệu năng mô hình đến các phương pháp được tập trung như MMD và ShapeDD. Từ đó, luận văn đề xuất cải tiến ShapeDD bằng IDW-MMD nhằm giảm chi phí kiểm định và kiểm soát báo động giả tốt hơn trong môi trường cửa sổ trượt.

Mục tiêu tiếp theo là xây dựng SE-CDT, một bộ phát hiện và phân loại loại drift không cần nhãn, lấy cảm hứng từ CDT-MSW [5] nhưng thay vì dựa trên tín hiệu tỉ lệ độ chính xác (accuracy ratio) bằng đặc trưng hình học của tín hiệu MMD. Kết quả phân loại này từ đó được dùng cho phần tiếp theo, nơi chiến lược cập nhật mô hình được chọn theo loại drift đã nhận diện. Cuối cùng, luận văn đánh giá thuật toán trên các tập dữ liệu có ground truth và kiểm thử nguyên mẫu hệ thống trên Kafka. Việc triển khai trong môi trường sản xuất và đánh giá chịu tải quy mô lớn nằm ngoài phạm

vi hiện tại.

1.4 Phạm vi đề tài và đối tượng nghiên cứu

1.4.1 Phạm vi đề tài

Trong môi trường sản xuất công nghiệp, nhân thật (y) thường không có sẵn trong thời gian thực do chi phí và thời gian kiểm định chất lượng cao [1]. Vì vậy nghiên cứu tập trung vào phát hiện concept drift theo hướng **không giám sát** (unsupervised), cụ thể là phát hiện sự thay đổi trong phân phối dữ liệu đầu vào $P(X)$ (còn gọi là covariate shift hoặc data drift), dựa trên giả định joint drift. Phát hiện sớm sự thay đổi trong $P(X)$ đóng vai trò tín hiệu cảnh báo, để hệ thống kích hoạt các cơ chế kiểm tra hoặc cập nhật mô hình khi cần. Luận văn tập trung phát triển một hệ thống kết hợp các thành phần sau:

- Nghiên cứu và cải thiện phương pháp ShapeDD để phát hiện concept drift không giám sát, so sánh với các phương pháp truyền thống.
- Đánh giá hiệu năng trên các tập dữ liệu tổng hợp và các kịch bản drift khác nhau.
- Xây dựng các chiến lược thích ứng mô hình tương ứng với từng loại drift được phát hiện.
- Tùy chỉnh tham số của phương pháp để đảm bảo hiệu quả với các kiểu dữ liệu và hiện tượng trôi dạt khác nhau.
- Thiết kế và triển khai kiến trúc xử lý dữ liệu luồng thời gian thực trên Apache Kafka theo mô hình event-driven, có khả năng mở rộng.

Hệ thống được thiết kế và đánh giá trên nhiều loại drift (sudden, gradual, incremental, recurrent, blip). Cách đánh giá hệ thống như sau:

1. **Đánh giá đa kịch bản:** Thực nghiệm trên nhiều kịch bản drift khác nhau, mỗi kịch bản chạy độc lập nhiều lần để có độ tin cậy thống kê.
2. **Phân tích đa tầng:** Đánh giá cả khả năng *phát hiện* lẫn khả năng *phân loại*, bao gồm phân loại theo nhóm chính và theo từng loại cụ thể.
3. **Đối chiếu với phương pháp CDT-MSW:** Đặt cạnh số liệu công bố của CDT-MSW có giám sát để hình dung khoảng cách giữa hai cách tiếp cận. Đây không phải so sánh trực tiếp trên cùng một benchmark; sự khác biệt về dữ liệu và mức giám sát được thảo luận chi tiết ở Mục 5.3.4.

1.4.2 Đối tượng nghiên cứu

Đối tượng nghiên cứu của luận văn gồm:

- **Phương pháp phát hiện drift:** Phương pháp ShapeDD và phiên bản cải tiến đề xuất ShapeDD-IDW, và các phương pháp so sánh gồm MMD, KS-Test, D3, DAWIDD.
- **Hệ thống phát hiện và phân loại drift:** SE-CDT, hệ thống tích hợp trong đó module phát hiện (ShapeDD-IDW) xác định vị trí drift và module phân loại xác định loại drift từ hình dạng tín hiệu MMD, lấy ý tưởng từ CDT-MSW.
- **Nền tảng xử lý streaming:** Apache Kafka cho việc triển khai hệ thống giám sát *real-time*.

1.5 Cấu trúc luận văn

Luận văn được tổ chức thành năm chương chính:

- **Chương 2 — Cơ sở lý thuyết:** Trình bày nền tảng toán học về concept drift, Maximum Mean Discrepancy (MMD), và phương pháp ShapeDD.
- **Chương 3 — Tổng quan nghiên cứu liên quan:** Khảo sát các phương pháp phát hiện drift hiện có, từ thống kê cổ điển đến học sâu, cùng với phương pháp phân loại drift CDT-MSW.
- **Chương 4 — Mô hình đề xuất:** Trình bày chi tiết hệ thống SE-CDT: module phát hiện ShapeDD-IDW (IDW-MMD + xấp xỉ Gamma), module phân loại drift không giám sát, khung thích ứng mô hình, và kiến trúc trên Apache Kafka.
- **Chương 5 — Thực nghiệm và đánh giá:** Đánh giá có hệ thống ba module (phát hiện, phân loại, thích ứng) trên 14 tập dữ liệu với 30 lần chạy độc lập.
- **Chương 6 — Kết luận và hướng phát triển:** Tổng kết đóng góp, phân tích hạn chế, và đề xuất các hướng nghiên cứu tiếp theo.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

Tóm tắt: Chương này trình bày nền tảng lý thuyết phục vụ cho phương pháp đề xuất. Chương bắt đầu với định nghĩa và phân loại concept drift, tiếp theo là lý thuyết Maximum Mean Discrepancy (MMD), và kết thúc bằng cơ chế hoạt động của ShapeDD – phương pháp được chọn làm cơ sở phát triển cho luận văn.

2.1 Khái niệm về concept drift và phân loại

2.1.1 Khái niệm về Concept Drift

Concept drift (trôi dạt khái niệm) đề cập đến những sự thay đổi không thể dự đoán trước trong phân phối thống kê của dữ liệu theo thời gian [6]. Hiện tượng này đặc biệt phổ biến trong các môi trường vận hành liên tục và năng động như hệ thống IoT [7] hay các quy trình công nghiệp, nơi môi trường thu thập dữ liệu luôn biến động.

Về mặt toán học, concept drift được định nghĩa là sự thay đổi của phân phối xác suất liên hợp (joint probability distribution) giữa dữ liệu đầu vào X và nhãn mục tiêu y tại hai thời điểm khác nhau t và $t + 1$. Nghĩa là:

$$P_t(X, y) \neq P_{t+1}(X, y) \quad (2.1)$$

Trong môi trường thực tế, sự thay đổi này làm cho các mô hình học máy được huấn luyện trên dữ liệu lịch sử trở nên lỗi thời, dẫn đến sự suy giảm nghiêm trọng về hiệu suất dự đoán so với khi được đánh giá trên tập thử nghiệm tĩnh.

2.1.2 Phân loại các dạng Concept Drift

Theo quy tắc nhân xác suất, xác suất liên hợp có thể được phân rã thành $P(X, y) = P(X) \times P(y|X)$. Dựa vào sự thay đổi của các thành phần này, concept drift được

chia thành các nhóm chính như sau [8]:

Phân loại theo sự thay đổi phân phối

- **Sự trôi dạt ảo (Virtual Drift / Covariate Shift):** Đề cập đến tình huống phân phối của các đặc trưng đầu vào $P(X)$ thay đổi, nhưng xác suất hậu nghiệm của lớp mục tiêu $P(y|X)$ (tức là ranh giới quyết định) vẫn không đổi [9]. Dù dữ liệu đầu vào biến đổi, mô hình có thể không bị giảm hiệu suất nghiêm trọng nếu ranh giới quyết định vẫn bao quát được.
- **Sự trôi dạt thực (Real Concept Drift):** Là sự thay đổi trực tiếp trong xác suất hậu nghiệm $P(y|X)$. Sự thay đổi này làm sai lệch ranh giới quyết định ban đầu và bắt buộc phải cập nhật lại mô hình học máy. Ví dụ điển hình là sự thay đổi trong sở thích của người dùng đối với các chủ đề tin tức: cùng một đặc trưng người dùng X , nhưng hành vi click y đã thay đổi [10].

Hai thành phần $P(X)$ và $P(y|X)$ trên lý thuyết có thể thay đổi độc lập, nhưng trong nhiều ứng dụng thực tế chúng thường thay đổi đồng thời (gọi là giả định **joint drift**) [11]. Luận văn dùng giả định này như cơ sở để phát hiện drift theo hướng không giám sát: khi quan sát $P(X)$ thay đổi đáng kể, hệ thống coi đó là tín hiệu cảnh báo có khả năng $P(y|X)$ cũng đang lệch. Trường hợp ngoại lệ (chỉ $P(y|X)$ thay đổi mà $P(X)$ giữ nguyên) nằm ngoài phạm vi quan sát của phương pháp và được xác nhận trên các tập đối chứng ở Mục 5.2.

Phân loại theo tốc độ và mô hình thay đổi

- **Sự trôi dạt đột ngột (Abrupt / Sudden Drift):** Phân phối dữ liệu được thay thế bằng một phân phối hoàn toàn mới gần như ngay lập tức tại một thời điểm t . Loại này dễ phát hiện nhưng đòi hỏi cơ chế thích ứng cực nhanh (ví dụ: cảm biến hỏng hoặc đại dịch COVID-19 bùng phát) [12].
- **Sự trôi dạt tiệm tiến (Gradual Drift):** Quá trình chuyển đổi diễn ra qua một giai đoạn xen kẽ. Trong giai đoạn này, dữ liệu từ cả khái niệm cũ và khái niệm mới cùng xuất hiện, nhưng xác suất xuất hiện của khái niệm mới tăng dần lên cho đến khi thay thế hoàn toàn khái niệm cũ [10].
- **Sự trôi dạt tăng dần (Incremental Drift):** Khái niệm cũ tiến hóa thành khái niệm mới thông qua một chuỗi các khái niệm trung gian (intermediate concepts). Sự thay đổi diễn ra liên tục, đơn điệu với biên độ nhỏ, ví dụ như hiệu suất của cảm biến giảm dần theo thời gian do hao mòn vật lý [8].

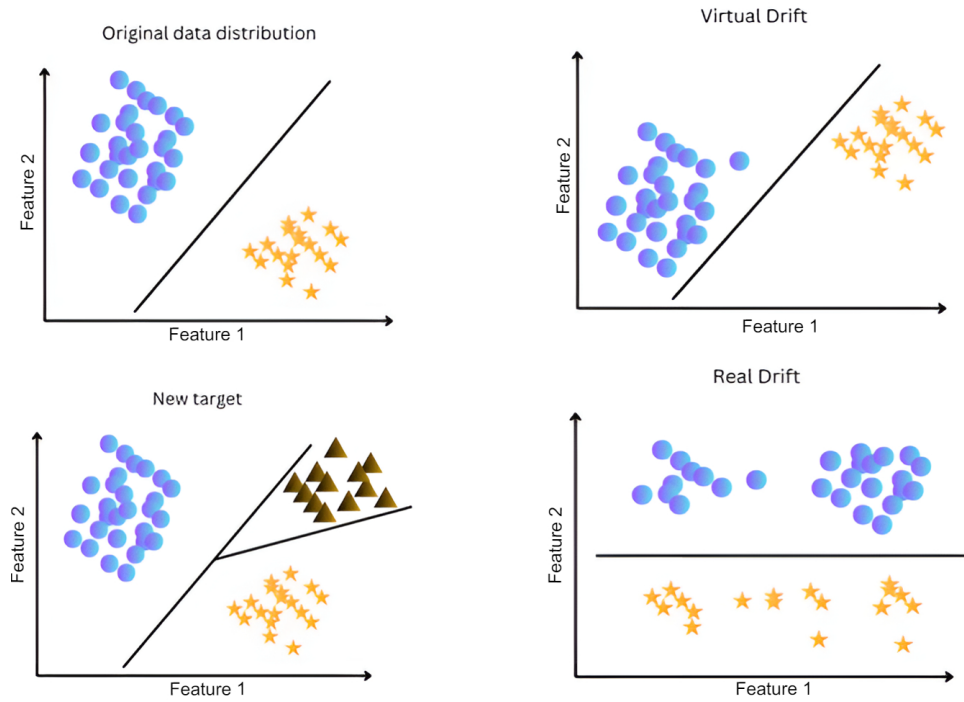
- **Sự trôi dạt lặp lại (Recurrent Drift):** Xảy ra khi một khái niệm cũ đã từng biến mất nay xuất hiện trở lại sau một khoảng thời gian không xác định (ví dụ: thói quen mua sắm thay đổi theo mùa hoặc chu kỳ kinh tế) [8].

Mỗi dạng thay đổi gợi ý một cách thích ứng khác nhau cho mô hình sản xuất. Bảng 2.1 không phải là luật cứng, mà là ánh xạ vận hành thường dùng khi kết hợp phát hiện drift với cập nhật mô hình [10, 11, 13].

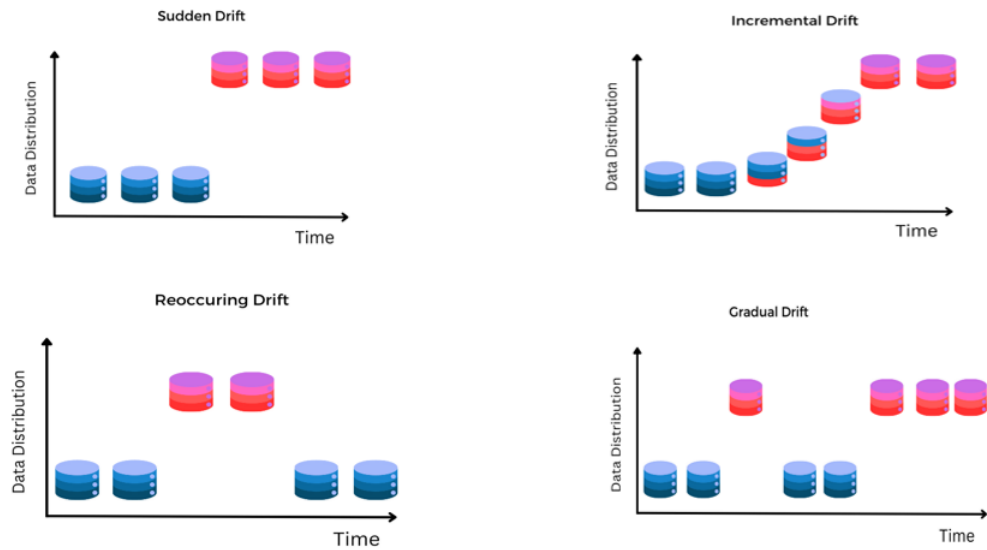
Bảng 2.1 Liên hệ giữa loại thay đổi dữ liệu và chiến lược thích ứng mô hình

Loại thay đổi	Tín hiệu thường gặp	Chiến lược phù hợp
Sudden	Một bước nhảy rõ, phân phối mới thay thế nhanh phân phối cũ	Reset hoặc retrain nhanh trên cửa sổ mới
Gradual	Giai đoạn chuyển tiếp có dữ liệu cũ và mới cùng xuất hiện	Cập nhật theo trọng số thời gian, ưu tiên mẫu gần đây
Incremental	Trung bình/ranh giới dịch chuyển liên tục qua nhiều bước nhỏ	Warm-start hoặc online update để tránh học lại từ đầu
Recurrent	Phân phối từng gặp quay lại theo chu kỳ hoặc bối cảnh	Concept/model cache, chọn lại mô hình đã lưu
Blip	Xung ngắn hạn rồi quay về phân phối cũ	Bỏ qua hoặc cập nhật tối thiểu để tránh quên nhầm

Lựa chọn chiến lược còn phụ thuộc vào chi phí vận hành: nếu false alarm rất đắt (ví dụ retrain mô hình lớn hoặc dừng dây chuyền), ngưỡng phát hiện nên chặt hơn và cần cơ chế xác nhận. Nếu bỏ sót drift gây rủi ro chất lượng hoặc an toàn cao, hệ thống nên ưu tiên recall, chấp nhận nhiều cảnh báo hơn và có human approval trước khi thay thế mô hình trong miền rủi ro cao. Độ trễ nhãn cũng quan trọng: khi nhãn đến muộn, tăng giám sát $P(X)$ như SE-CDT chỉ nên được xem là tín hiệu sớm, còn quyết định cập nhật cuối cùng có thể kết hợp thêm nhãn khi nhãn xuất hiện.



Hình 2.1 Phân loại dựa trên sự thay đổi phân phối: Virtual Drift (chỉ $P(X)$ đổi) và Real Drift ($P(Y|X)$ đổi).



Hình 2.2 Phân loại dựa trên tốc độ thay đổi: Sudden (đột ngột), Gradual (dần dần), Incremental (tăng dần) và Recurrent (lặp lại). Trục ngang biểu diễn thời gian, trục dọc biểu diễn phân phối dữ liệu.

2.1.3 Ảnh hưởng của concept drift

Concept drift có thể ảnh hưởng lớn đến hiệu suất của mô hình dự đoán, đặc biệt là khi mô hình học từ dữ liệu đã lâu trong quá khứ. Một loạt các dịch vụ/ứng dụng trong bối cảnh hệ thống và mạng truyền thông có thể bị cản trở bởi concept drift như [7]:

- **Hệ thống phát hiện xâm nhập (IDS):** Các mẫu tấn công mạng liên tục thay đổi, đòi hỏi IDS phải thích ứng với các mối đe dọa mới
- **Hệ thống phân loại và dự đoán lưu lượng:** Mẫu lưu lượng mạng thay đổi theo thời gian, ảnh hưởng đến độ chính xác dự đoán
- **Industrial IoT (IIoT):** Đặc biệt trong các kỹ thuật bảo trì dựa trên tình trạng (Condition-Based Maintenance - CBM) được sử dụng để dự đoán các điều kiện bất thường và thời gian bảo trì thông qua phân tích dữ liệu IIoT [14]
- **Thành phố thông minh:** Dữ liệu được thu thập cho nhiều mục đích như đảm bảo an ninh mạng, dự đoán ô nhiễm không khí, dự đoán giao thông đường bộ và dự báo tải điện

Phân phối dữ liệu có thể thay đổi theo thời gian do máy móc lão hóa, do quy trình bảo trì, hoặc do các yếu tố môi trường thay đổi. Một kỹ thuật CBM không có khả năng xử lý concept drift sẽ hoạt động kém trong các điều kiện này [14], kéo theo độ tin cậy và độ chính xác của việc phân tích dữ liệu cũng bị ảnh hưởng [15].

Vì vậy, mô hình dự đoán cần có cơ chế phát hiện và tự thích ứng với concept drift để giữ hiệu suất ổn định. Việc thích ứng có thể là cập nhật trên dữ liệu mới, điều chỉnh tham số và cấu trúc, hoặc thay thế hoàn toàn mô hình cũ tùy theo loại drift xảy ra [10, 11].

2.2 ShapeDD - Phương pháp phát hiện trôi dạt dựa trên hình dạng và khử nhiễu tín hiệu

2.2.1 Giới thiệu

Trong học máy cổ điển, dữ liệu thường được giả định là *độc lập và phân phối đồng nhất* (IID, Independent and identically distributed) theo một phân phối tĩnh P_X . Trong các ứng dụng thực tế như luồng dữ liệu từ mạng xã hội, thiết bị IoT hay cảm biến, phân phối dữ liệu thường thay đổi theo thời gian.

Các phương pháp **không giám sát (unsupervised)** cho bài toán phát hiện drift thường dựa trên việc đo *độ khác biệt* giữa hai phân phối dữ liệu trong hai cửa sổ thời gian liên tiếp. Do số lượng mẫu trong mỗi cửa sổ thường nhỏ, các phép đo này dễ bị **nh nhiễu**, khiến việc phân biệt giữa thay đổi thật và dao động ngẫu nhiên trở nên khó khăn.

ShapeDD (Shape-Based Drift Detection) được đề xuất để xử lý vấn đề này, bằng cách khai thác *đặc trưng hình học* của tín hiệu trôi dạt. Phương pháp đặc biệt hiệu

quả với **drift đột ngột (abrupt drift)**.

2.2.2 Quy ước ký hiệu

Trước khi đi vào chi tiết, phần này thống nhất các ký hiệu được sử dụng trong chương:

Bảng 2.2 Bảng ký hiệu thống nhất

Ký hiệu	Ý nghĩa	Giá trị mặc định
l, l_1, l_2	Kích thước cửa sổ	$l_1 = 50, l_2 = 150$
P, Q	Hai phân phối xác suất	—
D_t	Phân phối tại thời điểm t	—
$\sigma(t)$	Drift magnitude signal	—
$h_l(t)$	Hình dạng tam giác đặc trưng	—
γ	Kernel bandwidth (RBF), $k(x, y) = e^{-\gamma\ x-y\ ^2}$	median heuristic
$k(x, y)$	Kernel function	Gaussian RBF

Quy ước ký hiệu: Luận văn dùng $\sigma(t)$ (hoặc $\sigma_{d,l}$) cho drift magnitude signal, và γ cho kernel bandwidth của RBF kernel. Trong các phần lý thuyết trích từ bài báo gốc, σ_k hoặc σ với subscript vẫn được giữ nguyên và có thể hiểu là γ .

2.2.3 Cơ sở lý thuyết

Đại lượng độ lớn trôi dạt (σ)

Xét luồng dữ liệu mà phân phối sinh dữ liệu tại thời điểm t được ký hiệu là D_t . Khi xảy ra trôi dạt, $D_t \neq D_s$ với một số $s < t$. ShapeDD định nghĩa đại lượng **độ lớn trôi dạt** σ như là độ đo sự khác biệt giữa hai phân phối quan sát được trong hai cửa sổ thời gian lân cận.

Với hai cửa sổ $W_l(t) = [t - \frac{l}{2}, t + \frac{l}{2}]$ và $W_l(s) = [s - \frac{l}{2}, s + \frac{l}{2}]$, định nghĩa:

$$\sigma_{d,l}(s, t) = d(P_{W_l(s)}, P_{W_l(t)}), \quad (2.2)$$

trong đó $P_{W_l(t)}$ là phân phối trung bình trên cửa sổ $W_l(t)$ và $d(\cdot, \cdot)$ là độ đo khoảng cách (ví dụ: MMD, Wasserstein, KL, v.v.).

Hình dạng đặc trưng của drift đột ngột

Kết quả lý thuyết trung tâm của ShapeDD được phát biểu chính thức như sau: Cho luồng dữ liệu với phân phối nền p_t thay đổi **tức thời** tại $t = 0$ từ phân phối P sang phân phối Q . Giả sử sử dụng cửa sổ trượt có kích thước l và độ đo khoảng cách $\|\cdot\|$ thỏa mãn các điều kiện metric tiêu chuẩn. Khi đó, tín hiệu độ lớn trôi dạt được phân

tách thành:

$$\sigma_{\|\cdot\|,l,p}(t) = \underbrace{\|P - Q\|}_{\text{Độ mạnh drift}} \cdot \underbrace{h_l(t)}_{\text{Hình dạng drift}}, \quad (2.3)$$

trong đó hàm hình dạng $h_l(t)$ được định nghĩa bởi:

$$h_l(t) = \max \left(0, 1 - \frac{|t|}{l} \right). \quad (2.4)$$

Hàm $h_l(t)$ có dạng tam giác cân trên khoảng $[-l, l]$, đạt cực đại $h_l(0) = 1$ tại thời điểm xảy ra drift ($t = 0$), và suy giảm tuyến tính về hai phía cho tới khi triệt tiêu tại $t = \pm l$. Đặc tính này không phụ thuộc vào dạng phân phối hay độ đo khoảng cách sử dụng, mà chỉ tỷ lệ thuận với hệ số $\|P - Q\|$. Đây là cơ sở để ShapeDD có thể áp dụng cho nhiều miền dữ liệu khác nhau mà không cần thay đổi kernel hay metric.

Tuy nhiên, Triangle Shape Property (2.2.3) chỉ đúng trực tiếp cho sudden drift do tính chất tức thời của nó. Các loại drift khác (gradual, incremental, recurrent) không tạo ra hình dạng tam giác đặc trưng, đây là thiết kế gốc của ShapeDD để tập trung vào phát hiện abrupt drift, tuy nhiên có thể mở rộng để nhận biết các loại drift khác bằng cách phân tích hình dạng tín hiệu trôi dạt:

- Gradual drift tạo peak rộng với slope thoải hơn.
- Incremental drift có thể tạo chuỗi nhiều peak nhỏ liên tục.
- Recurrent drift tạo chuỗi peak lặp lại theo chu kỳ.

Do đó, phương pháp SE-CDT (Chương 4) bổ sung các đặc trưng hình học và thống kê khác trên tín hiệu MMD để phân loại các loại drift này, thay vì chỉ dựa vào hình dạng tam giác.

Tổng quát hoá cho nhiều sự kiện drift

Khi tồn tại nhiều điểm trôi dạt đột ngột tại các thời điểm $t_1 < t_2 < \dots < t_n$, và độ dài của sổ l đủ nhỏ để các tam giác không chồng lên nhau, tổng tín hiệu trôi dạt được biểu diễn như:

$$\sigma_{\|\cdot\|,l,p}(t) = \sum_{i=1}^n \|P_{i-1} - P_i\| h_l(t - t_i). \quad (2.5)$$

Như vậy, tín hiệu tổng thể $\sigma(t)$ có thể xem là tổng chập (linear superposition) của nhiều “hình tam giác” trôi dạt độc lập.

2.2.4 Ứng dụng vào lọc nhiễu và phát hiện drift

Nhận biết được dạng hình học của tín hiệu trôi dạt cho phép thực hiện ước lượng tham số (parametric estimation) cho σ . Thay vì dùng trực tiếp giá trị ước lượng nhiễu $\hat{\sigma}$, ShapeDD khớp một hàm tham số $\tilde{\sigma}$ có dạng:

$$\tilde{\sigma}(t) = \sum_i a_i h_l(t - t_i), \quad (2.6)$$

trong đó a_i biểu diễn biên độ (độ mạnh drift) và t_i là vị trí thời gian của sự kiện drift. Do hàm $h_l(t)$ có cấu trúc hình học cố định và chỉ phụ thuộc vào l , việc khớp mô hình $\tilde{\sigma}$ giúp lọc nhiễu ngẫu nhiên trong $\hat{\sigma}$, xác định chính xác hơn thời điểm trôi dạt t_i , và giảm tỉ lệ báo động giả. Trong thực nghiệm, quá trình khớp hàm này được thực hiện bằng Fast Sequential Function Fitting, áp dụng trên các tín hiệu thu được từ nhiều độ đo (như MMD) và các độ dài cửa sổ khác nhau, nhằm tạo ra ước lượng cuối cùng có độ chính xác cao.

ShapeDD tiếp cận bài toán phát hiện concept drift từ góc nhìn xử lý tín hiệu: khi drift đột ngột xảy ra, độ lớn thay đổi của phân phối tạo ra một tín hiệu có hình tam giác đặc trưng. Nhờ đó, phương pháp có thể lọc nhiễu, ước lượng thời điểm drift và tăng độ tin cậy trong các kịch bản abrupt/sudden drift.

2.2.5 Maximum Mean Discrepancy (MMD)

Như đã nhắc đến ở trên, ShapeDD dựa trên Maximum Mean Discrepancy (MMD) [16], một thước đo thống kê được sử dụng để so sánh hai phân phối xác suất P và Q . Ý tưởng cốt lõi là ánh xạ dữ liệu từ không gian gốc vào không gian feature nhiều chiều, nơi việc so sánh trở nên nhạy cảm hơn với sự khác biệt phân phối.

MMD đo lường khoảng cách giữa hai phân phối bằng cách tìm hàm f có thể phân biệt tốt nhất giữa chúng. Nếu hai phân phối giống nhau, giá trị kỳ vọng của bất kỳ hàm nào áp dụng lên chúng sẽ giống nhau. Ngược lại, nếu khác nhau, sẽ tồn tại một hàm cho phép phân biệt rõ ràng.

MMD được định nghĩa chính thức như:

$$\text{MMD}(P, Q) = \sup_{f \in \mathcal{F}} |\mathbb{E}_{X \sim P}[f(X)] - \mathbb{E}_{Y \sim Q}[f(Y)]| \quad (2.7)$$

trong đó:

- P và Q là hai phân phối cần được so sánh
- $X \sim P$ đại diện cho biến ngẫu nhiên được lấy mẫu từ phân phối P

- $Y \sim Q$ đại diện cho biến ngẫu nhiên được lấy mẫu từ phân phối Q
- $\mathcal{F}$ là một lớp hàm f sao cho $\|f\|_{\mathcal{H}} \leq 1$ trong Reproducing Kernel Hilbert Space (RKHS) [17]
- sup biểu thị giá trị lớn nhất có thể (supremum)

Trong không gian ban đầu, hai phân phối có thể khó phân biệt. Bằng cách ánh xạ vào RKHS thông qua hàm kernel [17], các cấu trúc phức tạp của phân phối trở nên rõ ràng hơn. MMD đo lường khoảng cách giữa “trung bình” (mean embedding) của hai phân phối trong không gian này.

Việc tìm giá trị lớn nhất trên $\mathcal{F}$ là không khả thi về mặt tính toán, do đó MMD thường được triển khai trong RKHS [17] sử dụng kernel trick với hàm kernel $k(x, y)$:

$$k(x, y) = \langle \phi(x), \phi(y) \rangle_{\mathcal{H}} \quad (2.8)$$

Trong đó $\phi(x)$ ánh xạ điểm x vào RKHS $\mathcal{H}$ và $\langle \cdot, \cdot \rangle_{\mathcal{H}}$ là phép nhân vô hướng (inner product) trong $\mathcal{H}$. Gaussian RBF kernel được sử dụng rộng rãi do tính chất universal (có thể xấp xỉ bất kỳ hàm liên tục nào):

$$k(x, y) = \exp(-\gamma \|x - y\|^2), \quad \gamma = \frac{1}{2\sigma_k^2} \quad (2.9)$$

Tham số γ (bandwidth, ký hiệu thống nhất với Bảng 2.2) điều khiển độ nhạy: giá trị lớn tập trung vào khác biệt cục bộ, giá trị nhỏ nắm bắt cấu trúc toàn cục. Ở các phần sau, $\sigma(t)$ luôn chỉ tín hiệu drift magnitude theo thời gian, không phải bandwidth của kernel. Khi đó, MMD bình phương trong RKHS trở thành:

$$\text{MMD}^2(P, Q) = \mathbb{E}_{X, X' \sim P}[k(X, X')] + \mathbb{E}_{Y, Y' \sim Q}[k(Y, Y')] - 2\mathbb{E}_{X \sim P, Y \sim Q}[k(X, Y)] \quad (2.10)$$

Trong đó:

- $\mathbb{E}_{X, X' \sim P}[k(X, X')]$ - độ tương đồng trung bình giữa các điểm trong phân phối P
- $\mathbb{E}_{Y, Y' \sim Q}[k(Y, Y')]$ - độ tương đồng trung bình giữa các điểm trong phân phối Q
- $-2\mathbb{E}_{X \sim P, Y \sim Q}[k(X, Y)]$ - độ tương đồng chéo giữa hai phân phối (có dấu âm)

Nếu $P = Q$, ba thành phần này cân bằng nhau và $\text{MMD}^2 = 0$. Nếu $P \neq Q$, sự khác biệt trong cấu trúc nội bộ và tương tác chéo dẫn đến $\text{MMD}^2 > 0$.

2.2.6 Cách thức hoạt động của ShapeDD

Các phương pháp phát hiện drift cơ bản sẽ được trình bày chi tiết ở Chương 3. Để có ngữ cảnh cho ShapeDD, phần này tóm tắt ngắn gọn các đặc điểm chính của các phương pháp phổ biến:

Các phương pháp dựa trên hiệu suất mô hình như DDM, EDDM, MDDM, FHD-DMS theo dõi thống kê lỗi của mô hình và phát hiện drift khi hiệu suất giảm đáng kể. Các phương pháp này phụ thuộc vào nhãn ground truth và có thể bị trễ vì cần thu thập đủ lỗi.

ADWIN đại diện cho nhóm cửa sổ thích ứng: thuật toán điều chỉnh động kích thước cửa sổ dựa trên sự thay đổi được quan sát, đạt cân bằng giữa độ nhạy và ổn định với đảm bảo lý thuyết $O(\log W)$ về bộ nhớ [18].

DAWIDD phát hiện drift bằng cách đo sự phụ thuộc giữa đặc trưng và thời gian, cho phép phát hiện sớm mà không cần nhãn.

So với các phương pháp này, ShapeDD mang lại góc nhìn khác biệt bằng cách sử dụng Maximum Mean Discrepancy (MMD) để so sánh trực tiếp phân phối dữ liệu trong không gian kernel, kết hợp với kỹ thuật phát hiện hình dạng (shape detection) để giảm nhiễu và tăng độ chính xác định vị drift. ShapeDD hoạt động theo bốn giai đoạn chính như sau [4]:

Giai đoạn 1: Thu thập dữ liệu (Data Collection)

Giai đoạn đầu tiên bao gồm thu thập dữ liệu sử dụng kỹ thuật cửa sổ trượt. ShapeDD sử dụng chiến lược cửa sổ đôi (double window) với kích thước $2l_1$ để so sánh hai đoạn dữ liệu liên tiếp:

- **Cửa sổ tham chiếu:** l_1 điểm dữ liệu đầu tiên $[t - 2l_1 + 1, t - l_1]$
- **Cửa sổ hiện tại:** l_1 điểm dữ liệu tiếp theo $[t - l_1 + 1, t]$

Đối với luồng dữ liệu $\mathcal{S} = \{x_1, x_2, \dots, x_n\}$, cửa sổ trượt W_t có kích thước tổng cộng $2l_1$ được duy trì tại thời điểm t :

$$W_t = \{x_{t-2l_1+1}, x_{t-2l_1+2}, \dots, x_t\} \quad (2.11)$$

Kích thước cửa sổ l_1 là tham số quan trọng nhất của ShapeDD. Cửa sổ nhỏ nhạy với drift nhanh nhưng dễ bị nhiễu; cửa sổ lớn ổn định hơn nhưng phát hiện trễ. Khi cần điều chỉnh tự động, có thể đặt l_1 tỉ lệ với độ dài luồng.

Lưu ý về cấu hình của sổ đối xứng và không đối xứng. Cách trình bày ở trên dùng cửa sổ đôi đối xứng (l_1 mẫu cho mỗi nửa, tổng $2l_1$) vì đây là dạng đơn giản nhất để

suy ra tính chất hình tam giác: khi hai nửa có cùng kích thước, tín hiệu MMD tăng và giảm cân đối quanh điểm drift. Tuy nhiên, phần triển khai và toàn bộ thực nghiệm của luận văn dùng cấu hình *không đối xứng* với cửa sổ tham chiếu $l_1 = 50$ và cửa sổ kiểm tra $l_2 = 150$ (Chương 5 và Phụ lục A.1). Cửa sổ kiểm tra dài hơn cho ước lượng phân phối mới ổn định hơn, đổi lại hình tam giác bị lệch (sườn lên dốc hơn sườn xuống) thay vì cân đối. Tính chất định vị đỉnh vẫn được giữ, nên các suy luận về hình dạng ở các mục sau vẫn áp dụng được; chỉ cần hiểu l trong các công thức lý thuyết là kích thước nửa cửa sổ tham chiếu l_1 .

Giai đoạn 2: Xây dựng feature (Feature Construction)

Trong giai đoạn này, ma trận tương đồng (similarity matrix) được xây dựng sử dụng hàm kernel để nắm bắt mối quan hệ giữa các điểm dữ liệu. Gaussian RBF kernel thường được sử dụng với cùng quy ước γ như Bảng 2.2:

$$k(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2) \quad (2.12)$$

Điều này tạo ra ma trận kernel đối xứng $K \in \mathbb{R}^{2l_1 \times 2l_1}$ trong đó $K_{ij} = k(x_i, x_j)$ biểu thị sự tương đồng giữa các điểm dữ liệu x_i và x_j .

Bandwidth γ có thể được chọn tự động bằng median heuristic:

$$\gamma = \frac{1}{2 \text{median}(\{\|x_i - x_j\| : i, j \in W_t, i \neq j\})^2} \quad (2.13)$$

Phương pháp này đảm bảo kernel thích nghi với scale của dữ liệu.

Ma trận kernel có thể được cập nhật tăng dần khi cửa sổ trượt:

- Tính toán đầy đủ: $O(l_1^2)$ cho mỗi cửa sổ
- Cập nhật tăng dần: Chỉ cần tính $O(l_1)$ phần tử mới khi thêm điểm dữ liệu

Giai đoạn 3: Tính toán sự khác biệt (Difference Computation)

Cốt lõi của ShapeDD bao gồm tính toán sự khác biệt thống kê giữa hai nửa của cửa sổ trượt sử dụng MMD có trọng số. Hàm trọng số $w(t)$ được định nghĩa để tạo ra trọng số tương phản (+1 và -1) cho hai nửa của cửa sổ:

$$w(t) = \begin{cases} +\frac{1}{l_1} & \text{nếu } t \in [1, l_1] \quad (\text{cửa sổ tham chiếu}) \\ -\frac{1}{l_1} & \text{nếu } t \in [l_1 + 1, 2l_1] \quad (\text{cửa sổ hiện tại}) \end{cases} \quad (2.14)$$

Thống kê MMD có trọng số sau đó được tính như:

$$\text{MMD}_t^2 = \sum_{i,j=1}^{2l_1} w_i w_j K_{ij} \quad (2.15)$$

Khai triển công thức trên:

$$\begin{aligned} \text{MMD}_t^2 &= \frac{1}{l_1^2} \sum_{i,j=1}^{l_1} K_{ij} \quad (\text{tương đồng trong cửa sổ tham chiếu}) \\ &+ \frac{1}{l_1^2} \sum_{i,j=l_1+1}^{2l_1} K_{ij} \quad (\text{tương đồng trong cửa sổ hiện tại}) \\ &- \frac{2}{l_1^2} \sum_{i=1}^{l_1} \sum_{j=l_1+1}^{2l_1} K_{ij} \quad (\text{tương đồng chéo}) \end{aligned} \quad (2.16)$$

Đây chính là công thức MMD giữa hai phân phối được định nghĩa bởi hai nửa cửa sổ. Tính toán này được thực hiện trên toàn bộ luồng dữ liệu sử dụng phương pháp cửa sổ trượt, tạo ra chuỗi thời gian các giá trị MMD: $\{\text{MMD}_1^2, \text{MMD}_2^2, \dots, \text{MMD}_T^2\}$.

Khi drift xảy ra tại thời điểm t_d , chuỗi MMD có diễn biến như sau:

- Trước drift ($t < t_d - l_1$): Cả hai nửa cửa sổ đều từ phân phối cũ $\Rightarrow \text{MMD}_t^2 \approx 0$
- Tại drift ($t_d - l_1 \leq t \leq t_d$): Cửa sổ tham chiếu từ phân phối cũ, cửa sổ hiện tại từ phân phối mới $\Rightarrow \text{MMD}_t^2$ tăng mạnh
- Sau drift ($t > t_d + l_1$): Cả hai nửa đều từ phân phối mới $\Rightarrow \text{MMD}_t^2 \approx 0$

Điều này tạo ra hình dạng tam giác (triangular shape) đặc trưng trong chuỗi MMD tại vị trí drift.

Giai đoạn 4: Xác thực thống kê (Statistical Validation)

Giai đoạn cuối cùng bao gồm hai bước: phát hiện hình dạng (shape detection) và xác thực thống kê.

Để phát hiện hình dạng tam giác, ShapeDD sử dụng bộ lọc convolution h'_l được thiết kế để nhạy với biên tăng-giảm:

$$h'_l(t) = \begin{cases} +1 & \text{nếu } t \in [0, l] \\ -1 & \text{nếu } t \in (l, 2l] \end{cases} \quad (2.17)$$

Tín hiệu shape được tính bằng convolution:

$$\text{shape}_t = (h'_l * \text{MMD}^2)(t) = \sum_{i=0}^{2l} h'_l(i) \cdot \text{MMD}_{t-i}^2 \quad (2.18)$$

Drift candidate được xác định khi tín hiệu shape đổi dấu (từ dương sang âm hoặc ngược lại):

$$\text{Candidate}(t) = \text{sign}(\text{shape}_t) \neq \text{sign}(\text{shape}_{t-1}) \quad (2.19)$$

Sau bước shape detection, mỗi drift candidate được xác thực bằng permutation test [19] để loại bỏ false positive:

1. Tính $\text{MMD}_{\text{obs}}^2$ từ dữ liệu gốc tại vị trí candidate

2. Lặp N_{perm} lần (thường 1000-5000):

(a) Hoán vị ngẫu nhiên nhãn của hai nửa cửa sổ

(b) Tính $\text{MMD}_{\text{perm}}^2$ với dữ liệu hoán vị

3. Tính p-value:

$$p\text{-value} = \frac{|\{\text{MMD}_{\text{perm}}^2 \geq \text{MMD}_{\text{obs}}^2\}|}{N_{\text{perm}}} \quad (2.20)$$

4. Nếu $p\text{-value} < \alpha$ (thường $\alpha = 0.05$), chấp nhận drift

Nếu không có drift thực sự, việc hoán vị nhãn không nên thay đổi nhiều MMD^2 . Nếu có drift, $\text{MMD}_{\text{obs}}^2$ sẽ lớn hơn đáng kể so với các giá trị permutation.

Permutation test là bước tốn thời gian nhất: $O(N_{\text{perm}} \cdot l_1^2)$. Tuy nhiên, vì chỉ áp dụng cho các candidate (không phải mọi thời điểm), chi phí trung bình vẫn chấp nhận được.

Hạn chế là số lần resampling N_{perm} thường phải lấy vài nghìn để p-value có độ phân giải tốt, và mỗi lần phải tính lại kernel matrix. Chi phí kiểm định vì vậy lớn hơn nhiều lần so với chi phí tính MMD thô. Độ chính xác thống kê của ngưỡng phát hiện cũng chỉ đảm bảo nếu phân phối dưới giả thuyết không trôi dạt được ước lượng phù hợp.

Hạn chế này là động lực cho IDW-MMD ở Chương 4: thay permutation test (Thuật toán 1) bằng p-value dựa trên xấp xỉ Gamma, đồng thời dùng trọng số nghịch biến mật độ để nhấn mạnh vùng biên phân phối. Phương pháp lấy cảm hứng từ “Optimally-Weighted MMD” của Bharti et al. [20], nhưng dùng heuristic đơn giản hơn để phù hợp với dữ liệu luồng.

Algorithm 1 Shape-based Drift Detector (ShapeDD)**Require:** Stream S , window size l_1 , bandwidth γ , significance level α **Ensure:** Set of detected drift points D

```

1:  $D \leftarrow \emptyset$ 
2: Initialize sliding window  $W$  of size  $2l_1$ 
3: for each new sample  $x_t$  in  $S$  do
4:   Update  $W$  with  $x_t$ 
5:   if  $|W| = 2l_1$  then
6:      $\gamma \leftarrow \text{MedianHeuristic}(W)$  ▷ estimate kernel bandwidth
7:      $K_{ij} \leftarrow \exp(-\gamma \|x_i - x_j\|^2)$  for all  $i, j$  ▷ compute kernel matrix
8:     Define contrast weights  $w(t)$  and compute  $\text{MMD}_t^2 \leftarrow \sum_{i,j} w_i w_j K_{ij}$ 
9:      $\text{shape}_t \leftarrow (h' * \text{MMD}^2)(t)$  ▷ convolve with derivative of smoothing kernel
10:    if  $\text{sign}(\text{shape}_t) \neq \text{sign}(\text{shape}_{t-1})$  then ▷ sign change  $\Rightarrow$  drift candidate
11:       $t_c \leftarrow t$ 
12:      Compute p-value via  $N_{\text{perm}}$  permutations of the kernel matrix
13:      if p-value  $< \alpha$  then
14:         $D \leftarrow D \cup \{t_c\}$  ▷ drift confirmed at  $t_c$ 
15:      end if
16:    end if
17:  end if
18: end for return  $D$ 

```

Ưu điểm nổi bật của ShapeDD

So với các phương pháp phát hiện drift truyền thống, điểm mạnh chính của ShapeDD là khai thác hình dạng tín hiệu thay vì phản ứng trực tiếp với mọi dao động của MMD. Cơ chế lọc shape giúp giảm nhiễu thống kê và giảm báo động giả, vì chỉ những biến động có dạng phù hợp với drift thật mới được giữ lại. Nhờ khớp dạng tam giác, ShapeDD cũng định vị được điểm thay đổi gần với vị trí drift thực, trong khi nhiều phương pháp cửa sổ đôi chỉ báo rằng drift đã xảy ra trong một khoảng thời gian.

Trong điều kiện phù hợp, đặc biệt là sudden drift với $P(X)$ thay đổi rõ, ShapeDD đạt hiệu năng phát hiện cao theo báo cáo gốc [4]. Phương pháp cũng có thể kết hợp nhiều độ dài cửa sổ và nhiều độ đo để tăng độ bao phủ. Về tính toán, phần convolution/shape detection có chi phí $O(l)$, còn kernel matrix có chi phí $O(l^2)$ cho mỗi cửa sổ; chi phí lớn nhất trong bản gốc nằm ở bước permutation test.

Hạn chế và điều kiện áp dụng hiệu quả

Bên cạnh ưu điểm, ShapeDD phù hợp nhất với các thay đổi rời rạc và đủ rõ. Lý thuyết hình dạng giả định mỗi khoảng thời gian chỉ có tối đa một sự kiện drift nổi bật; nếu các drift xảy ra liên tục hoặc quá gần nhau, các tam giác có thể chồng lấn và bộ lọc shape không còn nhận dạng đúng. Và vì thiết kế tập trung vào abrupt drift, hiệu suất có thể giảm khi đối mặt với gradual hoặc incremental drift, nơi tín hiệu trôi dạt không tạo ra hình tam giác rõ ràng.

Hiệu quả của ShapeDD cũng phụ thuộc mạnh vào độ dài của số l . Cửa sổ quá nhỏ làm ước lượng $\hat{\sigma}(t)$ nhiễu, còn cửa sổ quá lớn làm mờ hiệu ứng drift và tăng độ trễ. Việc kết hợp nhiều l giúp giảm rủi ro bỏ sót nhưng tăng chi phí và độ phức tạp triển khai. Ngoài ra, các ngưỡng kiểm định và biên độ vẫn cần hiệu chỉnh; đặt quá chặt sẽ giảm recall, đặt quá lỏng sẽ tăng báo động giả.

Trong dữ liệu nhiều chiều phức tạp, nếu drift chỉ ảnh hưởng một nhóm thuộc tính nhỏ, khoảng cách MMD toàn cục có thể yếu. Khi đó, ShapeDD có thể cần kết hợp với lựa chọn đặc trưng hoặc kiểm định theo từng chiều. Những hạn chế này là lý do Chương 4 cải tiến bước kiểm định bằng IDW-MMD và tách tín hiệu phát hiện khỏi tín hiệu phân loại.

CHƯƠNG 3

CÔNG TRÌNH LIÊN QUAN

Tóm tắt: Chương này trình bày tổng quan các phương pháp phát hiện hiện tượng trôi dạt đã và đang được nghiên cứu, đồng thời giới thiệu cách phân loại trôi dạt dựa trên cửa sổ trượt. Các phương pháp được trình bày theo nhiều góc nhìn khác nhau để so sánh, đánh giá và làm tiền đề cho việc phát triển mô hình phát hiện trôi dạt của luận văn.

3.1 Các phương pháp phát hiện trôi dạt

Trải qua nhiều thập kỷ, bài toán phát hiện hiện tượng trôi dạt đã được nghiên cứu với nhiều cách tiếp cận khác nhau. Từ những năm 1990, các hệ thống học máy đầu tiên có khả năng thích ứng với concept drift chủ yếu dựa trên chiến lược cửa sổ trượt hoặc bỏ qua mẫu cũ để dần thích nghi với dữ liệu mới. Giai đoạn này chưa có phương pháp “phát hiện” drift tách rời, mà các mô hình thường tự điều chỉnh dựa trên độ lỗi hoặc trọng số mẫu theo thời gian.

Sự thay đổi diễn ra vào giữa những năm 2000 khi các thuật toán chuyên biệt để phát hiện điểm thay đổi bắt đầu ra đời. Các phương pháp phát hiện trôi dạt có thể chia thành bốn nhóm chính theo kỹ thuật cốt lõi: (1) nhóm phương pháp thống kê (phân tích thay đổi phân phối dữ liệu), (2) nhóm phương pháp theo dõi hiệu năng mô hình (dựa trên sai số hoặc độ chính xác), (3) nhóm phương pháp học máy truyền thống (ví dụ dùng ensemble hoặc mô hình phụ để nhận biết drift), và (4) nhóm phương pháp học sâu (deep learning).

3.1.1 Các phương pháp thống kê

Nhóm phương pháp thống kê tập trung vào việc phát hiện thay đổi phân phối dữ liệu một cách trực tiếp, thường thông qua các phép kiểm định thống kê hoặc quan sát

cửa sổ trượt trên dòng dữ liệu. Một trong những kỹ thuật kinh điển là sử dụng các biểu đồ kiểm soát thống kê liên tục như *CUSUM* [21] hay kiểm định *Page-Hinkley* (*PH*).

Chẳng hạn, kiểm định *PH* [21, 22] được áp dụng trong bối cảnh concept drift để giám sát trung bình của một chuỗi số liệu và phát hiện khi nào trung bình thay đổi đáng kể. *PH* thực chất triển khai biểu đồ *CUSUM* để nhạy với những thay đổi nhỏ trong giá trị trung bình và có thể phát hiện cả xu hướng tăng lẫn giảm. Tương tự, các kiểm định thống kê hai mẫu (như Kolmogorov–Smirnov, Student t-test) cũng được tận dụng để so sánh phân phối của hai cửa sổ dữ liệu (mới và cũ) nhằm tìm ra sự khác biệt có ý nghĩa.

Một dấu mốc quan trọng trong nhóm này là phương pháp **ADWIN** (Adaptive Windowing) do Bifet và Gavalda đề xuất năm 2007 [18]. **ADWIN** sử dụng cửa sổ trượt có kích thước thay đổi một cách thích ứng: nó duy trì một cửa sổ dữ liệu với độ dài biến thiên sao cho giả thiết “không có sự thay đổi” được đảm bảo bên trong cửa sổ đó.

Tại mỗi thời điểm, **ADWIN** tách cửa sổ hiện tại thành hai phần W_0 và W_1 liên tiếp rồi so sánh sự khác biệt về giá trị trung bình giữa W_0 và W_1 . Nếu chênh lệch trung bình vượt quá một ngưỡng thống kê (xác định dựa trên khoảng tin cậy với mức ý nghĩa δ đã chọn), thuật toán kết luận rằng phân phối dữ liệu đã thay đổi và kích hoạt báo động drift.

ADWIN có giới hạn lý thuyết trên tỉ lệ báo động giả và tự động điều chỉnh kích thước cửa sổ thay vì yêu cầu người dùng cố định trước, nhờ đó phát hiện được cả drift đột ngột lẫn dần dần và là nền tảng cho nhiều kỹ thuật nâng cao sau này. Hai nhược điểm chính là chi phí tính toán cao do phải kiểm tra nhiều kích thước cửa sổ, và độ nhạy với nhiễu trong dữ liệu [3].

Một phương pháp khác là **KSWIN** (Kolmogorov–Smirnov WINdowing [23]), kết hợp kiểm định Kolmogorov–Smirnov với cửa sổ trượt nhằm phát hiện thay đổi phân phối mà không cần giả định trước về dạng phân phối. Các phương pháp thống kê thuần túy này có ưu điểm là không phụ thuộc vào mô hình học máy cụ thể, do đó có thể áp dụng trong môi trường phi giám sát (không cần nhãn). Mặt khác, chúng thường đòi hỏi giả định mạnh về độc lập và phân phối dữ liệu, và có thể bỏ sót những thay đổi nhỏ nếu kích thước mẫu không đủ lớn.

Dù vậy, nhóm phương pháp thống kê đã đặt nền móng quan trọng cho lĩnh vực phát hiện drift, đặc biệt cho các bài toán quan tâm trực tiếp đến sự thay đổi của dữ liệu hơn là hiệu năng mô hình.

3.1.2 Các phương pháp dựa trên hiệu năng mô hình

Đây là nhóm phương pháp phổ biến trong giai đoạn giữa những năm 2000, khi các nhà nghiên cứu tập trung vào việc giám sát chất lượng dự đoán của mô hình học máy theo thời gian để phát hiện drift. Ý tưởng chính là nếu độ chính xác của mô hình học máy đột nhiên trở nên kém đi (tỷ lệ lỗi tăng lên đáng kể khi so sánh với nhãn thật) thì có khả năng dữ liệu đã có thay đổi.

Phương pháp DDM (Drift Detection Method) ra đời năm 2004 [24], nhanh chóng trở thành nền tảng quan trọng và đặt nền móng cho các phương pháp sau này.

DDM (Drift Detection Method) theo dõi tỷ lệ lỗi p_i và độ lệch chuẩn s_i của mô hình trên dòng dữ liệu. Ý tưởng là trong quá trình hoạt động, nếu phân phối dữ liệu ổn định, tỷ lệ dự đoán sai sẽ giảm dần và dao động xung quanh một giá trị thấp. Khi concept drift xảy ra, tỷ lệ lỗi của mô hình đột ngột tăng cao, vượt ra khỏi khoảng tin cậy. DDM ghi nhận giá trị tối thiểu của $p_i + s_i$ trong quá trình học, và đặt hai ngưỡng cảnh báo:

- **Warning level:** Khi $p_i + s_i \geq p_{min} + 2 \cdot s_{min}$ (ngưỡng 2-sigma), hệ thống bắt đầu lưu dữ liệu mới để chuẩn bị huấn luyện lại
- **Drift level:** Khi $p_i + s_i \geq p_{min} + 3 \cdot s_{min}$ (ngưỡng 3-sigma), drift được xác nhận, mô hình cũ bị loại bỏ và huấn luyện lại từ dữ liệu đã lưu

DDM đơn giản, hiệu quả cho sudden drift, nhưng phản ứng chậm với gradual drift do chỉ dựa vào sự gia tăng đột biến của lỗi [24].

EDDM (Early Drift Detection Method) cải thiện khả năng phát hiện gradual drift bằng cách thay đổi metric theo dõi. Thay vì theo dõi trực tiếp tỷ lệ lỗi, EDDM tập trung vào giá trị khoảng cách giữa các lần dự đoán sai liên tiếp [25]. Ý tưởng là khi không có drift, các lỗi phân bố ngẫu nhiên với khoảng cách lớn, khi drift bắt đầu xuất hiện, các lỗi xuất hiện thường xuyên hơn. EDDM tính trung bình và độ lệch chuẩn của khoảng cách giữa các lỗi, phát hiện drift khi khoảng cách giảm mạnh so với giá trị tối đa đã ghi nhận. Nhờ vậy, EDDM nhạy hơn với các thay đổi từ từ, báo hiệu sớm hơn DDM trong nhiều trường hợp [25]. Sau DDM và EDDM, hàng loạt thuật toán dựa trên theo dõi hiệu năng ra đời, cho thấy sự phát triển đáng kể trong giai đoạn sau này.

HDDM (Hoeffding's Drift Detection Method) [26] là một hướng cải tiến quan trọng, áp dụng các công cụ thống kê chặt chẽ hơn để giảm giả định phân phối. Thay vì giả định rằng tỷ lệ lỗi tuân theo phân phối cụ thể (như DDM giả định phân phối nhị thức), HDDM sử dụng bất đẳng thức Hoeffding để xây dựng tiêu chí phát hiện drift mà không cần giả định về dạng phân phối. HDDM có hai biến thể chính: **HDDM_A**

sử dụng trung bình tỷ lệ lỗi với Hoeffding bound, và **HDDM_W** dùng tỷ lệ lỗi có trọng số, gán trọng số cao hơn cho dữ liệu gần đây.

Bất đẳng thức Hoeffding cho phép HDDM đặt ngưỡng phát hiện drift mà không cần giả định về phân phối của dữ liệu, làm cho phương pháp linh hoạt hơn trong các tình huống thực tế.

FHDDM (Fast HDDM) [27] được giới thiệu như phiên bản tăng tốc của HDDM, cải thiện cả tốc độ và độ nhạy trong việc phát hiện drift. FHDDM vẫn dựa trên bất đẳng thức Hoeffding nhưng tối ưu thuật toán tính toán để phù hợp với dòng dữ liệu tốc độ cao (high-speed data streams). Các cải tiến về mặt kỹ thuật giúp FHDDM phản ứng nhanh hơn với cả drift đột ngột lẫn drift dần dần, đồng thời giảm độ phức tạp tính toán so với HDDM gốc. **FHDDMS (Fast HDDM with Multiple Sliding Windows)** mở rộng FHDDM bằng cách duy trì hai cửa sổ trượt kích thước khác nhau: *cửa sổ ngắn* nhạy với thay đổi cục bộ nên bắt sudden drift nhanh, còn *cửa sổ dài* có nhiều dữ liệu hơn để ước lượng ổn định nên giảm false negative cho gradual drift.

FHDDMS liên tục so sánh giá trị trung bình hiện tại với giá trị trung bình tối đa đã quan sát trong mỗi cửa sổ. Drift được báo hiệu nếu sự khác biệt của *bất kỳ* cửa sổ nào vượt quá ngưỡng Hoeffding tương ứng. Thiết kế dual-window này cho phép FHDDMS tận dụng được ưu điểm của cả hai kích thước cửa sổ, cân bằng giữa độ nhạy (sensitivity) và độ tin cậy (reliability).

MDDM (McDiarmid's Drift Detection Method) [28] là phương pháp áp dụng bất đẳng thức McDiarmid – một công cụ thống kê dạng concentration inequality – để phát hiện những thay đổi đáng kể trong phân phối dữ liệu. Điểm đặc biệt của MDDM là việc áp dụng các lược đồ trọng số (weighting schemes) cho các phần tử trong cửa sổ trượt:

- **MDDM-A (Arithmetic):** Trọng số tuyến tính $w_i = i$
- **MDDM-G (Geometric):** Trọng số hình học $w_i = r^{i-1}$ với $r > 1$
- **MDDM-E (Euler):** Trọng số theo hàm mũ $w_i = e^{(i-1)/c}$

Các lược đồ trọng số này gán trọng số cao hơn cho các mẫu gần đây, giúp MDDM phản ứng nhanh hơn với thay đổi mới nhất trong dòng dữ liệu.

RDDM (Reactive DDM) [29] tập trung vào việc tối ưu thời gian phản ứng bằng cách rút ngắn giai đoạn cảnh báo (warning phase) của DDM gốc. RDDM điều chỉnh các ngưỡng để phát hiện nhanh hơn các thay đổi đột ngột, đồng thời vẫn duy trì khả năng xử lý drift lặp lại (recurrent drift). Cải tiến này đặc biệt hữu ích trong các ứng dụng yêu cầu phản ứng thời gian thực.

Những năm gần đây còn xuất hiện các biến thể hướng đến các tình huống đặc thù (xem Bảng 3.1 để so sánh tổng hợp):

- **ACDDM** [30] (Accurate DDM, 2020): Cải thiện độ chính xác cho drift lặp lại bằng cách lưu trữ và tái sử dụng các concept cũ.
- **DMDDM** [31] (Diversity-Measure DDM, 2020): Tận dụng tính đa dạng (diversity) của ensemble classifier để tăng độ nhạy phát hiện drift.
- **DDM-FPW** [32] (2020): Kiểm soát tỷ lệ dương tính giả trong luồng dữ liệu IoT đa nhãn.

Mục tiêu chung của các cải tiến này là tăng độ nhạy với nhiều loại drift, đồng thời giảm báo động giả và phân biệt được drift thực sự với nhiễu.

Bảng 3.1 Các phương pháp DDM-family

Phương pháp	Năm	Cải tiến chính	Điểm mạnh
DDM [24]	2004	Theo dõi error rate	Đơn giản, phát hiện sudden drift tốt
EDDM [25]	2006	Khoảng cách giữa lỗi	Phát hiện gradual drift sớm hơn
HDDM [26]	2015	Hoeffding bound	Không giả định dạng phân phối cụ thể
FHDDM [27]	2016	Tối ưu tốc độ	Nhanh cho high-speed stream
FHDDMS [27]	2016	Dual windows	Cân bằng sudden/gradual detection
MDDM [28]	2018	McDiarmid + weights	Nhấn mạnh dữ liệu gần đây
RDDM [29]	2017	Rút ngắn warning	Phản ứng nhanh với sudden drift

Nhóm phương pháp dựa trên hiệu năng mô hình bắt đầu từ DDM [24] và đã được mở rộng thành nhiều biến thể cân bằng hơn giữa tốc độ, độ chính xác và độ tin cậy.

3.1.3 Các phương pháp học máy truyền thống

Bên cạnh việc dùng trực tiếp thống kê hoặc sai số mô hình, các nhà nghiên cứu còn phát triển những kỹ thuật phát hiện drift dựa trên mô hình học máy hoặc tổ hợp nhiều mô hình. Những phương pháp này thường có tính “chủ động” hơn: thay vì chờ hiệu năng giảm, chúng tìm dấu hiệu thay đổi trong cấu trúc dữ liệu hoặc trong phản ứng của nhiều mô hình khác nhau.

Paired Learners [33] là một ví dụ tiêu biểu, sử dụng hai mô hình học song song trên dòng dữ liệu: một mô hình “ổn định” (stable learner) được huấn luyện trên toàn bộ dữ liệu tích lũy (dài hạn), và một mô hình “nhanh” (reactive learner) liên tục huấn luyện trên cửa sổ ngắn hạn gần đây. Khi có concept drift, mô hình nhanh sẽ thích nghi sớm và phân phối dự đoán của nó sẽ khác biệt rõ so với mô hình ổn định. Paired Learners theo dõi mức độ bất đồng (disagreement) giữa hai mô hình này: nếu sai khác vượt ngưỡng, ta kết luận đã có drift. Cách tiếp cận này cho phép phát hiện cả drift đột

ngột (mô hình nhanh thay đổi tức thời) lẫn drift dần dần (mô hình nhanh dần dần thay đổi so với mô hình cũ).

Một hướng khác là tận dụng **ensemble learning**, dùng nhiều mô hình phân loại chạy song song để nhận biết concept drift thông qua sự đa dạng (diversity) của ensemble. Phương pháp **ACE** (Adaptive Classifier Ensemble [34], 2007) duy trì một tập phân loại và liên tục đánh giá mỗi mô hình trên luồng dữ liệu; mô hình suy giảm sẽ bị thay thế bởi mô hình mới huấn luyện trên dữ liệu gần đây, và chính việc thay thế này là tín hiệu drift.

Learn++.NSE [35] mở rộng theo hướng tương tự: gán trọng số thời gian cho từng classifier theo độ tuổi, để classifier mới học concept mới còn classifier cũ dần mờ nhạt. Cả hai phương pháp phát hiện drift gián tiếp qua thay đổi cấu trúc hoặc thành phần ensemble.

Ngoài ra, một số phương pháp sử dụng **Classifier-based Drift Detection**: huấn luyện một mô hình phân loại nhằm phân biệt “dữ liệu cũ” và “dữ liệu mới”; nếu mô hình này đạt độ chính xác cao nghĩa là hai phân phối khác nhau rõ (drift lớn). Trong môi trường unsupervised, các phương pháp **one-class** như OCDD (One-Class Drift Detector) [36] dùng one-class SVM hoặc autoencoder để mô tả dữ liệu và khi reconstruction error tăng cao, đó là dấu hiệu drift.

Nhóm phương pháp này phát triển mạnh khoảng 2008–2015, khi khả năng tính toán cho phép triển khai nhiều mô hình song song. Chúng tận dụng sức mạnh của ensemble và mô hình meta để phát hiện các thay đổi tinh vi, tạo bước chuyển từ các kỹ thuật thuần thống kê sang các hệ thống linh hoạt, kết hợp chặt chẽ quá trình phát hiện drift với quá trình học liên tục.

3.1.4 Các phương pháp học sâu

Với sự bùng nổ của deep learning, bài toán concept drift được nghiên cứu trong bối cảnh các mô hình mạng neural phức tạp. Thách thức đặt ra là các mô hình deep learning thường có hàng triệu tham số và quá trình học phức tạp, làm sao tích hợp hoặc thiết kế bộ phát hiện drift hiệu quả? Các hướng tiếp cận chính bao gồm: (I) tích hợp các detector truyền thống để giám sát đầu ra hoặc đặc trưng trung gian của mạng deep; (II) thiết kế kiến trúc mạng có khả năng tự nhận biết drift và điều chỉnh cấu trúc; (III) sử dụng chính các mô hình deep để trực tiếp phân tích sự thay đổi phân phối trong dữ liệu.

Hướng (I): External detectors. Cách tiếp cận phổ biến là gắn các bộ phát hiện drift cổ điển vào pipeline của mô hình deep learning. Ví dụ, trong giám sát mô hình CNN phân loại ảnh, có thể theo dõi độ bất định (entropy) của phân phối dự đoán, khi

entropy tăng đột biến (mô hình trở nên kém chắc chắn), một bộ phát hiện như ADWIN được kích hoạt để dò tìm thay đổi trong chuỗi entropy theo thời gian. Jourdan và cộng sự [1] đã áp dụng thành công cách này để phát hiện drift trong hệ thống giám sát sản xuất công nghiệp: sử dụng ADWIN theo dõi độ lệch xác suất dự đoán của mạng CNN, khi ADWIN báo drift thì hệ thống kích hoạt huấn luyện lại mô hình trên dữ liệu mới.

Tương tự, trong mô hình phát hiện bất thường dùng LSTM, các nghiên cứu trước đây cũng nhúng các detector như DDM hoặc Page-Hinkley để liên tục canh chừng lỗi dự đoán theo thời gian. Việc tích hợp này khai thác tính chất “model-agnostic” của các thuật toán drift detection truyền thống, biến chúng thành cảm biến giám sát bên ngoài cho các mô hình deep phức tạp.

Hướng (II): Self-adaptive architectures. Các mạng neural tự điều chỉnh cấu trúc khi drift xảy ra đang trở thành một hướng nghiên cứu được quan tâm. **NADINE** [37] là một mạng MLP nhiều lớp có khả năng tiến hóa cấu trúc: NADINE tích hợp cơ chế phát hiện drift chủ động dựa trên cửa sổ trượt thích ứng kết hợp bất đẳng thức Hoeffding (tương tự ý tưởng của FHDDM) để liên tục kiểm tra dữ liệu mới. Mỗi khi detector phát hiện tín hiệu drift, NADINE sẽ “tiến hóa” bằng cách thêm hoặc bớt các nút ẩn và tầng ẩn tương ứng để thích nghi với concept mới. Cơ chế này giúp mạng tự điều chỉnh độ phức tạp: tăng nếu concept mới phức tạp hơn, hoặc giảm nếu cần tránh overfitting.

Hướng (III): Intrinsic detection. Khai thác trực tiếp năng lực biểu diễn của mô hình deep để nhận biết drift. Ví dụ, một autoencoder có thể học nén dữ liệu đầu vào; nếu phân phối dữ liệu thay đổi, lỗi tái tạo (reconstruction error) của autoencoder sẽ tăng cao. **AEDetect** [38] giám sát phân phối lỗi tái tạo của autoencoder, khi phân phối này dịch chuyển nhiều thì phát tín hiệu drift. Trong lĩnh vực chuỗi thời gian, **LSTM-NDT** (Neural Drift Detector dùng LSTM) [2] học mô hình chuỗi và dùng sai số dự báo của LSTM làm tiêu chí phát hiện drift. Các phương pháp này tận dụng được thông tin trong dữ liệu nhờ đặc trưng học được của network, nhưng khó tách bạch giữa “drift” và “nhiều” bên trong mô hình.

Bảng 3.2 Tóm tắt một số mốc chính trong phát triển phương pháp phát hiện concept drift

Năm	Phương pháp	Đặc điểm chính
1996	FLORA [39]	Cửa sổ trượt thích ứng đầu tiên, xử lý drift bằng chọn bộ nhớ ngắn hạn.
2004	DDM [24]	Giám sát tỉ lệ lỗi mô hình, đặt ngưỡng cảnh báo và drift theo luật SPC; phát hiện drift đột ngột hiệu quả.
2006	EDDM [25]	Mở rộng DDM, cải thiện phát hiện drift dần dần bằng cách theo dõi khoảng cách giữa các lỗi.
2007	ADWIN [18]	Cửa sổ trượt thích ứng với đảm bảo lý thuyết, so sánh thống kê hai cửa sổ con để phát hiện thay đổi phân phối.
2015	HDDM [26]	Sử dụng bất đẳng thức Hoeffding (không giả định phân phối) để phát hiện drift trên dòng lỗi; giảm giả định so với DDM.
2016	FHDDM [27]	Phiên bản nhanh của HDDM, tăng tốc độ và độ nhạy khi drift (đột ngột và dần dần).
2017	RDDM [29]	Phản ứng nhanh với drift đột ngột, rút ngắn giai đoạn cảnh báo của DDM; phù hợp cả drift lặp lại.
2020	DAWIDD [40]	Phương pháp phi tham số dùng kiểm định độc lập (independence test) thay vì mô hình dự đoán; linh hoạt với nhiều dạng drift khác nhau.
2019–22	Học sâu thích ứng	Xuất hiện các mô hình deep learning tích hợp phát hiện drift (NADINE [37], ADL [41], DEV DAN [42],...); mạng tự mở rộng hoặc dùng detector gắn ngoài để duy trì hiệu năng dưới concept drift.

Các phương pháp cũng đã được thực nghiệm trên các tập dữ liệu khác nhau và thực tế chuẩn để đánh giá hiệu năng. Bảng 3.4 tổng hợp kết quả từ các nghiên cứu. *Lưu ý: các số liệu được tổng hợp từ nhiều bài báo độc lập với thiết lập thực nghiệm khác nhau (mô hình cơ sở, cách chia dữ liệu, số lần chạy), do đó không nên so sánh trực tiếp giữa các hàng mà chỉ nên xem như tham khảo định hướng.* Bảng 3.3 phân loại các bộ dò theo nhóm chiến lược và cơ chế cốt lõi.

Bảng 3.3 Phân loại và đặc tính kỹ thuật của các bộ dò Concept Drift tiêu biểu

Thuật toán	Nhóm chiến lược	Cơ chế cốt lõi	Ưu điểm	Hạn chế
DDM	Statistical Process Control	Binomial Distribution	Ít tốn bộ nhớ, chạy nhanh [24]	Hiệu suất giảm khi cửa sổ lớn [3]
EDDM	Statistical Process Control	Distance between errors	Nhạy với gradual drift [25]	Dễ báo động giả lúc mới học [8]
ADWIN	Sliding Window	Adaptive Windowing	Có đảm bảo lý thuyết (theoretical bounds) về tỷ lệ FP [18]	Tốn bộ nhớ hơn DDM/EDDM [3]
HDDM	Sliding Window	Hoeffding's Bound	Phù hợp cho abrupt/gradual drift [26]	Cần dữ liệu nhãn liên tục [8]
FHDDM	Sliding Window	Hoeffding + Sliding Win.	Chỉ duy trì 2 biến số, cực nhẹ [27]	Cần chọn kích thước cửa sổ phù hợp [3]
D3	Virtual Classifier	Unsupervised Separation	Tốc độ nhanh, không cần nhãn [43]	Chỉ phát hiện Covariate Shift (thay đổi $P(X)$) [3]
ShapeDD	Meta-Statistic	Shape-based Filtering	Định vị điểm drift chính xác [4]	Độ trễ phụ thuộc kích thước cửa sổ [4]
DAWIDD	Block-Based	Independence Test (HSIC — Hilbert-Schmidt Independence Criterion)	Phù hợp với gradual/reoccurring drift [40]	Tỷ lệ báo động giả khá cao [3]

Số lượng công trình kết hợp giữa deep learning và phát hiện concept drift đã tăng nhanh trong những năm gần đây. Ban đầu, các mô hình deep thường dựa vào các “cảm biến” drift bên ngoài (như ADWIN, DDM) để quyết định khi nào cần tái huấn luyện. Sau đó, các kiến trúc deep thích ứng (NADINE [37], DEN [44], ADL [41], DEVDAN [42]) ra đời với khả năng tự điều chỉnh cấu trúc khi drift xảy ra. Các phương pháp này vẫn đang phát triển, nhưng hứa hẹn giúp mô hình học sâu vận hành ổn định trong môi trường dữ liệu biến động.

Bảng 3.4 Tổng hợp hiệu năng phát hiện drift từ các nghiên cứu gốc

Thuật toán	Accuracy (%)				Delay		β -score	
	Elec2	Cov.	Hyp.	SEA	Elec2	Hyp.	Elec2	Hyp.
D3	86.69	87.17	85.29	88.08	—	—	—	—
ADWIN	81.33	80.48	87.28	77.59	—	—	—	—
DDM	79.30	83.36	84.01	85.32	—	—	—	—
EDDM	78.25	82.76	80.27	75.73	—	—	—	—
HDDM-A	85.71	87.24	86.91	—	—	—	—	—
FHDDM	73.62 [†]	67.76 [†]	85.08	—	—	—	—	—
DAWIDD	—	—	—	—	21.00	37.18	—	—
ShapeDD	—	—	—	—	—	—	87.73	96.00

Ghi chú: Các chỉ số được tổng hợp từ các nghiên cứu gốc.

[†] Tính toán từ tỷ lệ lỗi (Error-rate) của mô hình Hoeffding Tree.

Accuracy: Độ chính xác tích lũy (Prequential) của mô hình kết hợp với bộ phát hiện drift.

Delay: Số mẫu trung bình từ khi drift xảy ra đến khi được phát hiện (thấp = tốt).

β -score: Chỉ số đánh giá chất lượng phát hiện, cân bằng Precision và Recall (cao = tốt).

Như bảng 3.2 tóm tắt, lĩnh vực phát hiện concept drift đã đi từ các ý tưởng của số trượt và giám sát sai số mô hình sang các kiểm định thống kê, ensemble, meta-

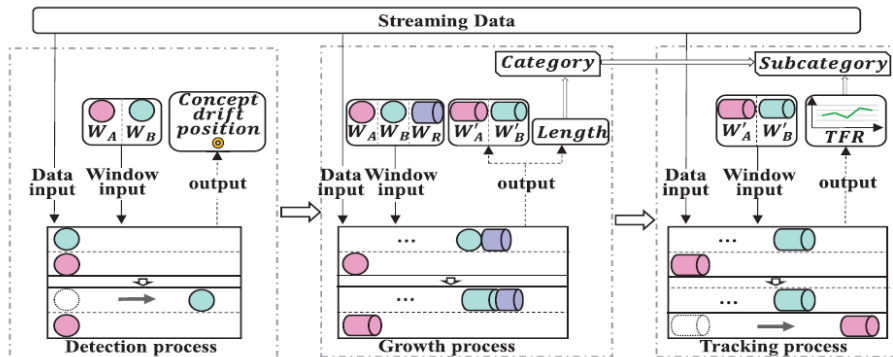
learning và gần đây là học sâu [8]. Xu hướng chung là giảm phụ thuộc vào giả định cứng, xử lý được nhiều dạng drift hơn, và chuẩn bị cho các hệ thống vừa phát hiện vừa thích ứng với dữ liệu biến đổi.

3.2 Phương pháp phân loại drift CDT-MSW

CDT-MSW [5] là phương pháp *có giám sát* phân loại loại drift dựa trên hành vi của độ chính xác mô hình trên nhiều cửa sổ trượt (Hình 3.1). Đây là nguồn cảm hứng trực tiếp cho SE-CDT (Chương 4); luận văn kế thừa hai ý chính sau và thay thế phần lõi tín hiệu, chi tiết kỹ thuật của ba giai đoạn xem trong bài báo gốc.

Phân nhóm TCD vs PCD theo độ dài trôi dạt: CDT-MSW dùng phương sai độ chính xác trên cửa sổ phụ W_R để ước lượng độ dài m của sự trôi dạt. Thủ tục ước lượng độ dài này được bài báo gốc gọi là **Growth process** (Algorithm 2 trong [5]): sau khi phát hiện được vị trí drift, cửa sổ phụ được mở rộng dần cho tới khi phương sai độ chính xác trở lại mức ổn định, và số bước mở rộng chính là m . Thuật ngữ “Growth process” được dùng lại ở Chương 4 và Chương 6 với đúng nghĩa này. Khi độ dài $m = 1$ (phân phối ổn định ngay lập tức), drift thuộc nhóm **Transient Concept Drift (TCD)** bao gồm Sudden, Blip và Recurrent. Khi $m > 1$ (chuyển tiếp dần), drift thuộc nhóm **Progressive Concept Drift (PCD)** bao gồm Incremental và Gradual. Hai nhóm này được phân biệt tiếp ở giai đoạn theo dõi bằng hình dáng của đường cong Tracking Flow Ratio (TFR) và Micro TFR (MTFR).

Hai mức đánh giá phân loại CAT và SUB: Phân loại được đánh giá ở hai mức: nhóm chính TCD/PCD (gọi tắt là CAT) và tiểu loại Sudden/Blip/Recurrent/Incremental/Gradual (gọi tắt là SUB). Hai thuật ngữ CAT/SUB tái sử dụng trong các thí nghiệm phân loại của luận văn (Mục 5.3).



Hình 3.1 Tổng quan ba giai đoạn của CDT-MSW: phát hiện vị trí trôi dạt, ước lượng độ dài, theo dõi và phân loại tiểu loại bằng TFR/MTFR.

Tuy nhiên, việc dùng độ chính xác mô hình làm tín hiệu khiến CDT-MSW phụ thuộc vào nhãn y , không khả dụng cho môi trường không giám sát của luận văn. SE-CDT thay tín hiệu tỉ lệ độ chính xác (accuracy ratio) bằng tín hiệu MMD đo trên cửa sổ trượt nhưng giữ lại cách phân nhóm CAT/SUB ở trên (Chương 4).

3.3 Các chiến lược cập nhật mô hình thích ứng

3.3.1 Tổng quan

Trong các hệ thống học trực tuyến (online learning systems), việc phát hiện trôi dạt chỉ là bước đầu tiên. Điều quan trọng hơn là cách **mô hình thích ứng** (model adaptation) được cập nhật sau khi trôi dạt xảy ra. Các chiến lược cập nhật mô hình nhằm duy trì độ chính xác, ổn định và tránh hiện tượng quên ngắn hạn (catastrophic forgetting) trong quá trình học liên tục.

Theo Guo *et al.* [5], thông tin về loại trôi dạt (drift type) do CDT-MSW cung cấp giữ vai trò quan trọng trong việc lựa chọn cơ chế cập nhật phù hợp. Các nghiên cứu gần đây [1] đã mở rộng khung thích ứng này sang hệ thống học sâu, trong đó chiến lược cập nhật phụ thuộc đồng thời vào loại trôi dạt, cấu trúc mô hình, mức độ sẵn có của nhãn và chi phí tính toán.

3.3.2 Phân loại chiến lược cập nhật

Tổng hợp từ các tài liệu trên và các khảo sát về học dưới concept drift [10, 11, 13], có thể chia các chiến lược cập nhật mô hình thành bốn nhóm chính:

- (a) Cập nhật cục bộ (Local Reset / Fine-tuning) phù hợp với *sudden drift*, khi phân phối dữ liệu thay đổi mạnh trong thời gian ngắn. Mô hình được tái huấn luyện một phần hoặc toàn bộ trên cửa sổ dữ liệu mới. Phương pháp này phù hợp với các bộ phát hiện trôi dạt dựa trên cửa sổ như ADWIN hoặc CDT-MSW, khi $L_d < \lambda$ (độ dài trôi dạt ngắn).

Ví dụ, trong hệ thống giám sát quá trình sản xuất (process monitoring), Jourdan *et al.* [1] thực hiện *partial reinitialization* của các tầng cuối trong mạng nơ-ron khi độ bất định (uncertainty) vượt ngưỡng.

- (b) Cập nhật dần (Gradual / Incremental Update) được dùng khi mô hình gặp *gradual* hoặc *incremental drift*. Vì sự thay đổi diễn ra từ từ, mô hình thường cần cập nhật mềm với hệ số học η_t nhỏ hơn. Một cách mô tả tổng quát là cập nhật tham số theo gradient trên dữ liệu mới với hệ số học hoặc trọng số quên phụ thuộc vào

tốc độ drift:

$$\theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} \mathcal{L}(x_t, y_t), \quad \text{với } \eta_t = f(L_d, r_t),$$

Trong đó L_d là độ dài trôi dạt, r_t là tốc độ thay đổi. Khi r_t tăng đều, η_t được tăng dần để theo kịp xu hướng dữ liệu mới. Các kỹ thuật như *online fine-tuning*, *stochastic gradient with forgetting factor* hoặc *momentum adaptation* thường được sử dụng trong nhóm này.

- (c) Cập nhật theo ngữ cảnh (Recurrent / Memory-based Adaptation) xử lý *recurrent drift*, khi khái niệm cũ tái xuất hiện theo chu kỳ. Mô hình nên duy trì một *bộ nhớ khái niệm* (concept repository), lưu các snapshot $\{M_i\}$ tương ứng với từng khái niệm và lựa chọn mô hình phù hợp nhất khi phân phối dữ liệu hiện tại $P_t(x)$ tương tự với một snapshot trước đó:

$$M^* = \arg \min_{M_i} \text{Dist}(P_t(x), P_{M_i}(x)).$$

Phương pháp này được áp dụng trong các framework học sâu có bộ nhớ động như Deep Ensemble Replay hoặc Continual Learning with Experience Buffer.

- (d) Cập nhật cấu trúc (Structural or Hybrid Adaptation) thay đổi cả tham số lẫn *cấu trúc mạng học sâu* khi có drift. Ví dụ, thêm tầng mới hoặc nhánh mới để học khái niệm mới mà không làm mất kiến thức cũ, biểu diễn bởi:

$$\mathcal{M}_{t+1} = \mathcal{M}_t \cup \Delta \mathcal{M},$$

Trong đó $\Delta \mathcal{M}$ là tập các node hoặc layer mới được thêm. Phương pháp này hữu ích với các trôi dạt dài hạn (progressive drift) hoặc trôi dạt phức hợp (hybrid drift).

Bảng 3.5 Mối quan hệ giữa loại trôi dạt và chiến lược cập nhật mô hình thích ứng

Loại trôi dạt	Chiến lược cập nhật đề xuất
Sudden drift	Tái huấn luyện nhanh (local reset), bỏ dữ liệu cũ; tăng η_t ; phù hợp với incremental retraining, ADWIN, CDT-MSW.
Gradual drift	Cập nhật mềm (soft update), giảm hệ số học; áp dụng momentum nhỏ hoặc sliding window dài.
Incremental drift	Cập nhật liên tục (continuous adaptation), sử dụng gradient online và drift-aware scheduler.
Recurrent drift	Duy trì bộ nhớ khái niệm; lựa chọn mô hình tương tự (ensemble selection / memory replay).
Progressive drift	Thay đổi cấu trúc mô hình (dynamic architecture), hoặc học chuyển giao (transfer learning) giữa các giai đoạn.

Các kết quả trên cho thấy thông tin về loại trôi dạt có thể giúp chọn chiến lược thích ứng phù hợp hơn, thay vì huấn luyện lại toàn bộ trong mọi trường hợp. Các hệ thống hiện đại thường kết hợp tín hiệu từ bộ phát hiện với đặc trưng động của mô hình để điều chỉnh learning rate, chuyển mô hình hoặc thay đổi cấu trúc khi cần. Đây là tiền đề cho khung thích ứng theo loại drift được xây dựng ở Chương 4.

3.4 Lý do lựa chọn ShapeDD làm nền tảng

Từ khảo sát ở các phần trên, có thể rút ra nhận xét chung để định hướng cho phần đề xuất của luận văn.

Các phương pháp theo dõi hiệu năng mô hình như DDM và EDDM hoạt động tốt nhưng cần nhãn dữ liệu (y) để đo lỗi. Yêu cầu này không khả thi trong môi trường sản xuất công nghiệp mà luận văn nhắm đến. Các phương pháp thống kê không cần nhãn như ADWIN hoặc KS-Test phù hợp hơn về mặt giám sát luồng, nhưng chưa trực tiếp cung cấp *vị trí chính xác* của điểm drift theo cách ShapeDD khai thác hình dạng tín hiệu. Các phương pháp kernel-based như MMD và DAWIDD gần hơn với yêu cầu, nhưng chi phí tính toán lớn và không có sẵn cơ chế phân loại loại drift.

ShapeDD phù hợp với mục tiêu của luận văn vì hoạt động không giám sát, định vị được điểm drift nhờ đặc trưng hình học tam giác (Triangle Shape Property), và có nền tảng lý thuyết đủ rõ để cải tiến có chủ đích. Lựa chọn này không được hiểu là khẳng định ShapeDD là phương pháp tốt nhất tuyệt đối. Thay vào đó, ShapeDD được chọn vì nó cung cấp đúng tín hiệu mà luận văn cần: phát hiện không cần nhãn, có localization theo thời gian, có trace hình dạng để mở rộng sang phân loại drift, và có thể so sánh thực nghiệm với các detector đại diện như DAWIDD, MMD, D3 và KS-Test. ShapeDD gốc vẫn phụ thuộc vào permutation test tốn kém và chưa có cơ

chế phân loại loại drift. Hai hạn chế này là điểm xuất phát cho các đóng góp được trình bày trong Chương 4.

Luận văn vì vậy đề xuất cải tiến ShapeDD theo hai hướng bổ sung cho nhau: dùng IDW-MMD với xấp xỉ Gamma cho bước kiểm định ứng viên drift, và xây dựng hệ thống SE-CDT tích hợp phát hiện và phân loại loại drift — trong đó module phân loại dựa trên hình dạng tín hiệu MMD mà không cần nhãn. Cải tiến cụ thể được trình bày ở Chương 4; lý thuyết nền của ShapeDD đã được giới thiệu ở Chương 2.

CHƯƠNG 4

HỆ THỐNG SE-CDT: PHÁT HIỆN, PHÂN LOẠI VÀ THÍCH ỨNG CONCEPT DRIFT

Tóm tắt: Trên cơ sở lý thuyết ShapeDD ở Chương 2 và khảo sát các phương pháp phát hiện – phân loại drift ở Chương 3, chương này trình bày hệ thống SE-CDT (ShapeDD-Enhanced Concept Drift Type) — một hệ thống phát hiện và phân loại drift tích hợp không giám sát. Bước phát hiện dùng ShapeDD-IDW (ShapeDD cải tiến bằng IDW-MMD với xấp xỉ Gamma); bước phân loại phân tích hình dạng tín hiệu MMD để xác định loại drift mà không cần nhãn. SE-CDT đóng vai trò tăng giám sát cho mô hình sản xuất, không thay thế mô hình dự đoán chính. Luận văn cũng trình bày khung thích ứng mô hình theo loại drift và kiến trúc xử lý luồng dữ liệu.

4.1 Động lực và tổng quan cách tiếp cận

ShapeDD gốc có nhiều ưu điểm về độ chính xác định vị vị trí xảy ra drift và khả năng khử nhiễu, nhưng khi triển khai trong các hệ thống thời gian thực quy mô lớn gặp ba hạn chế: chi phí permutation test cao, độ nhạy với kernel bandwidth và kích thước của số, và cần cơ chế thích ứng sau khi phát hiện drift [2] như một hệ thống hoàn chỉnh.

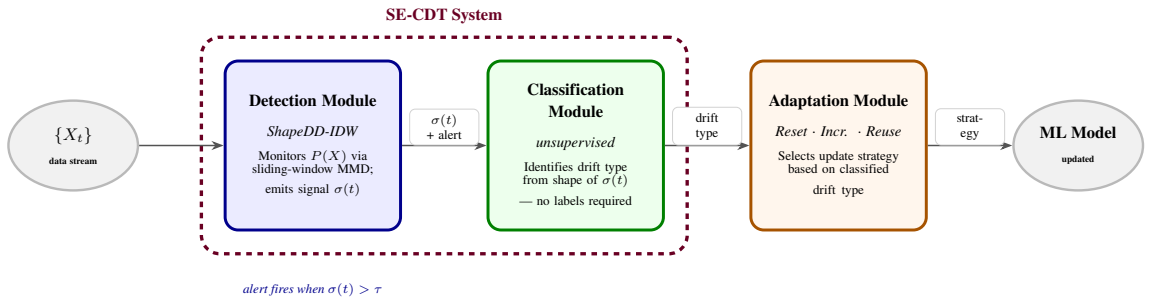
Luận văn xây dựng hệ thống SE-CDT để xử lý ba hạn chế này. Về thuật toán, ShapeDD-IDW kết hợp tín hiệu Standard MMD với bước kiểm định IDW-MMD (Inverse Density-Weighted MMD) dựa trên xấp xỉ Gamma [45] để thay cho permutation test tốn kém. Bảng thông kernel được xử lý bằng cách ước lượng γ một lần bằng

median heuristic và dùng chung cho toàn bộ tín hiệu trượt (Mục 4.2.1), thay vì để mỗi cửa sổ tự ước lượng; luận văn không tiến hành ablation riêng về độ nhạy với γ nên không đưa ra kết luận định lượng về điểm này. Về chiến lược, luận văn xây dựng khung thích ứng theo loại drift, dùng kết quả phân loại từ SE-CDT để kích hoạt cách cập nhật phù hợp (Reset, Incremental, hoặc Reuse) [10, 11]. Về kiến trúc, hệ thống được đề xuất triển khai trên nền tảng Apache Kafka cho phép xử lý luồng phân tán, độ trễ thấp và khả năng chịu lỗi.

Trong pipeline sản xuất, mô hình cần bảo vệ là mô hình nghiệp vụ như bộ phân loại chất lượng sản phẩm, dự đoán lỗi, hoặc bảo trì dự đoán. SE-CDT chỉ quan sát luồng đặc trưng đầu vào và sinh sự kiện drift gồm thời điểm, loại drift và tín hiệu hỗ trợ. Adaptation Manager mới là thành phần quyết định hành động tiếp theo: bỏ qua cảnh báo, yêu cầu xác nhận, huấn luyện lại, cập nhật online hoặc chuyển sang mô hình đã lưu. Phân tích này giúp tránh hiểu nhầm rằng SE-CDT là mô hình dự đoán chất lượng; SE-CDT là lớp monitoring và điều phối phản ứng trước khi mô hình sản xuất suy giảm.

4.1.1 Tổng quan kiến trúc ba module

Trước khi đi vào chi tiết từng thành phần, Hình 4.1 minh họa tổng quan luồng xử lý của hệ thống. Ba module được kết nối nối tiếp, đầu ra của module trước là đầu vào của module sau.



Hình 4.1 Kiến trúc tổng quan: Hệ thống SE-CDT (gồm Detection Module chạy ShapeDD-IDW và Classification Module phân tích hình dạng $\sigma(t)$) nhận luồng $\{X_t\}$ và sản xuất nhãn loại drift; Adaptation Module dùng nhãn này để chọn chiến lược cập nhật mô hình phù hợp.

- **Detection Module** (Mục 4.2.1–4.2.1): ShapeDD-IDW liên tục theo dõi luồng $\{X_t\}$ qua cửa sổ trượt, tính tín hiệu MMD $\sigma(t)$, và phát cảnh báo khi $\sigma(t)$ vượt ngưỡng. Đây là thành phần duy nhất tiếp xúc trực tiếp với dữ liệu thô.
- **Classification Module — SE-CDT** (Mục 4.3): Nhận tín hiệu Standard MMD $\sigma(t)$ từ bước phát hiện và phân tích hình dạng để xác định loại drift (TCD: sudden, blip, recurrent; hoặc PCD: gradual, incremental) mà không cần nhãn. Ở cấp

độ thuật toán, hai vai trò Detection và Classification đều được triển khai trong cùng một lớp SE-CDT; phân tách thành hai tiến trình độc lập chỉ áp dụng ở cấp độ triển khai Kafka (Mục 4.5).

- **Adaptation Module** (Mục 4.4): Dựa trên loại drift từ SE-CDT, chọn chiến lược cập nhật mô hình phù hợp (*Reset* cho sudden, *Incremental* cho gradual/incremental, hoặc *Reuse* cho recurrent).

Toàn bộ ba module được triển khai bằng mô phỏng, sau đó được triển khai trên Apache Kafka trong Mục 4.5, với mỗi module chạy như một tiến trình độc lập giao tiếp qua các topics riêng biệt. Hệ thống đề xuất được gọi là **SE-CDT** (*ShapeDD-Enhanced Concept Drift Type identification*). Tên này nhấn mạnh vào năng lực phân loại loại drift, vốn là đóng góp gốc của luận văn so với các phương pháp phát hiện thuần túy trước đây. Module phát hiện của SE-CDT được đặt tên riêng là **ShapeDD-IDW** vì nó là một cải tiến trực tiếp của ShapeDD [4] bằng IDW-MMD và xấp xỉ Gamma; cách đặt tên có gạch nối phản ánh quan hệ kế thừa với phương pháp gốc.

4.2 Đề xuất cải tiến phương pháp phát hiện drift: IDW-MMD với xấp xỉ Gamma

4.2.1 Inverse Density-Weighted MMD (IDW-MMD): Tối ưu hóa hiệu suất và độ chính xác

Vấn đề của ước lượng MMD truyền thống

Trong phương pháp gốc, thống kê MMD được tính toán dựa trên trọng số đều, coi vai trò của mọi điểm dữ liệu trong cửa sổ là như nhau. Nhắc lại từ Chương 2, công thức MMD có dạng:

$$\text{MMD}_{\text{uniform}}^2(X, Y) = \frac{1}{n^2} \sum_{i,j} k(x_i, x_j) + \frac{1}{m^2} \sum_{p,q} k(y_p, y_q) - \frac{2}{nm} \sum_{i,p} k(x_i, y_p) \quad (4.1)$$

trong đó các tổng chạy qua tất cả các cặp chỉ số, kể cả $i = j$. Với kernel RBF, thành phần tự đối sánh $k(x_i, x_i) = 1$, nghĩa là mỗi điểm đều đóng góp một giá trị bằng 1 vào tổng, làm giá trị MMD cao hơn thực tế một chút. Hạn chế này sẽ được khắc phục trong công thức IDW-MMD trình bày ngay dưới đây, bằng cách loại các cặp điểm trùng nhau khỏi phép tính.

Với cửa sổ nhỏ ($n, m < 200$), MMD dễ dao động và khó tách drift thật khỏi nhiễu mẫu. Bước permutation test của ShapeDD gốc cũng tốn thời gian vì phải lặp lại phép

tính MMD nhiều lần để ước lượng p-value. Một nguyên nhân khác là trọng số đều coi mọi điểm trong cửa sổ như nhau, trong khi các thay đổi đột ngột thường xuất hiện rõ hơn ở vùng biên phân phối.

Ý tưởng cải tiến bằng cách dùng trọng số thay đổi theo vị trí

Thay vì giữ trọng số đều, luận văn gán trọng số theo mật độ cục bộ: điểm ở vùng thưa được nhấn mạnh hơn, còn điểm ở vùng trung tâm được giảm bớt ảnh hưởng. Để hình thức hóa, dạng tổng quát của MMD có trọng số là:

$$\text{MMD}_{\text{weighted}}^2(X, Y) = \sum_{i,j} w_i w_j k(x_i, x_j) + \sum_{p,q} v_p v_q k(y_p, y_q) - 2 \sum_{i,p} w_i v_p k(x_i, y_p) \quad (4.2)$$

trong đó w_i và v_p là trọng số của từng điểm (thỏa mãn $\sum_i w_i = 1$, $\sum_p v_p = 1$). Câu hỏi đặt ra là: tính w_i như thế nào để phát hiện drift hiệu quả?

Giải pháp đề xuất: Trọng số nghịch biến mật độ (Inverse Density Weighting)

Luận văn đề xuất phương pháp **IDW-MMD** (Inverse Density-Weighted MMD), một cách gán trọng số đơn giản lấy cảm hứng từ ý tưởng weighted MMD trong Optimally-Weighted MMD của Bharti et al. [20].

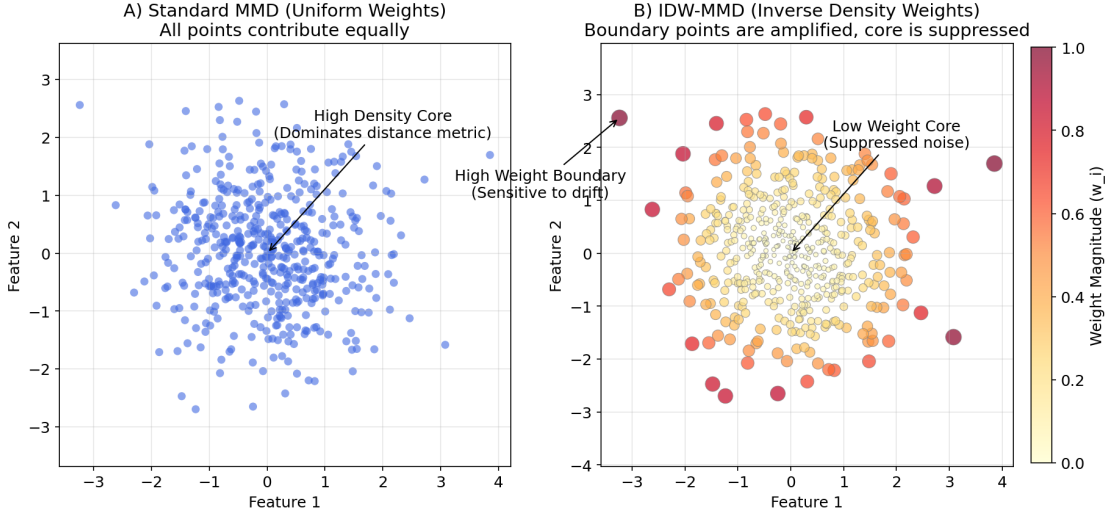
Phương pháp của Bharti et al. được thiết kế cho bài toán suy luận thống kê, đề xuất trọng số tối ưu phức tạp và không liên quan đến bài toán drift detection. Luận văn chỉ kế thừa ý tưởng “gán trọng số cho từng điểm dữ liệu trong MMD” và áp dụng một heuristic đơn giản hơn (trọng số nghịch biến với căn bậc hai của mật độ kernel) để phù hợp với bài toán drift detection.

Trọng số w_i được tính dựa trên mật độ cục bộ, chỉ dùng các cặp điểm khác nhau (off-diagonal) để tránh bias tự tương quan:

$$w_i \propto \frac{1}{\sqrt{\sum_{j \neq i} k(x_i, x_j)} + \epsilon} \quad (\text{Inverse Density Weighting}) \quad (4.3)$$

trong đó ϵ là một hằng số nhỏ (ví dụ $\epsilon = 0.5$) để đảm bảo tính ổn định số học (numerical stability).

Khi một điểm nằm ở vùng trung tâm, tổng mật độ $d(x_i)$ lớn và trọng số của điểm đó giảm. Ngược lại, điểm ở vùng biên có ít lân cận hơn nên nhận trọng số cao hơn (Hình 4.2). Luận văn dùng $1/\sqrt{d_i}$ thay cho $1/d_i$ để tránh khuếch đại quá mạnh các điểm biên đơn lẻ, vốn dễ bị nhiễu chi phối.



Hình 4.2 Cơ chế Trọng số Nghịch mật độ (IDW-MMD). Khác với MMD coi trọng mọi điểm như nhau (A), IDW-MMD (B) tự động nén trọng số của vùng lõi (core) và khuếch đại tín hiệu vùng biên (boundary) — nơi drift thường xảy ra đầu tiên.

Dựa trên phân tích trên, công thức IDW-MMD được xây dựng như sau. Đặt $d(x_i) = \sum_{j \neq i} k(x_i, x_j)$ là mật độ cục bộ của x_i , và $\tilde{w}_i = 1/(\sqrt{d(x_i)} + \epsilon)$ là trọng số thô. Ma trận trọng số W_{ij}^{XX} lấy tích $\tilde{w}_i \tilde{w}_j$ cho các cặp $i \neq j$ rồi chuẩn hóa để $\sum_{i \neq j} W_{ij}^{XX} = 1$ (đường chéo được loại trừ theo U-statistic style). Tương tự cho W_{pq}^{YY} trên cửa sổ Y . Công thức hoàn chỉnh:

$$\text{MMD}_{\text{IDW}}^2(X, Y) = \underbrace{\sum_{i \neq j} W_{ij}^{XX} k(x_i, x_j)}_{\text{Weighted } XX} + \underbrace{\sum_{p \neq q} W_{pq}^{YY} k(y_p, y_q)}_{\text{Weighted } YY} - 2 \underbrace{\sum_{i=1}^n \sum_{p=1}^m \frac{1}{nm} k(x_i, y_p)}_{\text{Uniform } XY} \quad (4.4)$$

Thành phần so sánh chéo giữa hai cửa sổ giữ trọng số đồng nhất $1/(nm)$ vì đây là số hạng đo sự chông chéo giữa hai phân phối. Nếu gán trọng số IDW cho cross-term, thống kê có thể tạo khoảng cách giả ngay cả khi hai cửa sổ có cùng phân phối, chỉ do cấu trúc mật độ cục bộ khác nhau. Cross-term vì vậy đóng vai trò trung lập, còn Weighted XX và YY làm rõ tín hiệu ở biên.

Hệ quả của thiết kế này là $\text{MMD}_{\text{IDW}}^2$ có một độ lệch dương nhỏ ngay cả khi $P = Q$. Độ lệch này được xử lý ở bước tính p-value bằng xấp xỉ Gamma (trình bày ở phần phân tích độ phức tạp bên dưới), thay vì đặt ngưỡng thủ công trên giá trị MMD.

Ảnh hưởng đến tín hiệu phát hiện

Khi dùng MMD gốc, các cặp điểm ở vùng trung tâm thường chiếm phần lớn tổng thống kê, làm tín hiệu drift ở vùng biên dễ bị chìm. IDW-MMD giảm ảnh hưởng của

vùng trung tâm và tăng trọng số cho các điểm vùng biên, nơi sudden drift thường biểu hiện sớm hơn.

Tuy nhiên, chính cơ chế này làm IDW-MMD ít nhạy hơn với gradual drift — loại drift thay đổi dần từ vùng trung tâm. Hạn chế này được thảo luận chi tiết ở cuối mục.

Thuật toán chi tiết như sau, trong đó mật độ cục bộ được tính độc lập trong từng cửa sổ:

Algorithm 2 Inverse Density-Weighted MMD (IDW-MMD).

Require: Reference window $X = \{x_1, \dots, x_n\}$, test window $Y = \{y_1, \dots, y_m\}$, kernel $k(\cdot, \cdot)$, stability constant $\epsilon = 0.5$

Ensure: IDW-MMD statistic $\widehat{\text{MMD}}_{\text{IDW}}^2$

```

1: for  $i = 1$  to  $n$  do                                ▷ inverse density weights for reference window  $X$ 
2:    $d(x_i) \leftarrow \sum_{j \neq i} k(x_i, x_j)$                 ▷ off-diagonal density; excludes self-kernel  $k(x_i, x_i)$ 
3:    $\tilde{w}_i \leftarrow 1 / (\sqrt{d(x_i)} + \epsilon)$ 
4: end for
5:  $\tilde{W}_{ij}^{XX} \leftarrow \tilde{w}_i \tilde{w}_j$  for  $i \neq j$ , else 0
6:  $W_{ij}^{XX} \leftarrow \tilde{W}_{ij}^{XX} / \sum_{i \neq j} \tilde{W}_{ij}^{XX}$                 ▷ normalize so  $\sum_{i,j} W_{ij}^{XX} = 1$ 
7: for  $p = 1$  to  $m$  do                                ▷ inverse density weights for test window  $Y$ 
8:    $d(y_p) \leftarrow \sum_{q \neq p} k(y_p, y_q)$ 
9:    $\tilde{v}_p \leftarrow 1 / (\sqrt{d(y_p)} + \epsilon)$ 
10: end for
11:  $\tilde{W}_{pq}^{YY} \leftarrow \tilde{v}_p \tilde{v}_q$  for  $p \neq q$ , else 0
12:  $W_{pq}^{YY} \leftarrow \tilde{W}_{pq}^{YY} / \sum_{p \neq q} \tilde{W}_{pq}^{YY}$ 
13:  $\widehat{\text{MMD}}_{\text{IDW}}^2 \leftarrow \sum_{i,j} W_{ij}^{XX} k(x_i, x_j) + \sum_{p,q} W_{pq}^{YY} k(y_p, y_q) - \frac{2}{nm} \sum_{i=1}^n \sum_{p=1}^m k(x_i, y_p)$  ▷ cross-term uses uniform
    weights  $1/(nm)$  as a geometric anchor; diagonal excluded from  $XX, YY$  terms (U-statistic style) return
     $\widehat{\text{MMD}}_{\text{IDW}}^2$ 

```

Phân tích độ phức tạp tính toán

Độ phức tạp của IDW-MMD được phân tích trong Bảng 4.1:

Bảng 4.1 Độ phức tạp tính toán của IDW-MMD và các phương pháp liên quan

Thành phần	Thời gian	Bộ nhớ	Ghi chú
Kernel Matrix	$O(n^2)$	$O(n^2)$	n = window size
Density weights	$O(n)$	$O(n)$	Sum over rows
Weighted MMD	$O(n^2)$	$O(1)$	Scalar output
Xấp xỉ Gamma	$O(B \cdot n^2)$	$O(n^2)$	Hoán vị chỉ số, tái sử dụng K
IDW-MMD + xấp xỉ Gamma	$O(B \cdot n^2)$	$O(n^2)$	Bỏ vòng tính lại kernel
ShapeDD gốc (permutation)	$O(N_{\text{perm}} \cdot n^2)$	$O(n^2)$	Tính lại kernel mỗi vòng

IDW-MMD có độ phức tạp $O(n^2)$, tương đương MMD. Permutation test gốc phải tính lại MMD^2 rất nhiều lần (vài nghìn vòng) để ước lượng phân phối dưới giả thuyết không drift, tốn $O(N_{\text{perm}} \cdot n^2)$. Thay vào đó, kiểm định được thực hiện bằng xấp xỉ Gamma theo Gretton et al. [45]. Kỹ thuật *hoán vị chỉ số tái sử dụng kernel matrix*

giúp giảm chi phí xuống $O(B \cdot n^2)$, với B nhỏ hơn nhiều bậc so với N_{perm} .

Theo Gretton et al. [16], phân phối giới hạn của thống kê MMD_u^2 phụ thuộc vào giả thuyết. Dưới giả thuyết có drift, MMD_u^2 tiệm cận một Gaussian xoay quanh giá trị MMD^2 thực. Nhưng dưới giả thuyết không có drift, MMD_u^2 không tiệm cận Gaussian; nó tiệm cận một χ^2 . Vì vậy, dùng Gaussian để xấp xỉ phân phối null có thể làm lệch mức ý nghĩa thực tế so với α đã đặt.

Cách tiếp cận theo [45] là khớp một phân phối Gamma vào hai moment đầu (trung bình và phương sai) của phân phối dưới giả thuyết không drift. Hai moment được ước lượng bằng một số ít lần hoán vị chỉ số trên cùng một ma trận kernel đã tính sẵn, nên bước kiểm định không phải tính lại kernel ở mỗi vòng như permutation test đầy đủ. Cụ thể, sau khi tính kernel matrix K một lần cho hai cửa sổ:

1. Tính $\widehat{\text{MMD}}_{\text{IDW}}^2$ trên cấu hình ban đầu của hai cửa sổ.
2. Hoán vị ngẫu nhiên các chỉ số một số ít lần và tính lại $\widehat{\text{MMD}}_{\text{IDW}}^2$ cho mỗi lần. Mỗi lần là một mẫu của null distribution; vì K đã sẵn, mỗi lần chỉ tốn $O(n^2)$ phép cộng-nhân.
3. Khớp Gamma vào trung bình và phương sai của các mẫu null vừa thu được, sau đó tính p-value (đuôi phải).
4. Nếu $p < \alpha$ (mặc định $\alpha = 0.05$), kết luận có drift.

Trong thực nghiệm ở Chương 5, ShapeDD-IDW chạy nhanh hơn ShapeDD gốc trong phép đo đầu-cuối vì không phải chạy permutation test đầy đủ cho mỗi ứng viên. Khi một tín hiệu có nhiều đỉnh ứng viên, các p-value được đọc với hiệu chỉnh Bonferroni theo số đỉnh được kiểm định trong cùng trace. Lợi ích chính của xấp xỉ Gamma là làm cho bước xác nhận drift có cơ sở thống kê rõ hơn tại mức ý nghĩa $\alpha = 0.05$ (Mục 5.2.5), chứ không chỉ là một tối ưu tốc độ.

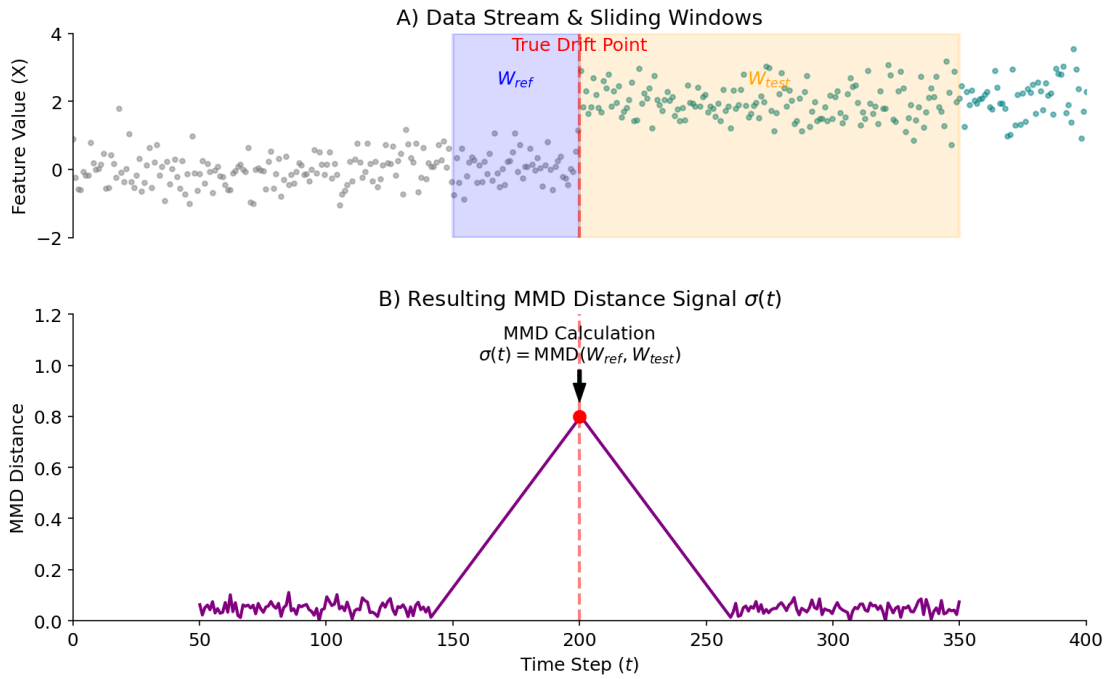
Hạn chế và khi nào nên dùng

Mặc dù IDW-MMD giúp bước kiểm định ổn định hơn trên các thay đổi rõ ở biên phân phối, điều này đặc biệt hiệu quả đối với sudden drift, nơi mà các thay đổi xảy ra đột ngột và rõ ràng, phương pháp có hai giới hạn cần lưu ý: Vì giảm trọng số vùng trung tâm, IDW-MMD có thể kém nhạy với các thay đổi nhẹ xảy ra chủ yếu trong vùng này. Cơ chế weighting cũng không phù hợp để phân loại loại drift, vì nó có thể làm mờ các đặc trưng hình dạng cần thiết cho gradual hoặc incremental drift. Do đó, SE-CDT (Mục 4.3) phân tích hình dạng của tín hiệu Standard MMD thay vì dùng trực tiếp tín hiệu IDW-MMD.

Kiến trúc kết hợp của Detection Module

Trong triển khai cuối cùng, Detection Module ShapeDD-IDW không dùng IDW-MMD ở mọi bước mà sử dụng kiến trúc kết hợp hai vai trò, tận dụng điểm mạnh của cả Standard MMD và IDW-MMD:

1. Tín hiệu trượt $\sigma(t)$ được tính bằng Standard MMD (unbiased U-statistic, không trọng số). Tín hiệu này giữ được hình dạng tổng thể của drift, đặc biệt với gradual/incremental vốn thay đổi chậm trong vùng trung tâm phân phối. Hình 4.3 minh họa cách hai cửa sổ trượt trên luồng dữ liệu tạo ra một điểm trên tín hiệu MMD.



Hình 4.3 Cơ chế Cửa sổ trượt tạo tín hiệu MMD. Mỗi điểm $\sigma(t)$ trên đồ thị MMD (dưới) là kết quả đo khoảng cách phân phối giữa cửa sổ tham chiếu W_{ref} và cửa sổ kiểm tra W_{test} trên luồng dữ liệu thô (trên).

2. Sau khi tìm đỉnh từ tín hiệu Standard MMD, mỗi đỉnh ứng viên được kiểm định bằng IDW-MMD với p-value dựa trên xấp xỉ Gamma (xem Mục 4.2.1). IDW-MMD nhạy với các thay đổi rõ ràng ở biên phân phối, còn xấp xỉ Gamma cung cấp phân phối null cho bước kiểm định.

Bảng thông γ được ước lượng một lần bằng median heuristic trên đoạn đầu của luồng, sau đó dùng nhất quán cho cả ba ma trận kernel K_{XX} , K_{YY} , K_{XY} trong toàn bộ tín hiệu trượt. Cách này tránh dao động khi mỗi cửa sổ tự ước lượng γ riêng.

Với thiết kế này, tín hiệu Standard MMD $\sigma(t)$ giữ được đầy đủ thông tin hình dạng cho bước phân loại, trong khi IDW-MMD chỉ được dùng ở bước xác nhận thống kê

của module phát hiện ShapeDD-IDW. Ở giao diện giữa phát hiện và phân loại, SE-CDT chỉ cần ba đầu ra ở mức thuật toán: cờ phát hiện drift, chuỗi tín hiệu Standard MMD $\sigma(t)$, và danh sách p-value của các ứng viên. Logic phân loại chỉ dùng $\sigma(t)$; nó không gọi IDW-MMD trực tiếp. Kết quả thực nghiệm của bước phát hiện và bước hiệu chỉnh H_0 được trình bày ở Mục 5.2 và Mục 5.2.5.

Kiến trúc kết hợp trên cho phép hệ thống phát hiện drift với độ chính xác cao và chi phí tính toán thấp; tuy nhiên phát hiện chỉ là bước đầu, để chọn chiến lược thích ứng phù hợp hệ thống còn cần biết *loại* drift đã xảy ra. Phương pháp SE-CDT, trình bày trong mục kế tiếp, đảm nhiệm bài toán phân loại drift không giám sát.

4.3 Phương pháp phân loại drift SE-CDT

Sau khi biết rằng drift đã xảy ra là chưa đủ — hệ thống còn cần biết drift đó thuộc loại nào, vì mỗi loại drift đòi hỏi chiến lược cập nhật khác nhau. SE-CDT (ShapeDD-Enhanced Concept Drift Type) được thiết kế để giải quyết bài toán này.

Phương pháp được tham khảo là CDT-MSW [5] với khả năng phát hiện và phân loại drift rất tốt, nhưng CDT-MSW cần nhãn dữ liệu: nó phân loại drift bằng cách so sánh độ chính xác phân loại giữa hai cửa sổ (accuracy ratio $\tilde{P} = a_B/a_A$). Trong môi trường không giám sát, không có nhãn nào để tính chỉ số này.

SE-CDT bỏ qua nhãn hoàn toàn và đọc trực tiếp *hình dạng* của tín hiệu MMD $\sigma(t)$ — tín hiệu vốn đã có sẵn từ bước phát hiện. Ý tưởng nền tảng là: mỗi loại drift để lại dấu ấn hình dạng khác nhau trên tín hiệu MMD. Sudden tạo đỉnh sắc nét và hẹp. Gradual tạo gò rộng và thoải. Incremental tạo xu hướng tăng dần. Recurrent tạo nhiều đỉnh lặp lại đều đặn. SE-CDT đọc những dấu ấn này để ra quyết định phân loại.

4.3.1 Cơ sở lý thuyết

Theo phân loại của CDT-MSW, drift chia thành hai loại chính:

- **TCD (Transient Concept Drift):** Drift tạm thời với thời gian chuyển đổi ngắn, gồm sudden, blip và recurrent (do tính chất lặp lại của các trạng thái ổn định).
- **PCD (Progressive Concept Drift):** Drift tiến triển với thời gian chuyển đổi dài, gồm gradual và incremental.

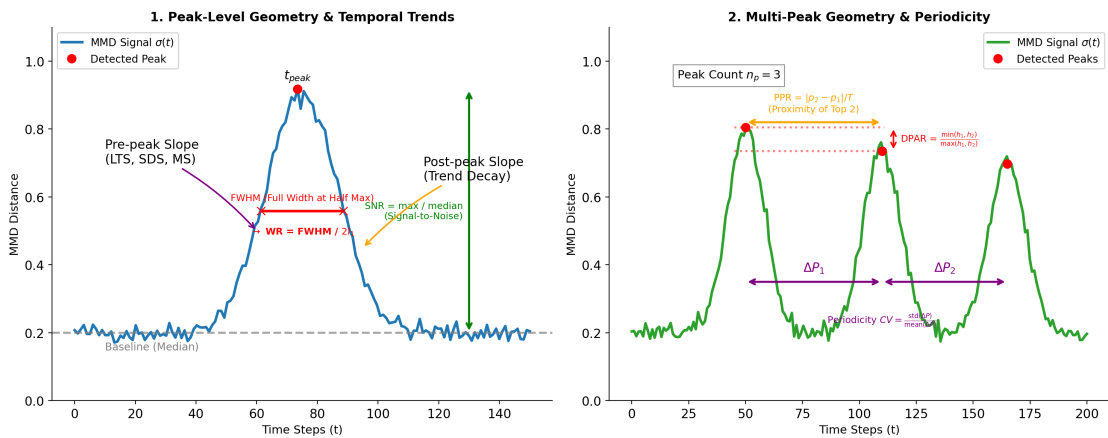
Tuy nhiên, SE-CDT chỉ nhìn thấy dữ liệu đầu vào X , không nhìn thấy nhãn Y , nên thứ nó phân loại là *hình dạng thay đổi của* $P(X)$ dưới giả định joint drift (Mục 2.1.2). Các tập đối chứng chỉ thay đổi $P(Y|X)$ — Hyperplane, LED Abrupt, Standard SEA

— nằm ngoài phạm vi quan sát và được dùng để kiểm tra hệ thống không báo nhầm (Mục 5.2).

4.3.2 Đặc trưng phân loại từ tín hiệu $\sigma(t)$

Thay vì dùng một thống kê duy nhất, SE-CDT trích xuất 9 đặc trưng từ hình dạng của tín hiệu MMD $\sigma(t)$. Sáu đặc trưng đầu mô tả hình dạng các đỉnh (Hình 4.4):

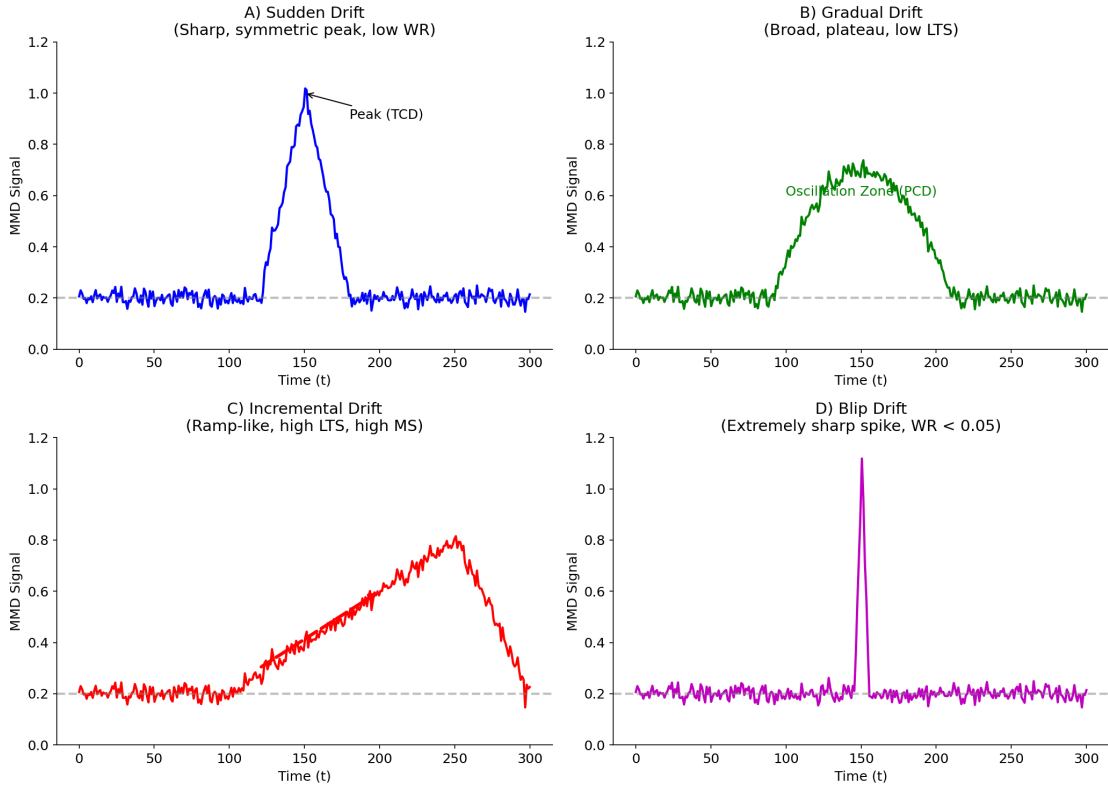
1. **Peak Width Ratio:** $WR = \frac{FWHM}{2l_1}$, trong đó FWHM (Full Width at Half Maximum) là độ rộng của đỉnh MMD đo tại nửa chiều cao cực đại. Đỉnh hẹp ứng với drift đột ngột, đỉnh rộng ứng với drift kéo dài.
2. **Peak Count (n_p):** Số lượng peaks. Sudden ít peak; Recurrent nhiều peak đều đặn.
3. **Periodicity CV:** Hệ số biến thiên khoảng cách giữa peaks. Recurrent có CV nhỏ (peak xuất hiện đều đặn).
4. **Signal-to-Noise Ratio:** $SNR = \frac{\max(\sigma)}{\text{median}(\sigma)}$. TCD có SNR cao do peak sắc nét, PCD có SNR thấp.
5. **Peak Proximity Ratio (PPR):** Khoảng cách gần nhất giữa hai peak chia cho độ dài tín hiệu. Blip drift có PPR nhỏ (hai peak gần nhau).
6. **Dual-Peak Amplitude Ratio (DPAR):** Tỷ lệ chiều cao giữa hai peak được chọn cho kiểm tra Blip. Với tín hiệu có hai hoặc ba peak, hệ thống lấy hai peak cao nhất để giảm ảnh hưởng của peak nhiễu nhỏ.



Hình 4.4 Phân tích tín hiệu MMD trong SE-CDT. Các đặc trưng hình học như FWHM (quyết định Width Ratio) và Prominence (quyết định SNR) được trích xuất trực tiếp từ hình dạng của đỉnh so với đường cơ sở (baseline).

Ba đặc trưng còn lại mô tả xu hướng thời gian của tín hiệu:

1. **Linear Trend Strength (LTS):** Hệ số R^2 của hồi quy tuyến tính trên tín hiệu; Incremental có LTS cao.
2. **Step Detection Score (SDS):** Tỷ lệ các bước nhảy đáng kể trong tín hiệu.
3. **Monotonicity Score (MS):** Tỷ lệ chiều hướng thống trị (tăng hoặc giảm) trong tín hiệu.



Hình 4.5 Hình dạng tín hiệu MMD đặc trưng của các loại Drift. A) Sudden tạo đỉnh tam giác sắc nét; B) Gradual tạo gò rộng, dao động, xu hướng tuyến tính thấp; C) Incremental tạo sườn dốc tăng dần với độ đơn điệu cao; D) Blip tạo gai nhọn cực hẹp.

Hình 4.5 minh họa hình dạng tín hiệu MMD đặc trưng của từng loại drift, là nền tảng trực quan cho các đặc trưng nói trên.

Các đặc trưng hình học và thời gian ở trên đều được trích xuất từ *hình dạng* của tín hiệu MMD $\sigma(t)$. Tuy nhiên, thực nghiệm cho thấy hai loại drift chậm (Gradual và Incremental) thường để lại hình dạng $\sigma(t)$ gần như giống hệt nhau (cùng là một gò rộng), khiến các đặc trưng LTS/MS/SDS không đủ sức phân tách. Để khắc phục, luận văn bổ sung một đặc trưng tính trực tiếp trên *dữ liệu thô*, gọi là **Variance Ratio (VR)**, dựa trên **Định luật Tổng Phương Sai (Law of Total Variance)**.

Xét một cửa sổ quan sát bắc qua điểm drift. Gọi C là biến chỉ thị khái niệm (concept) mà mỗi mẫu được sinh ra. Định luật Tổng Phương Sai phân rã phương sai

trong cửa sổ thành hai thành phần:

$$\underbrace{\text{Var}(X)}_{\text{tổng}} = \underbrace{\mathbb{E}[\text{Var}(X | C)]}_{\text{phương sai nội tại của từng khái niệm}} + \underbrace{\text{Var}(\mathbb{E}[X | C])}_{\text{phương sai giữa các trung bình khái niệm}}. \quad (4.5)$$

Hai loại PCD tác động lên (4.5) theo cách khác nhau:

- **Gradual drift** là một *hỗn hợp xác suất* (probabilistic mixture): trong giai đoạn chuyển tiếp, mẫu được rút từ khái niệm cũ *hoặc* mới với xác suất thay đổi dần. Vì cửa sổ chứa đồng thời hai cụm phân phối tách biệt, thành phần *between-concept* $\text{Var}(\mathbb{E}[X | C])$ tăng vọt, làm tổng phương sai trong cửa sổ **phình to** so với giai đoạn ổn định.
- **Incremental drift** là một *dịch chuyển trung bình liên tục* (continuous mean shift): tại mỗi thời điểm chỉ có một phân phối duy nhất, trung bình của nó trượt từ từ. Trong một cửa sổ đủ nhỏ, dữ liệu xấp xỉ một phân phối đơn với phương sai gần như **giữ nguyên**; thành phần *between-concept* không bùng nổ như trường hợp hỗn hợp.

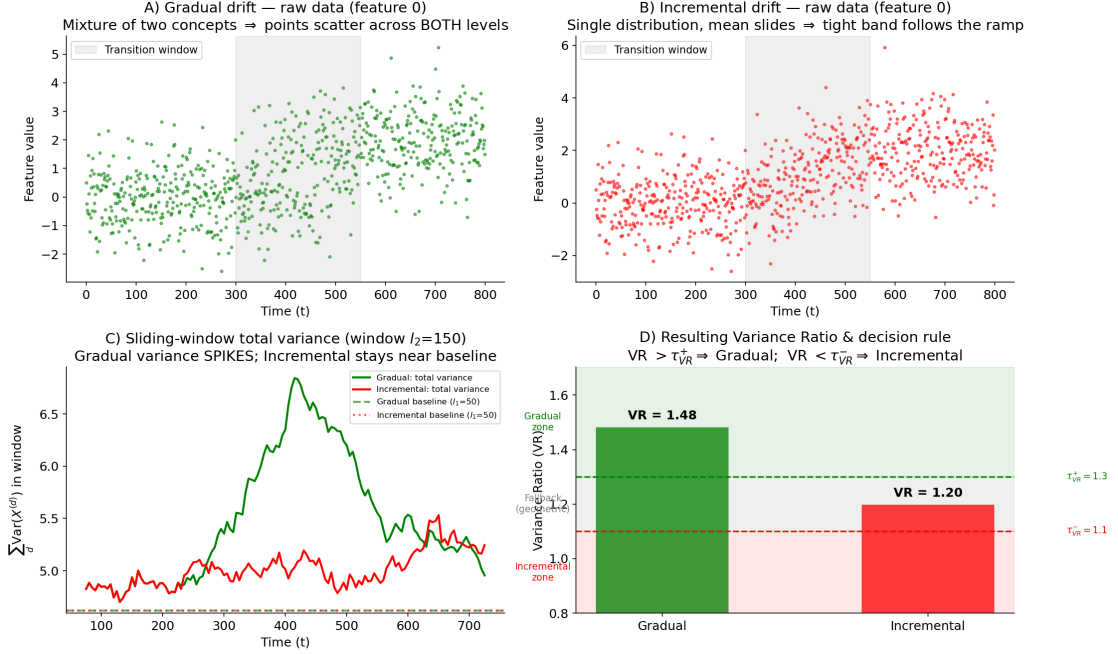
Từ nhận xét này, ta định nghĩa **Variance Ratio (VR)** là tỉ số giữa phương sai cực đại quan sát được trên các cửa sổ trượt kích thước l_2 và phương sai cơ sở (baseline) ước lượng trên l_1 mẫu đầu:

$$\text{VR} = \frac{\max_i \sum_{d=1}^D \text{Var}\left(X_{[i, i+l_2)}^{(d)}\right)}{\sum_{d=1}^D \text{Var}\left(X_{[0, l_1)}^{(d)}\right)}, \quad (4.6)$$

trong đó $X^{(d)}$ là chiều thứ d của dữ liệu (D chiều), cửa sổ trượt được dịch với bước 5 mẫu để giảm chi phí. Theo phân tích trên: VR *lớn* ($> \tau_{\text{VR}}^+$) báo hiệu phương sai phình to do hỗn hợp $\Rightarrow$ **Gradual**; VR *gần 1* ($< \tau_{\text{VR}}^-$) báo hiệu phương sai ổn định do dịch chuyển đơn $\Rightarrow$ **Incremental**. Khi VR rơi vào vùng trung gian $[\tau_{\text{VR}}^-, \tau_{\text{VR}}^+]$ (thường gặp với dữ liệu rời rạc/phân loại), hệ thống lùi về bộ quy tắc hình học LTS/MS/SDS để tránh quyết định thiếu cơ sở.

Hình 4.6 minh họa trực quan cơ chế này trên dữ liệu thô. Hàng trên cho thấy: ở Gradual, do mẫu được rút từ hỗn hợp hai khái niệm nên dữ liệu trải rộng trên cả hai mức trong cửa sổ chuyển tiếp; còn ở Incremental, dữ liệu luôn là một dải hẹp bám theo trung bình đang trượt. Hàng dưới định lượng quan sát đó: phương sai của cửa sổ trượt của Gradual *nhô lên rõ rệt* so với baseline, trong khi của Incremental gần như đi ngang. Kết quả VR tương ứng là $\text{VR}_{\text{Gradual}} = 1.48$ và $\text{VR}_{\text{Incremental}} = 1.20$. Trường hợp Gradual vượt $\tau_{\text{VR}}^+ = 1.3$ nên được VR gán trực tiếp là Gradual. Trường

hợp Incremental lại minh hoạ đúng vùng khó: giá trị 1.20 nằm trong vùng trung gian $[\tau_{VR}^-, \tau_{VR}^+] = [1.1, 1.3]$, tức VR *không* tự kết luận được và hệ thống phải lùi về bộ quy tắc hình học LTS/MS/SDS. Ví dụ này vì vậy cho thấy VR tách được chiều phương sai theo đúng hướng lý thuyết dự đoán, nhưng biên độ tách trên dữ liệu mô phỏng còn hẹp — đây chính là nguyên nhân của tỉ lệ nhầm lẫn Gradual/Incremental được phân tích ở Mục 5.3.3.



Hình 4.6 Minh hoạ cơ chế Variance Ratio (VR) trên dữ liệu thô. (A, B) Dữ liệu thô của Gradual và Incremental drift: hỗn hợp xác suất làm dữ liệu Gradual trải trên hai mức, còn Incremental là một dải hẹp trượt dần. (C) Phương sai trên cửa sổ trượt: Gradual nhô lên trên baseline (do thành phần between-concept theo Eq. (4.5)), Incremental giữ gần baseline. (D) Giá trị VR thu được và vùng quyết định: $VR > \tau_{VR}^+ \Rightarrow \text{Gradual}$; $VR < \tau_{VR}^- \Rightarrow \text{Incremental}$; vùng giữa lùi về quy tắc hình học.

Bảng 4.2 tổng hợp toàn bộ 10 đặc trưng, giúp tra cứu nhanh ý nghĩa và điều kiện phân loại của từng đặc trưng:

Bảng 4.2 Tổng hợp các đặc trưng phân loại của SE-CDT

Đặc trưng	Ký hiệu	Công thức / Cách tính	Miền giá trị	Ý nghĩa phân loại
<i>Nhóm 1: Đặc trưng hình học (Geometric Features)</i>				
Peak Width Ratio	WR	$\text{FWHM}/(2l_1)$	$[0, 1]$	Nhỏ $\Rightarrow$ Sudden; Lớn $\Rightarrow$ Gradual/Incremental
Peak Count	n_p	Số đỉnh vượt ngưỡng $\bar{\sigma}_s + 0.3\sigma_{\text{std}}$	$\mathbb{N}_0$	≤ 3 : Sudden; ≥ 4 : Recurrent
Periodicity CV	CV	$\text{std}(\Delta P)/\text{mean}(\Delta P)$	$[0, \infty)$	< 0.3 : Peaks đều (Recurrent)
Signal-to-Noise	SNR	$\max(\sigma_s)/\text{median}(\sigma_s)$	$[1, \infty)$	> 2.0 : Peaks sắc nét (TCD)
Peak Proximity	PPR	Khoảng cách giữa hai peak được chọn/ $ \sigma $	$[0, 1]$	Nhỏ: hai peaks gần (Blip)
Dual-Peak Amp.	DPAR	$\min(h_1, h_2)/\max(h_1, h_2)$	$[0, 1]$	Trung bình–cao: hai peaks cùng nổi bật (Blip)
<i>Nhóm 2: Đặc trưng thời gian (Temporal Features)</i>				
Linear Trend	LTS	R^2 của hồi quy tuyến tính trên σ_s	$[0, 1]$	> 0.5 : Xu hướng tuyến tính (Incremental)
Step Detection	SDS	Tỉ lệ bước nhảy $> 1.5\sigma$ trong σ_s	$[0, 1]$	> 0.12 : Nhiều bước nhảy (Incremental)
Monotonicity	MS	$ n^+ - n^- /(n^+ + n^-)$	$[0, 1]$	> 0.6 : Đơn điệu mạnh (Incremental)
<i>Nhóm 3: Đặc trưng phương sai trên dữ liệu thô (Raw-data Variance Feature)</i>				
Variance Ratio	VR	$\max_i \text{Var}(X_{[i, i+l_2]})/\text{Var}(X_{[0, l_1]})$	$[0, \infty)$	> 1.3 : Hỗn hợp (Gradual); < 1.1 : Dịch chuyển đơn (Incremental)

4.3.3 Thuật toán phân loại

Bảng 4.3 liệt kê toàn bộ siêu tham số của thuật toán cùng giá trị mặc định.

Bảng 4.3 Siêu tham số của SE-CDT và giá trị mặc định

Tham số	Giá trị mặc định	Vai trò
σ_g	4	Bảng thông bộ lọc Gauss làm mượt tín hiệu
α	0.3	Hệ số nhân ngưỡng tìm đỉnh
τ_ℓ	1	Ngưỡng độ dài tách PCD khỏi TCD
$\tau_{\text{WR}}^{\text{blip}}$	0.30	Chặn trên WR cho Blip (nhánh compact)
τ_{PPR}	0.20	Chặn trên Peak Proximity Ratio cho Blip
τ_{DPAR}	0.60	Chặn dưới Dual-Peak Amplitude Ratio cho Blip
τ_{LTS}	0.5	Ngưỡng xu hướng tuyến tính mạnh
τ_{LTS}^-	0.3	Xu hướng tuyến tính yếu (điều kiện kết hợp)
τ_{MS}	0.6	Ngưỡng độ đơn điệu
τ_{SDS}	0.12	Ngưỡng phát hiện bước nhảy
τ_{VR}^+	1.3	Chặn trên Variance Ratio (vượt $\Rightarrow$ Gradual)
τ_{VR}^-	1.1	Chặn dưới Variance Ratio (thấp hơn $\Rightarrow$ Incremental)
τ_n^{blip}	3	Số đỉnh tối đa cho Blip
τ_n^{sud}	3	Số đỉnh tối đa cho Sudden
τ_n^{rec}	4	Số đỉnh tối thiểu cho Recurrent
τ_n^{inc}	7	Số đỉnh dự phòng cho Incremental
τ_{WR}	$0.15^\dagger$	Ngưỡng Peak Width Ratio cho Sudden
τ_{SNR}	$2.0^\dagger$	Ngưỡng Signal-to-Noise cho TCD
τ_{CV}	$0.30^\dagger$	Ngưỡng Periodicity CV cho Recurrent
τ_{match}	0.15	Ngưỡng so khớp Concept Memory (Mục 4.3.7)

[†] Ba ngưỡng này là giá trị tính mặc định; khi bật tự hiệu chỉnh chúng có thể được nối lên theo phân vị quan sát trên cửa sổ không drift (Mục 4.3.6).

Algorithm 3 trình bày toàn bộ quy trình phân loại theo từng bước:

Algorithm 3 SE-CDT — Unsupervised Drift Type Classification

Require: Drift magnitude signal $\sigma(t)$ (Standard MMD) from Detection Module
Require: Raw data window W (dùng cho Variance Ratio)
Require: Threshold parameter set Θ (see Table 4.3); τ_{WR} , τ_{SNR} , τ_{CV} self-calibrated (Section 4.3.6)
Ensure: Drift type: one of {TCD-Blip, TCD-Sudden, TCD-Recurrent, PCD-Gradual, PCD-Incremental}

```

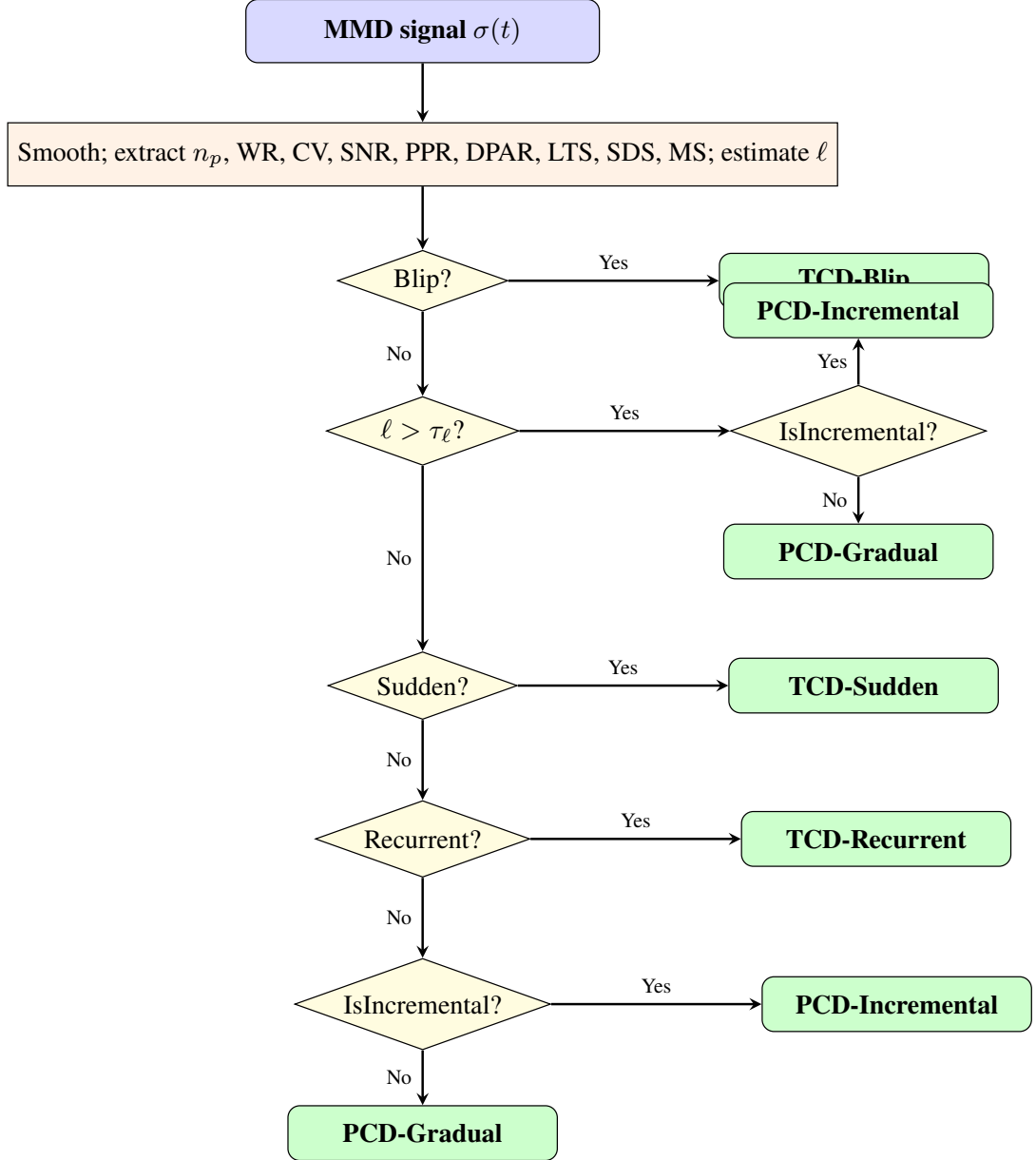
1:  $\sigma_s \leftarrow \text{GaussianFilter}(\sigma, \sigma_g)$  ▷ smooth signal
2:  $P \leftarrow \text{FindPeaks}(\sigma_s, \bar{\sigma}_s + \alpha \sigma_{\text{std}})$ 
3: Compute geometric features:  $n_p \leftarrow |P|$ , WR, CV, SNR, PPR, DPAR
4: Compute temporal features: LTS, SDS, MS
5:  $VR \leftarrow \text{VarianceRatio}(W)$  ▷ Eq. (4.6), trên dữ liệu thô
6:  $\ell \leftarrow \text{EstimateDriftDuration}(\sigma_s)$  ▷ via peak FWHM
7: if  $n_p \leq \tau_n^{\text{blip}}$  and  $WR < \tau_{WR}^{\text{blip}}$  and  $\text{BlipProfile}(\text{PPR}, \text{DPAR}, \text{SNR}, \text{LTS})$  then return TCD-Blip
8: end if
9: if  $\ell > \tau_\ell$  then ▷ wide peak  $\Rightarrow$  PCD
10:   if  $\text{ISINCREMENTAL}(VR, \text{LTS}, \text{MS}, \text{SDS}, n_p)$  then return PCD-Incremental
11:   elsereturn PCD-Gradual
12: end if
13: end if
14: if  $n_p \leq \tau_n^{\text{sud}}$  and  $WR < \tau_{WR}$  and  $\text{SNR} > \tau_{\text{SNR}}$  then return TCD-Sudden
15: end if
16: if  $n_p \geq \tau_n^{\text{rec}}$  and  $CV < \tau_{CV}$  and  $\text{LTS} < \tau_{\text{LTS}}$  then return TCD-Recurrent
17: end if
18: if  $\text{ISINCREMENTAL}(VR, \text{LTS}, \text{MS}, \text{SDS}, n_p)$  then ▷ PCD fallback return PCD-Incremental
19: elsereturn PCD-Gradual ▷ default
20: end if

21: function  $\text{ISINCREMENTAL}(VR, \text{LTS}, \text{MS}, \text{SDS}, n_p)$ 
22:   if  $VR > \tau_{VR}^+$  then return false ▷ phương sai phình to  $\Rightarrow$  Gradual
23:   end if
24:   if  $VR < \tau_{VR}^-$  then return true ▷ phương sai ổn định  $\Rightarrow$  Incremental
25:   end if
26:   // Vùng trung gian: lùi về quy tắc hình học
27:   if  $\text{LTS} > \tau_{\text{LTS}}$  then return true
28:   end if
29:   if  $\text{MS} > \tau_{\text{MS}}$  and  $\text{LTS} > \tau_{\text{LTS}}^-$  then return true
30:   end if
31:   if  $\text{SDS} > \tau_{\text{SDS}}$  and  $\text{LTS} > \tau_{\text{LTS}}^-$  then return true
32:   end ifreturn  $n_p \geq \tau_n^{\text{inc}}$ 
33: end function

```

Trong thuật toán, BlipProfile gồm hai trường hợp: cặp peak gần nhau với DPAR đủ cao, hoặc profile nhiều ngắn hạn có WR nhỏ, SNR trung bình, DPAR ở mức vừa phải và LTS thấp. Điều kiện này được kiểm tra trước nhánh PCD vì nhiều blip có FWHM hơi rộng do nhiễu, nếu xét $\ell > 1$ trước sẽ dễ bị gán nhầm thành Gradual.

Hình 4.7 minh họa trực quan luồng quyết định phân loại drift của SE-CDT (tương ứng với Algorithm 3):



Hình 4.7 Lưu đồ quyết định phân loại drift của SE-CDT (Algorithm 3). Điều kiện Blip được kiểm tra trước để tránh peak nhiễu ngắn bị chuyển nhầm vào nhánh PCD. Tín hiệu có đỉnh rộng ($\ell > \tau_\ell$) được phân loại trực tiếp là PCD; các trường hợp còn lại được kiểm tra lần lượt Sudden, Recurrent, trước khi fallback về PCD theo đặc trưng thời gian (Bảng 4.2).

4.3.4 Ưu điểm và độ phức tạp

SE-CDT có một số ưu điểm mang đến sự khác biệt so với CDT-MSW:

- **Không giám sát:** Hoạt động theo kiểu không giám sát, không cần nhãn dữ liệu, phù hợp với dữ liệu luồng.
- **Tích hợp:** Sử dụng tín hiệu Standard MMD $\sigma(t)$ đã tính trong quá trình phát hiện ứng viên drift, không cần thêm một nguồn tín hiệu riêng.

- **Thời gian thực:** Phân loại ngay sau khi phát hiện drift với chi phí $O(m)$ tuyến tính — rất nhỏ so với bước phát hiện.
- **Phân tách phát hiện và phân loại:** Toàn bộ pipeline phát hiện — tính tín hiệu Standard MMD, tìm đỉnh ứng viên, và kiểm định bằng IDW-MMD với xấp xỉ Gamma — thuộc module ShapeDD-IDW và được xem như một bước phát hiện độc lập. Module phân loại nhận lại tín hiệu Standard MMD $\sigma(t)$ từ module phát hiện và chỉ dùng $\sigma(t)$ này cho bước phân loại; logic phân loại của SE-CDT không gọi IDW-MMD trực tiếp.

Về độ phức tạp tính toán của toàn bộ quy trình:

- **Detection (ShapeDD-IDW):** $O(n^2 \cdot T/\text{step})$ với n = kích thước cửa sổ, T = độ dài stream, step = bước trượt. Đây là bước chiếm chi phí chính.
- **Classification (SE-CDT),** chạy sau khi bước phát hiện trả về $\sigma(t)$ với m điểm:
 - Làm mượt tín hiệu: $O(m)$
 - Tìm cực đại cục bộ: $O(m)$
 - Trích đặc trưng từ các peak: $O(n_p)$ với n_p là số peak
 - So sánh ngưỡng để phân loại: $O(1)$
- Tổng chi phí phân loại là $O(m)$, tuyến tính theo độ dài tín hiệu.
- Tổng chi phí quy trình vẫn chủ yếu nằm ở bước phát hiện: $O(n^2 \cdot T/\text{step})$.

Vì vậy, bước phân loại chỉ thêm một phần chi phí nhỏ so với phép tính MMD trong bước phát hiện.

4.3.5 Giải thích lựa chọn ngưỡng

Các ngưỡng trong Algorithm 3 được xác định trên synthetic datasets với cửa sổ chuẩn $n, m \approx 200$, dựa trên các quan sát sau: Blip được nhận diện bằng hai hoặc ba peak ngắn hạn, trong đó hai peak nổi bật nhất có quan hệ đủ gần về vị trí hoặc có profile phù hợp với một xung ngắn bị nhiễu. Sudden thường có ít peak, đỉnh hẹp và SNR cao. Recurrent có nhiều peak tương đối đều, còn incremental và gradual được tách bằng các đặc trưng thời gian như LTS, SDS và MS. Gradual là trường hợp mặc định khi tín hiệu kéo dài nhưng không có xu hướng tuyến tính đủ rõ.

Toàn bộ giá trị ngưỡng cụ thể nằm trong Algorithm 3 và được tối ưu cho cửa sổ chuẩn $n, m \approx 200$. Nếu thay đổi kích thước cửa sổ, các ngưỡng về độ rộng đỉnh,

SNR, hay khoảng cách peak cần được hiệu chỉnh lại; với dữ liệu thực tế, khuyến nghị tính chỉnh trên một tập dữ liệu nhỏ đặc thù cho từng miền ứng dụng.

Các ngưỡng trên là heuristic thực nghiệm, không phải kết quả suy diễn lý thuyết. Để giảm phụ thuộc vào các con số cứng, Mục 4.3.6 bổ sung cơ chế tự điều chỉnh ngưỡng theo mức nhiễu nền quan sát được, vẫn không cần nhân. Các hạn chế còn lại được thảo luận trong Chương 6.

4.3.6 Tự hiệu chỉnh ngưỡng phân loại (Online Self-Calibration)

Ngưỡng cứng có một vấn đề thực tế: ngưỡng SNR “đủ sắc nét” trên luồng dữ liệu sạch có thể không thể đạt được trên luồng có nhiễu nền cao. Kết quả là cùng một đỉnh sudden thật có thể bị bỏ qua chỉ vì nền nhiễu của luồng dữ liệu mới cao hơn luồng huấn luyện.

Để khắc phục mà vẫn không dùng nhân, luận văn bổ sung mô-đun **Rolling Feature Baseline**. Ý tưởng đơn giản: khi không có drift, hệ thống quan sát các đặc trưng SNR, WR, CV trông như thế nào và lưu lại làm chuẩn. Khi có drift mới cần phân loại, ngưỡng phân loại được điều chỉnh theo chuẩn này thay vì dùng giá trị cứng.

Cụ thể, mỗi khi một cửa sổ được kết luận là *không có drift*, ba đặc trưng SNR, WR, CV được trích xuất và đưa vào một bộ đệm vòng. Hệ thống dùng phân vị thực nghiệm từ bộ đệm này để điều chỉnh ngưỡng: lấy giá trị lớn hơn giữa ngưỡng tĩnh và phân vị quan sát được.

Cơ chế này tác động khác nhau lên từng đặc trưng:

- **Với SNR:** Sudden cần SNR lớn hơn ngưỡng. Kéo ngưỡng lên theo phân vị thứ 90 của baseline làm điều kiện khó đạt hơn, giúp tránh nhầm peak nhiễu thành Sudden khi nền nhiễu cao.
- **Với WR và CV:** điều kiện phân loại là “phải nhỏ hơn ngưỡng”. Kéo ngưỡng lên theo phân vị thứ 25 của baseline làm điều kiện dễ đạt hơn, giúp các peak Sudden trên stream nhiễu (vốn có WR/CV cao hơn bình thường) vẫn được bắt đúng.

Cơ chế này chỉ điều chỉnh theo một chiều để giữ nguyên hành vi trên các luồng dữ liệu sạch. Khi nhiễu nền ở mức bình thường, ngưỡng tĩnh tiếp tục được dùng; chỉ khi nền nhiễu cao bất thường, ngưỡng mới được điều chỉnh.

Baseline chỉ cập nhật khi cửa sổ vừa rồi không phát hiện drift và tín hiệu MMD đủ dài để các đặc trưng có ý nghĩa thống kê. Bộ đệm có cửa sổ làm nóng (warm-up): trước khi tích lũy đủ một số mẫu tối thiểu, hệ thống dùng ngưỡng tĩnh để tránh ngưỡng thích nghi chạy theo nhiễu ở giai đoạn đầu stream. Ba ưu điểm chính: (i) vẫn không cần nhân khi vận hành — baseline chỉ dùng tín hiệu MMD, không cần y ; (ii) ổn định

trên dữ liệu sạch — trên các luồng ít nhiễu, ngưỡng gần như giữ nguyên so với cấu hình tĩnh; (iii) để áp dụng giữa các miền dữ liệu — cùng một SE-CDT có thể thích nghi tốt hơn khi mức nhiễu nền thay đổi.

4.3.7 Bộ nhớ khái niệm cho Recurrent Drift (Concept Memory)

Recurrent drift có nghĩa là phân phối đã từng xuất hiện trước đây quay lại. Nếu không có cơ chế ghi nhớ, luật phân loại theo ngưỡng sẽ không biết điều này và phân loại nhầm mỗi lần lặp lại là một sự kiện sudden riêng lẻ. Luận văn bổ sung mô-đun **Concept Memory** — một bộ đệm vòng lưu lại các phân phối đã gặp — để giải quyết vấn đề này.

Bộ nhớ chứa tối đa M mẫu đại diện, mỗi mẫu S_k có kích thước n_s điểm và được lấy ngay sau khi một drift được phát hiện rồi ổn định trở lại. Cùng với mỗi S_k , hệ thống lưu cả giá trị γ_k được ước lượng bằng median heuristic tại thời điểm đó, để dùng cho bước tính bandwidth trung bình khi so khớp sau này.

Khi SE-CDT phát hiện một drift mới, quy trình tra cứu diễn ra như sau:

1. Trích xuất mẫu sau drift S^{new} từ cửa sổ trượt hiện tại.
2. Với mỗi $S_k \in \text{Memory}$, tính *Standard MMD* giữa S^{new} và S_k bằng RBF kernel với γ là trung bình $(\gamma_k + \gamma_{\text{new}})/2$.
3. Nếu tồn tại k^* sao cho $\text{MMD}(S^{\text{new}}, S_{k^*})$ nhỏ hơn ngưỡng so khớp τ_{match} , drift được gán là **TCD-Recurrent**.
4. Nếu không tìm được mẫu phù hợp, S^{new} được thêm vào bộ nhớ và mẫu cũ nhất bị thay thế.

Ngưỡng τ_{match} được chọn cao hơn rõ rệt so với độ nhiễu nền của MMD trên dữ liệu Gaussian đồng nhất, đủ để tránh báo nhầm hai concept khác nhau là giống nhau. Giá trị cụ thể của M , n_s , τ_{match} và độ nhiễu nền tham chiếu được liệt kê ở bảng tham số thực nghiệm (Chương 5).

Concept Memory dùng Standard MMD thay vì IDW-MMD vì bài toán ở đây là so khớp hai mẫu đại diện để xem chúng có đến từ cùng một phân phối hay không. IDW-MMD chỉ phục vụ bước xác nhận thống kê trong module phát hiện ShapeDD-IDW; logic phân loại và Concept Memory không gọi IDW-MMD trực tiếp. Đầu vào liên quan của Concept Memory là mẫu đại diện sau drift và tín hiệu Standard MMD $\sigma(t)$.

Concept Memory được gọi sau bước phân loại theo ngưỡng ban đầu và chỉ relabel thành Recurrent khi kết quả chưa phải Blip. Điều này tránh trường hợp một nhiễu ngắn hạn quay lại phân phối cũ bị đổi nhãn thành recurrent drift.

Kết quả phân loại từ SE-CDT được dùng để kích hoạt chiến lược thích ứng phù hợp, trình bày trong phần tiếp theo.

4.4 Khung chiến lược thích ứng mô hình

Dựa trên kết quả phân loại từ SE-CDT, hệ thống kích hoạt chiến lược cập nhật mô hình khác nhau cho từng loại drift. Cách tiếp cận này tổng hợp từ Guo *et al.* [5], Gama *et al.* [10], Lu *et al.* [11], và Yuan *et al.* [2].

4.4.1 Ma trận quyết định chiến lược

Thay vì áp dụng một chiến lược duy nhất (như huấn luyện lại toàn bộ) cho mọi trường hợp, hệ thống đề xuất sử dụng thông tin về loại drift để tối ưu hóa chi phí và hiệu suất. Bảng 4.4 mô tả ánh xạ giữa loại drift và chiến lược thích ứng:

Bảng 4.4 Chiến lược thích ứng theo loại Drift

Loại Drift	Chiến lược Thích ứng	Cơ sở lý luận
Sudden	Huấn luyện lại nhanh: Khởi tạo lại mô hình và huấn luyện trên cửa sổ dữ liệu mới nhất sau drift.	Dữ liệu cũ không còn giá trị, cần loại bỏ hoàn toàn để tránh nhiễu (negative transfer).
Incremental	Cập nhật liên tục: Cập nhật mô hình hiện tại với dữ liệu mới.	Sự thay đổi mang tính tiệm tiến, kiến thức cũ vẫn còn giá trị một phần.
Gradual	Cửa sổ có trọng số: Ưu tiên dữ liệu mới nhưng vẫn giữ lại một phần dữ liệu cũ.	Giai đoạn chuyển tiếp có sự pha trộn giữa hai phân phối, cần bộ nhớ đệm.
Recurrent	Lưu và dừng lại mô hình: Tìm kiếm trong kho lưu trữ mô hình cũ phù hợp và kích hoạt lại.	Tái sử dụng tri thức đã học, tiết kiệm chi phí huấn luyện lại.
Blip	Không cập nhật hoặc cập nhật rất nhẹ: Bỏ qua hoặc cập nhật với trọng số rất thấp.	Thay đổi chỉ là nhiễu tạm thời, tránh việc mô hình “quên” kiến thức ổn định.

Việc chọn chiến lược trong Bảng 4.4 cần xét thêm bốn yếu tố vận hành. Thứ nhất là chi phí false alarm: nếu mỗi cảnh báo kéo theo retrain tốn GPU hoặc dùng kiểm định chất lượng, hệ thống nên dùng cooldown dài hơn và ngưỡng chặt hơn. Thứ hai là độ trễ nhãn: khi nhãn chất lượng đến sau vài giờ hoặc vài ngày, SE-CDT chỉ đóng vai trò cảnh báo sớm và Adaptation Manager có thể chờ thêm xác nhận. Thứ ba là loại drift: sudden thường cần reset nhanh, gradual cần recency weighting, incremental hợp với warm-start/online update, recurrent hợp với cache, còn blip nên bỏ qua. Thứ tư là chi phí bỏ sót drift: trong miền rủi ro cao, false negative có thể làm mô hình suy giảm âm thầm, nên cần ưu tiên recall và thêm kiểm duyệt của chuyên gia trước khi triển khai mô hình mới.

4.4.2 Ảnh hưởng của báo động sai và bỏ sót drift

False alarm không chỉ là một lỗi thống kê mà còn là lỗi vận hành. Một cảnh báo sai có thể kích hoạt retrain không cần thiết, làm mô hình thay đổi liên tục (model churn), tăng độ trễ phục vụ, tiêu tốn compute và tạo thêm rủi ro kiểm thử khi mô hình mới được nạp nóng. Ngược lại, false negative làm drift thật không được xử lý; mô hình sản xuất tiếp tục chạy trên phân phối đã lệch, kéo theo model decay và giảm độ tin cậy của dự đoán chất lượng hoặc bảo trì.

Luận văn dùng một số cơ chế giảm thiểu ở mức prototype: cooldown sau mỗi drift event để tránh báo lại cùng một sự kiện, hiệu chỉnh Bonferroni khi một trace có nhiều đỉnh ứng viên, tinh chỉnh ngưỡng theo chi phí vận hành, và không cập nhật mô hình khi drift được phân loại là Blip. Với miền rủi ro cao, Adaptation Manager nên thêm bước human approval hoặc shadow deployment trước khi thay thế mô hình đang chạy; SE-CDT chỉ cung cấp bằng chứng drift, không tự quyết định triển khai mô hình mới.

4.4.3 Triển khai các chiến lược

Chiến lược cho Sudden Drift (Trọng tâm nghiên cứu)

Trong phạm vi luận văn, chiến lược cho *Sudden Drift* được cài đặt và đánh giá chi tiết nhất. Quy trình cụ thể như sau:

1. **Kích hoạt:** Khi SE-CDT phân loại drift là “Sudden”.
2. **Thu thập dữ liệu:** Hệ thống chờ và thu thập một lượng dữ liệu mới (N_{new}) ngay sau điểm drift.
3. **Huấn luyện lại:** Một instance mô hình mới (ví dụ: Logistic Regression) được khởi tạo với trọng số ngẫu nhiên và huấn luyện trên N_{new} .
4. **Thay thế:** Mô hình cũ bị loại bỏ hoàn toàn, mô hình mới được đưa vào sử dụng.

Model Caching theo Concept Memory

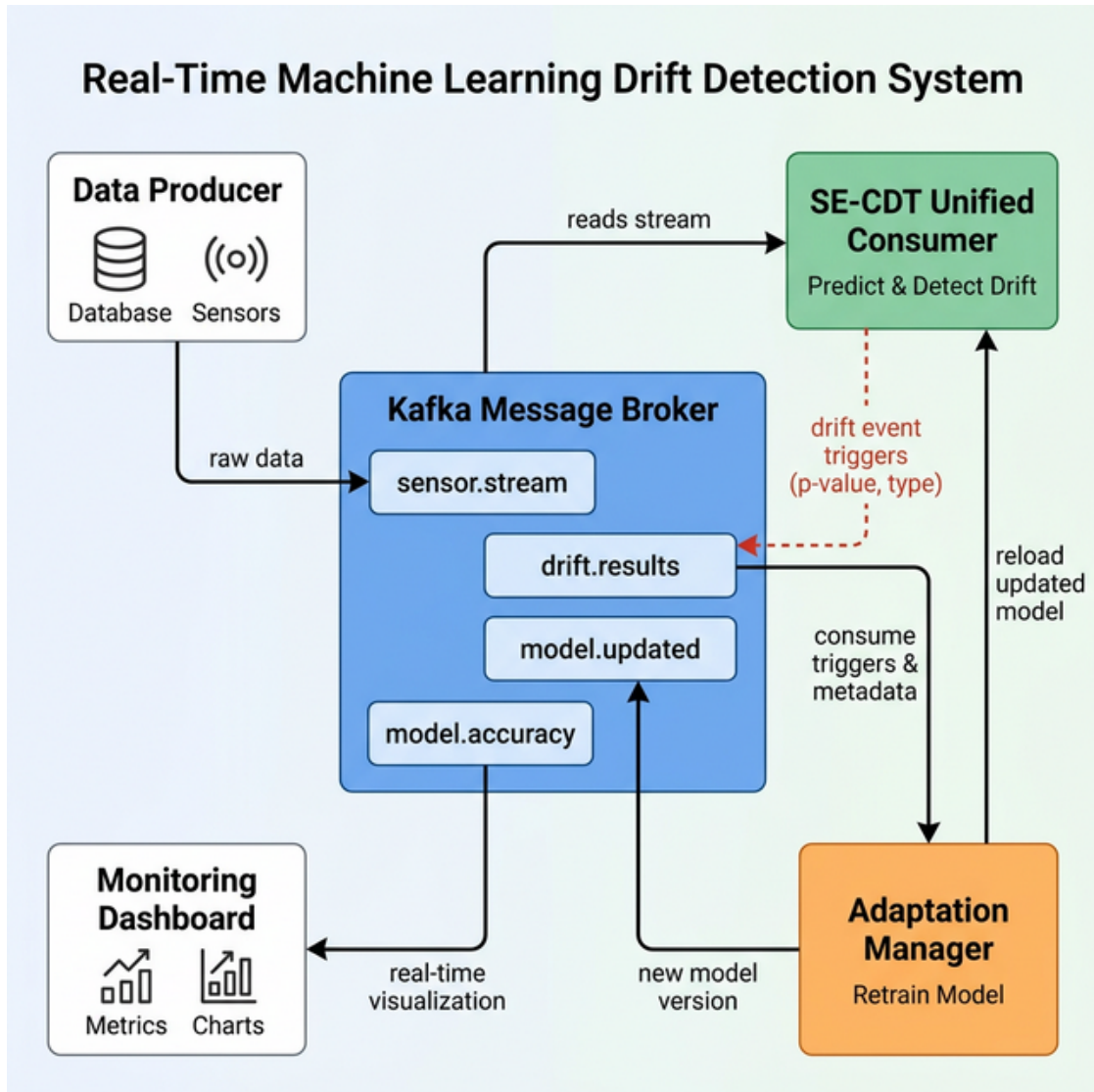
Khi phân loại trả về Recurrent, hệ thống tải lại mô hình đã được gắn với mẫu phân phối khớp trong Concept Memory (Mục 4.3.7) và tinh chỉnh nhẹ trên cửa sổ dữ liệu mới. Mỗi mẫu S_k trong bộ nhớ được lưu kèm tham chiếu tới mô hình đã huấn luyện trên giai đoạn ổn định tương ứng, tránh phải huấn luyện lại từ đầu cho khái niệm đã gặp.

4.5 Kiến trúc đề xuất cho hệ thống phát hiện trôi dạt và cập nhật mô hình thích ứng

Để hiện thực hóa các phương pháp trên trong môi trường dữ liệu luồng, luận văn đề xuất một kiến trúc xử lý phân tán dựa trên Apache Kafka [46, 47]. Kafka được chọn vì nó cung cấp một commit log phân tán, bền vững và có thể phân vùng, cho phép nhiều thành phần tiêu thụ cùng một luồng dữ liệu một cách độc lập [48, 49]; đây chính là tính chất mà một tầng giám sát drift cần, vì module phát hiện, module phân loại và bộ thích ứng phải đọc cùng luồng đặc trưng ở các nhịp khác nhau mà không chặn nhau. Nền tảng này cũng đã được dùng ở quy mô sản xuất cho các đường ống dữ liệu thời gian thực [50, 51]. Kiến trúc này được thiết kế theo hướng tách biệt thành phần, dễ mở rộng và dễ theo dõi. Tuy nhiên, phần này trình bày thiết kế kiến trúc và một nguyên mẫu kiểm thử trên nền tảng tương thích Kafka (Redpanda), không phải hệ thống vận hành hoàn chỉnh trong sản xuất. Các khía cạnh hạ tầng phân tán — độ trễ dưới tải cao, thông lượng khi tăng số phân vùng, và thời gian phục hồi khi có lỗi — nằm ngoài phạm vi đánh giá của luận văn và được xếp vào hướng phát triển tiếp theo (Chương 6). Các đánh giá trong Chương 5 tập trung vào hiệu năng thuật toán, không phải hiệu năng hạ tầng.

4.5.1 Tổng quan kiến trúc

Hệ thống được thiết kế theo mô hình **kiến trúc hướng sự kiện** (Hình 4.8), gồm các thành phần hoạt động độc lập và giao tiếp qua các Kafka topics:



Hình 4.8 Sơ đồ kiến trúc hệ thống phát hiện và thích ứng Drift

1. **Data Producer (Nguồn dữ liệu)**: Mô phỏng hoặc thu thập dữ liệu từ cảm biến/nguồn phát, gửi vào topic `sensor.stream`.
2. **SE-CDT Analysis Consumer (Bộ xử lý phân tích)**: Đọc dữ liệu từ luồng, thực hiện phát hiện drift bằng ShapeDD/MMD và phân loại loại drift nếu có thay đổi. Kết quả gồm vị trí và loại drift được gửi vào topic `drift.results`.
3. **Adaptation Manager (Bộ thích ứng)**: Lắng nghe sự kiện từ `drift.results`, lựa chọn chiến lược cập nhật mô hình phù hợp dựa trên loại drift đã phân loại.
4. **Monitoring Dashboard**: Trực quan hóa dữ liệu luồng, trạng thái hệ thống, và lịch sử các lần thích ứng.

4.5.2 Thiết kế chi tiết các thành phần

Producer và Data Ingestion

Producer được thiết kế để chịu lỗi và đảm bảo tính toàn vẹn dữ liệu:

- Sử dụng cơ chế **retry with exponential backoff** để xử lý mất kết nối mạng.
- Dữ liệu được đóng gói dưới dạng JSON gồm thời điểm, vector đặc trưng và thông tin mô tả.
- Hỗ trợ điều chỉnh tốc độ phát tin để mô phỏng các mức tải khác nhau.

Consumer và Cơ chế Windowing

Thành phần Consumer đóng vai trò quan trọng nhất, thực hiện phát hiện drift:

- **Circular Buffer:** Duy trì một bộ đệm tuần hoàn (750 mẫu trong nguyên mẫu, xem Phụ lục A.4) trong bộ nhớ để phục vụ tính toán của sổ trượt mà không cần truy xuất lại Kafka quá nhiều.
- **4-Phase Lifecycle:**
 1. *Pre-training:* Thu thập dữ liệu ban đầu để huấn luyện mô hình gốc.
 2. *Warm-up:* Giai đoạn ổn định, thiết lập baseline hiệu suất.
 3. *Monitoring:* Mô hình chạy ở chế độ dự đoán, liên tục kiểm tra drift bằng ShapeDD.
 4. *Adaptation:* Tạm dừng monitoring để cập nhật mô hình khi có drift.

Quản lý Topic và Giao tiếp

Hệ thống sử dụng các topic riêng biệt để phân tách luồng dữ liệu và luồng điều khiển:

Bảng 4.5 Danh sách Kafka Topics và Chức năng

Topic Name	Nội dung	Mục đích
sensor.stream	Raw data samples	Luồng dữ liệu đầu vào cho hệ thống phân tích.
drift.results	Drift info & Type	Kết quả phân tích từ SE-CDT (Vị trí + Loại Drift).
model.updated	Model metadata	Thông báo mô hình mới đã sẵn sàng (hot-reload).
model.accuracy	Performance metrics	Dữ liệu giám sát độ chính xác real-time.

4.6 Cơ chế chịu lỗi và đảm bảo độ tin cậy

Trong môi trường phân tán, các sự cố là không thể tránh khỏi. Hệ thống đề xuất tích hợp các cơ chế chịu lỗi (Fault Tolerance) ở nhiều cấp độ:

4.6.1 Xử lý lỗi ở mức ứng dụng

- **Suy giảm có kiểm soát:** Nếu bộ thích ứng gặp lỗi (ví dụ: tràn bộ nhớ khi huấn luyện), hệ thống tiếp tục dùng mô hình cũ và ghi cảnh báo thay vì dừng toàn bộ luồng xử lý.
- **Xử lý ngoại lệ:** Quá trình tính toán ShapeDD được bọc trong các khối an toàn để xử lý các ngoại lệ số học mà không làm dừng Consumer.

4.6.2 Xử lý lỗi ở mức hạ tầng (Kafka)

- **Replication:** Các topic quan trọng được cấu hình `replication.factor=3` để đảm bảo không mất dữ liệu ngay cả khi 1-2 broker bị lỗi.
- **Offset Management:** Consumer chỉ commit offset sau khi đã xử lý xong batch dữ liệu, đảm bảo ngữ nghĩa “at-least-once” (không bỏ sót dữ liệu).
- **Consumer Rebalancing:** Nếu một Consumer bị lỗi, Kafka sẽ tự động phân phối lại các partition cho các Consumer còn lại trong nhóm.

4.7 Kết luận chương

Chương này đã trình bày mô hình đề xuất **SE-CDT**, bao gồm cải tiến thuật toán phát hiện, phương pháp phân loại drift, khung thích ứng theo loại drift và kiến trúc hệ thống dựa trên Kafka. Cải tiến thuật toán được đánh giá thực nghiệm trong Chương 5, còn kiến trúc Kafka được xem như nguyên mẫu cho các bước triển khai thực tế sau này.

CHƯƠNG 5

THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1 Tổng quan thực nghiệm

5.1.1 Mục tiêu và cấu trúc chương

Chương này trình bày kết quả thực nghiệm để đánh giá hệ thống đề xuất. Đánh giá gồm ba phần, tương ứng với ba phần của hệ thống:

1. **Đánh giá khả năng phát hiện drift:** So sánh ShapeDD và biến thể cải tiến ShapeDD-IDW và IDW-MMD cùng với các phương pháp nền tảng trên nhiều tập dữ liệu, tập trung vào việc đánh giá trên các tập dữ liệu sudden drift là kịch bản ShapeDD được thiết kế để xử lý.
2. **Đánh giá khả năng phân loại drift của SE-CDT:** kiểm tra độ chính xác khi nhận diện loại drift (sudden, gradual, incremental, recurrent, blip) dựa trên tín hiệu drift magnitude.
3. **Đánh giá khả năng thích ứng:** đo hiệu quả của các chiến lược cập nhật mô hình tự động qua chỉ số Prequential Accuracy.

5.1.2 Môi trường và cấu hình

Các tham số môi trường và thuật toán được mô tả gắn với từng kịch bản (Detection, Classification, Adaptation) thay vì trình bày chung, để dễ có thể đánh giá một cách tổng quát và có thể thực hiện lại.

5.1.3 Tập dữ liệu thực nghiệm

Hệ thống được đánh giá trên bộ **14 tập dữ liệu**, gồm các tập tổng hợp (synthetic), một tập gradual drift được thiết kế thủ công, một tập recurrent drift dạng đối xứng (Gaussian Recurrent Stream) và một tập bán thực tế (Electricity semi-synthetic).

Cách đếm 14 tập dữ liệu. Mục này mô tả tổng cộng 15 tập, nhưng benchmark phát hiện chỉ dùng 14. Cụ thể: Stepping Drift được mô tả cùng nhóm Sudden cho liên mạch nhưng *chỉ* phục vụ đánh giá thích ứng (Mục 5.4), nên không nằm trong 14 tập của benchmark phát hiện. Trong 14 tập còn lại, tập Gaussian Stationary không chứa drift nên được tách riêng để đo báo động giả và không xuất hiện trong các bảng F1 theo dataset; vì vậy các bảng F1 (Bảng 5.4–5.7) liệt kê 13 dòng dataset. Ba con số 15, 14 và 13 do đó nhất quán với nhau, chỉ khác phạm vi sử dụng. Các tập tổng hợp được sinh bằng bộ sinh chuẩn của thư viện River [52], một framework mã nguồn mở phổ biến trong cộng đồng data stream mining. Lý do dùng dữ liệu tổng hợp thay vì dữ liệu thực tế thuần là vì với dữ liệu thực, thời điểm xảy ra concept drift (ground truth) hầu như không biết được, nên không thể đo F1-Score hay Detection Delay. Bộ sinh chuẩn cho phép kiểm soát vị trí, cường độ và loại drift, giúp kết quả có thể tái lập.

Cấu hình đánh giá phát hiện. Bộ đánh giá gồm 14 tập dữ liệu, mỗi tập chạy 30 seed độc lập. Hai tập được thêm để định nghĩa drift một cách chặt chẽ (dữ liệu recurrent dạng đối xứng $A \rightarrow B \rightarrow A \rightarrow B \dots$, đúng định nghĩa của Lu et al. [11]) và dữ liệu chuyển tiếp tuyến tính trên một khoảng xác định trước. Một tập dữ liệu tĩnh đóng vai trò đo báo động giả. Số lần chạy là 30 lần đủ để chạy kiểm định Friedman/Nemenyi tại $\alpha = 0.05$ với độ ổn định cao hơn cho các chỉ số trung bình.

Các tập dữ liệu được phân loại dựa trên bản chất của sự thay đổi phân phối:

Nhóm 1: Sudden Drift (Thay đổi $P(X)$ đột ngột): Đây là trọng tâm chính của module phát hiện SE-CDT (ShapeDD-IDW). Các dataset này chứa sự thay đổi đột ngột trong phân phối đầu vào $P(X)$ (Covariate Shift), nơi các phương pháp unsupervised cần phải phát hiện chính xác. Dạng drift này phản ánh các tình huống thực tế như: hệ thống IoT gặp sự cố cảm biến đột ngột, hoặc một dây chuyền sản xuất thay đổi nguyên liệu đầu vào.

- *Gaussian Shift (Moderate)*: Dữ liệu phân phối chuẩn đa chiều ($d = 10$) với sự dịch chuyển trung bình (mean shift) đột ngột ($\Delta\mu = 1.5$). Đây là trường hợp lý tưởng để kiểm tra độ nhạy của kernel RBF, vì phân phối Gaussian là nền tảng của hầu hết các mô hình thống kê. Kịch bản này được thiết kế dựa trên phương pháp đánh giá tham số (parametric artificial datasets) trong bài khảo sát của Hinder et al. [3], nơi các tác giả sử dụng chúng để so sánh có hệ thống các phương pháp phát hiện drift unsupervised trên cùng một nền tảng thống nhất.
- *Gaussian Concept Stream (GCS)*: Luồng dữ liệu 5 chiều với phân phối Gaussian, trong đó mỗi concept được định nghĩa bởi một vector trung bình khác nhau trên các chiều đặc trưng (concept 0: $\mu = +2.0$ trên x_0 ; concept 1: $\mu = -2.0$ trên x_0 ;

concept 2: $\mu = +1.5$ trên x_1). Chuỗi 3 concept tạo ra covariate shift $P(X)$ đột ngột, rõ ràng, phù hợp để kiểm tra các phương pháp phát hiện không giám sát. Cấu trúc xoay vòng concept lấy cảm hứng từ STAGGER (Schlimmer & Granger, 1986) [6], nhưng sử dụng đặc trưng Gaussian liên tục thay cho đặc trưng rời rạc, cho phép các kernel MMD quan sát $P(X)$ trực tiếp.

- *Random Uniform (Sensitivity Test)*: Bộ 4 dataset (Random Mild, Random Moderate, Random Severe, Random Ultra Severe) được sinh ngẫu nhiên với cường độ thay đổi (intensity) tăng dần từ 0.125 đến 2.0, sử dụng bộ sinh dữ liệu được tham khảo từ survey của Hinder et al. [3]. Bộ này đóng vai trò như một “nhiệt kế” dùng để đánh giá ngưỡng phát hiện (detection threshold) của các thuật toán: drift ở mức Random Mild ($\Delta\mu = 0.125$) gần như không thể phân biệt với nhiễu tự nhiên, trong khi Random Ultra Severe ($\Delta\mu = 2.0$) tạo ra sự thay đổi rõ ràng mà bất kỳ thuật toán nào cũng phải phát hiện được.
- *Stepping Drift (Cumulative)*: Biến thể được bổ sung để kiểm tra khả năng thích ứng. Khác với Sudden Drift thông thường (thường luân phiên $A \rightarrow B \rightarrow A$), Stepping Drift dịch chuyển phân phối tích lũy theo một hướng ($A \rightarrow B \rightarrow C \rightarrow D$). Kịch bản này mô phỏng môi trường vận hành liên tục tiến hóa mà không quay lại trạng thái cũ (ví dụ trời cảm biến công nghiệp), nên mô hình không thể “học vẹt” các trạng thái cũ mà buộc phải thích ứng với dữ liệu mới. *Kịch bản này chỉ được dùng trong đánh giá thích ứng (Mục 5.4), không thuộc phần đánh giá phát hiện ở Mục 5.2.*

Nhóm 2: Blip Drift (Thay đổi ngắn hạn): Trong thực tế vận hành, không phải mọi sự thay đổi đều mang tính vĩnh viễn. Một đợt tấn công mạng ngắn hạn, một sự kiện khuyến mãi flash sale, hay một lỗi tạm thời của cảm biến đều tạo ra các “xung nhiễu” (blip) trong luồng dữ liệu rồi nhanh chóng quay lại trạng thái bình thường. Khả năng phân biệt blip với drift thực sự là thách thức lớn đối với các thuật toán phát hiện.

- *RBF Blips*: Dữ liệu được sinh từ bộ sinh Random RBF Generator [52] với 50 trọng tâm Gaussian. Mỗi đoạn luân phiên sử dụng một cấu hình centroid khác nhau kết hợp với biến đổi tỉ lệ đặc trưng ($\times 1.5 + 0.3$) ở các đoạn lẻ, tạo ra sự thay đổi $P(X)$ đột ngột và xen kẽ trong không gian phi-Gaussian phức tạp nhiều centroid. Tập dữ liệu kiểm tra khả năng phát hiện distribution shift trên phân phối phức tạp hơn Gaussian đơn giản.

Nhóm 3: Real Drift / Thay đổi ranh giới quyết định (Control Group): Nhóm này đóng vai trò **nhóm đối chứng** (negative control), gồm các dataset chỉ có sự thay đổi

ở biên quyết định $P(Y|X)$ còn phân phối đầu vào $P(X)$ giữ nguyên (hoặc thay đổi không đáng kể). Đây là dạng *real drift* theo định nghĩa của Gama et al. [10], vốn nằm ngoài phạm vi quan sát của các phương pháp không giám sát chỉ dựa trên $P(X)$. Benchmark trên nhóm này giúp xác nhận thuật toán không bị “ảo giác” (hallucination) trước các thay đổi chỉ mang tính ngữ nghĩa nhần.

- *Standard SEA*: Bộ dữ liệu benchmark do Street và Kim đề xuất tại hội nghị ACM SIGKDD 2001 [53]. SEA (Streaming Ensemble Algorithm) ban đầu được thiết kế để đánh giá các phương pháp ensemble learning cho luồng dữ liệu. Bộ dữ liệu gồm 3 đặc trưng ngẫu nhiên, trong đó chỉ 2 đặc trưng đầu là liên quan; drift được tạo bằng cách thay đổi ngưỡng phân lớp $x_1 + x_2 > \theta$ (thường dùng các ngưỡng 8, 9, 7, 9.5). Đây là một trong những benchmark được trích dẫn nhiều nhất trong lĩnh vực concept drift.
- *Rotating Hyperplane*: Bộ dữ liệu do Hulten, Spencer và Domingos giới thiệu tại KDD 2001 [54] trong bối cảnh nghiên cứu khai phá luồng dữ liệu biến đổi theo thời gian (mining time-changing data streams). Mặt siêu phẳng d -chiều xoay dần trong không gian bằng cách thay đổi trọng số của các đặc trưng, mô phỏng gradual drift, dạng drift mà ranh giới quyết định dịch chuyển từ từ tương tự như sự thay đổi dần dần trong hành vi người dùng hay điều kiện thị trường.
- *LED Abrupt*: Bài toán phân lớp chữ số trên đèn LED 7 thanh, được sinh bằng bộ sinh chuẩn của River [52]. Bộ dữ liệu gồm 7 thuộc tính nhị phân (các thanh LED a–g), không thêm thuộc tính nhiễu. Drift được tạo bằng cách đảo ngược quy tắc phân lớp giữa các đoạn: đoạn chẵn phân loại chữ số chẵn là dương, đoạn lẻ đảo ngược lại, tạo ra sự thay đổi đột ngột trong $P(Y|X)$ mà không ảnh hưởng đến phân phối đặc trưng $P(X)$.

Nhóm 4: Dữ liệu bán thực tế (Semi-Synthetic) với điểm drift kiểm soát: Nhóm này bắc cầu giữa dữ liệu tổng hợp và dữ liệu thực tế thuần túy. Dataset lấy đặc trưng đầu vào từ dữ liệu thực (phức tạp, phi-Gaussian, có tương quan giữa các chiều) còn điểm drift thì được inject nhân tạo để có ground truth rõ ràng. Cách này cho phép kiểm tra hành vi của thuật toán trên phân phối phức tạp mà vẫn đo được các chỉ số đánh giá.

- *Electricity Semi-Synthetic*: Dựa trên bộ dữ liệu Electricity gốc của Harries [55] từ thị trường điện năng bang New South Wales, Australia (giai đoạn 1996–1998, 45,312 quan sát). Bộ dữ liệu gốc ghi nhận giá điện dao động theo thời gian thực với các đặc trưng như: nhu cầu điện năng, lượng chuyển giao liên bang, và chu kỳ thời gian. Đây là một trong những benchmark thực tế được sử dụng

phổ biến nhất trong nghiên cứu concept drift, do bản chất dữ liệu vốn đã chứa các yếu tố biến động tự nhiên (mùa vụ, sự kiện thị trường). Trong phiên bản semi-synthetic, 10 điểm drift được chèn tại các vị trí cố định bằng cách xáo trộn phân phối đặc trưng, cho phép kiểm tra hành vi của thuật toán trên dữ liệu phân phối phi-Gaussian phức tạp trong khi vẫn có ground truth xác định.

Nhóm 5: Recurrent, Gradual, và Stationary: Ba dataset trong nhóm này được thêm để định nghĩa Recurrent, Gradual và Stationary một cách chặt chẽ về mặt lý thuyết, đồng bộ với các định nghĩa drift trong khảo sát của Lu et al. [11].

- *Gaussian Recurrent Stream (GRS)*: Luồng Gaussian 5 chiều với 4 concept có vector trung bình khác nhau, lặp theo chu kỳ $A \rightarrow B \rightarrow A \rightarrow B \dots$ đúng với định nghĩa Recurrent của Lu et al. [11]. Mỗi concept được giữ ổn định một khoảng thời gian trước khi quay về concept đã từng xuất hiện, cho phép Concept Memory (Mục 4.3.7) phát huy đúng vai trò.
- *Gaussian Gradual Synthetic*: Dữ liệu Gaussian đa chiều với chuyển tiếp tuyến tính giữa hai mean trên một khoảng chuyển tiếp xác định trước (mặc định 400 mẫu). Khác với hyperplane (drift gradual chỉ ở $P(Y|X)$), dataset này cho drift gradual thực sự ở $P(X)$, do đó đo được khả năng phát hiện drift chậm của các phương pháp không giám sát. Đây cũng là điểm yếu cố hữu của IDW-MMD và là một trường hợp khó quan trọng để kiểm tra giới hạn của phương pháp.
- *Gaussian Stationary (No Drift)*: Luồng Gaussian tĩnh (không có sự kiện drift). Tập này đóng vai trò đo báo động giả trên dữ liệu tĩnh, bổ sung cho kiểm tra Type-I error ở Mục 5.2.5: nó cho biết liệu detector có sinh báo động giả khi luồng dữ liệu hoàn toàn không đổi hay không.

5.1.4 Chiến lược đánh giá

Hệ thống SE-CDT được đánh giá theo cấu trúc của hai thành phần bên trong cùng với khung thích ứng. **Thành phần phát hiện (ShapeDD-IDW)** chịu trách nhiệm cho việc phát hiện thời điểm drift dựa trên IDW-MMD và Gamma-approximation p-value, ưu tiên giảm FP và độ trễ. **Thành phần phân loại** của SE-CDT phân loại loại drift (sudden, gradual, incremental, recurrent, blip) bằng cách phân tích hình dạng của tín hiệu $\sigma(t)$ trên cửa sổ trượt kích thước W , hoàn toàn không giám sát. **Khung thích ứng** dùng nhãn loại drift do SE-CDT sản xuất để kích hoạt chiến lược cập nhật phù hợp (Full Reset cho sudden, Incremental Update cho gradual/incremental, Concept

Memory cho recurrent). Chiến lược đánh giá ở các mục sau bám theo cấu trúc ba giai đoạn này.

Các mục thực nghiệm dưới đây trả lời những câu hỏi khác nhau (độ chính xác định vị, độ chính xác phân loại, mức Type-I error, khả năng phục hồi của mô hình), nên dùng các cấu hình cửa sổ, δ , cooldown và nghĩa của FP khác nhau (Bảng 5.1). Khi đối chiếu số liệu giữa các mục cần lưu ý rằng cùng một phương pháp có thể cho EDR/FP khác nhau không phải do thay đổi thuật toán mà do mục đo đang trả lời câu hỏi khác.

Bảng 5.1 Quy ước thiết lập ở các mục đánh giá. Cùng một detector nhưng được đo dưới những cấu hình cửa sổ và dung sai khác nhau tùy câu hỏi mà mục đó trả lời; bảng này là nơi đối chiếu duy nhất cho các giá trị W , δ và cooldown xuất hiện rải rác trong chương.

Mục đánh giá	Cửa sổ	δ (mẫu)	Cooldown	Nghĩa của FP	Metric chính
Detection (Mục 5.2)	$l_1=50, l_2=150$	75	150 ($= 2\delta$)	Báo trên dữ liệu tĩnh / lệch định vị	F1, EDR, FP
Classification (Mục 5.3)	$W=50, s=10$	250	—	Báo sai vị trí ngoài khoảng chuyển tiếp	ACC_{cat} , ACC_{sub} , EDR
Hiệu chỉnh H_0 (Mục 5.2.5)	$W=300$	—	—	Type-I error trên dữ liệu tĩnh	Type-I error
Adaptation (Mục 5.4)	$l_1=50, l_2=150$	400	—	Báo dư trong khoảng phục hồi	Prequential Accuracy
Nguyên mẫu Kafka (Mục 5.5)	$W=250$	—	—	Không đo	Phục hồi độ chính xác

5.2 Đánh giá khả năng phát hiện của SE-CDT

Mục này đánh giá khả năng phát hiện của SE-CDT, tức ShapeDD-IDW, bằng cách đối chiếu với các phương pháp phát hiện drift tiêu biểu thuộc nhiều trường phái: KS-Test (thống kê cổ điển), MMD (kernel-based), DAWIDD (window-based), D3 (Discriminative Drift Detector [43]), và ShapeDD gốc [4]. Trọng tâm phân tích được đặt vào khả năng nhận diện. Sự thay đổi phân phối đầu vào $P(X)$ (Distribution Shift / Covariate Shift). Vì module phân loại của SE-CDT chỉ chạy sau khi module phát hiện báo drift, F1 phát hiện của SE-CDT trùng với ShapeDD-IDW; cả hai dòng được giữ lại trong bảng kết quả để bộc lộ rõ mối quan hệ này.

5.2.1 Thiết lập thực nghiệm phát hiện drift (Detection Setup)

Các tham số cho bài toán phát hiện drift được thiết lập như sau:

- Dữ liệu bao gồm 14 tập dữ liệu (13 tổng hợp + 1 bán thực tế, chi tiết ở Mục 5.1.2). Tập stationary (không có drift) được tách riêng để đo false-positive. Kích thước của luồng dữ liệu 10,000 mẫu mỗi dataset với 10 điểm drift được chèn tại các vị trí cố định (mỗi 1,000 mẫu) để tạo ground truth, trừ tập stationary (0 drift).
- Quy trình đánh giá được thực hiện 30 lần chạy độc lập (30 runs) với các seed ngẫu nhiên khác nhau để đảm bảo tính ổn định của kết quả (xem Mục 5.1.2).

- Độ trễ chấp nhận $\delta = 75$ và cooldown = 150 ($= 2\delta$) như Bảng 5.1.

5.2.2 Kết quả so sánh hiệu suất phát hiện

Bảng 5.2 trình bày kết quả tổng hợp của các phương pháp ShapeDD-IDW và SE-CDT và các phương pháp được tham khảo (MMD, KS-Test, D3, DAWIDD, ShapeDD). Các phương pháp ShapeDD-IDW và SE-CDT dùng chung thuật toán phát hiện nên có F1 trùng nhau; SE-CDT bổ sung bước phân loại được đánh giá ở Mục 5.3. Các chỉ số F1 tính dựa trên khả năng phát hiện đúng các điểm drift đã biết (ground truth), tổng hợp từ 30 lần chạy độc lập trên 14 tập dữ liệu.

Bảng 5.2 So sánh tổng hợp hiệu suất các phương pháp phát hiện drift. *Lưu ý:* ShapeDD-IDW là module phát hiện của SE-CDT; SE-CDT là toàn bộ hệ thống (bao gồm cả module phân loại). Ở benchmark phát hiện này, hai dòng dùng cùng thuật toán nên F1, Precision, Recall, FP trùng nhau theo thiết kế.

Method	Precision	Recall (EDR)	F1-Score	Delay	False Pos.
DAWIDD	0.435	0.734	0.532	38	10.3
IDW_MMD	0.432	0.737	0.531	36	9.6
SE_CDT	0.432	0.737	0.531	36	9.6
ShapeDD_IDW	0.432	0.737	0.531	36	9.6
MMD	0.429	0.734	0.527	35	10.5
ShapeDD	0.377	0.749	0.491	34	13.2
D3	0.558	0.472	0.489	16	0.6
KS	0.224	0.775	0.335	34	27.2

DAWIDD có F1 cao nhất về mặt số học (0.532), còn ShapeDD-IDW (cùng cấu hình IDW-MMD đứng riêng làm ablation) đạt $F_1 = 0.531$ với Precision = 0.432, Recall/EDR = 0.737 và 9.6 báo động giả trung bình mỗi run. Chênh lệch 0.001 này nhỏ hơn nhiều so với độ phân giải của benchmark; theo kiểm định Nemenyi ở Mục 5.2.3, khác biệt thứ hạng giữa hai phương pháp không đạt mức ý nghĩa thống kê. Đóng góp chính của ShapeDD-IDW vì vậy không nằm ở F1 mà ở chi phí tính toán (nhanh hơn ShapeDD gốc khoảng $7\times$, Bảng 5.9) và ở khả năng phân loại loại drift được đánh giá ở Mục 5.3.

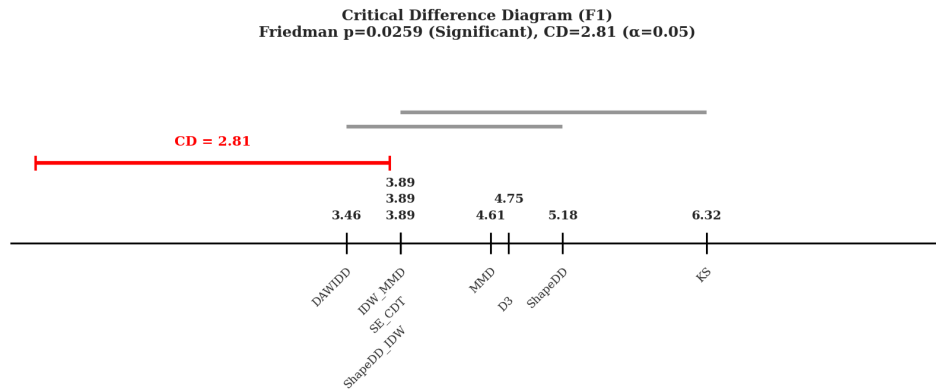
So với ShapeDD gốc, ShapeDD-IDW giữ F1 cao hơn (0.531 so với 0.491) và giảm FP từ 13.2 xuống 9.6. Cải tiến này đến từ cơ chế trọng số nghịch biến mật độ (Thuật toán 2) kết hợp xấp xỉ Gamma thay cho permutation test. Về thời gian chạy, benchmark cho thấy ShapeDD-IDW nhanh hơn khoảng $7\times$ so với ShapeDD gốc (Bảng 5.9). D3 có Precision cao nhất và FP thấp nhất nhưng Recall thấp hơn, còn KS-Test có Recall cao nhưng tạo quá nhiều báo động giả để phù hợp với hệ thống giám sát tự động.

Số FP $\sim 9.6/\text{run}$ của module phát hiện SE-CDT (ShapeDD-IDW) và cấu hình IDW-MMD đứng riêng cần được đọc cùng số lượng cửa sổ và số ứng viên được kiểm định. Trên bộ đánh giá này, mỗi seed có nhiều cửa sổ kiểm định và nhiều điểm drift

thật. Bước lọc hình dạng giúp giảm số ứng viên trước khi kiểm định bằng IDW-MMD, vì vậy FP thực tế thấp hơn trường hợp kiểm định mọi cửa sổ một cách độc lập. Tham số khoảng triệt báo cooldown = 150 (xem mục thiết lập) cũng giúp triệt tiêu phần lớn báo trùng trong vùng lân cận của mỗi điểm drift thật. Nếu muốn giảm FP hơn nữa, cần siết mức α hoặc dùng hiệu chỉnh đa kiểm định.

5.2.3 Phân tích ý nghĩa thống kê

Để kiểm tra liệu khác biệt hiệu suất giữa các phương pháp có ý nghĩa thống kê hay không, Hình 5.1 trình bày biểu đồ Critical Difference (CD) dùng kiểm định Friedman và Nemenyi post-hoc theo quy trình của Demšar [56] cho việc so sánh nhiều thuật toán trên nhiều tập dữ liệu.



Hình 5.1 Biểu đồ Critical Difference (CD) với mức ý nghĩa $\alpha = 0.05$. DAWIDD có thứ hạng trung bình tốt nhất; ShapeDD-IDW và cấu hình IDW-MMD đứng riêng xếp ngay sau (cách DAWIDD 0.429, nhỏ hơn ngưỡng CD = 2.806) nên không khác biệt có ý nghĩa thống kê với DAWIDD. SE-CDT có cùng thứ hạng vì dùng chung thuật toán phát hiện.

Bảng 5.3 Thứ hạng trung bình theo F1-score trong kiểm định Friedman/Nemenyi. ShapeDD-IDW và SE-CDT dùng chung thuật toán phát hiện nên có thứ hạng trung bình bằng nhau.

Method	Average Rank	Overall Rank
DAWIDD	3.464	1
IDW_MMD	3.893	2
SE_CDT	3.893	3
ShapeDD_IDW	3.893	4
MMD	4.607	5
D3	4.750	6
ShapeDD	5.179	7
KS	6.321	8
Critical Difference (CD) = 2.806		

Kết quả kiểm định Friedman/Nemenyi trong Bảng 5.3 cho thấy DAWIDD có thứ hạng trung bình tốt nhất (3.464); ShapeDD-IDW và cấu hình IDW-MMD đứng riêng

đạt thứ hạng trung bình 3.893, sát ngay sau DAWIDD (SE-CDT có cùng thứ hạng vì dùng chung thuật toán phát hiện). Với ngưỡng $CD = 2.806$ ($\alpha = 0.05$), khoảng cách giữa ShapeDD-IDW và DAWIDD chỉ $0.429 < CD$, nên khác biệt thứ hạng F1 không đạt mức ý nghĩa thống kê. Tương tự, khoảng cách so với MMD (0.714) cũng nhỏ hơn CD nên không thể kết luận khác biệt thống kê. Trong nhóm dựa trên MMD, các biến thể đề xuất có thứ hạng trung bình tốt hơn MMD (3.893 so với 4.607) nhưng cách biệt nằm dưới ngưỡng Nemenyi, nên không khác biệt thống kê đáng kể.

5.2.4 Đánh giá chi tiết trên từng loại dữ liệu

Hiệu suất của các phương pháp thay đổi tùy thuộc vào đặc tính của tập dữ liệu (Bảng 5.4–5.7).

Bảng 5.4 F1-Score theo dataset (Phần 1: Dữ liệu cơ bản và Mild/Moderate Drift)

Method	Electricity	Gaussian Gradual Synthetic	Gaussian Moderate	Random Mild
DAWIDD	0.250	0.472	0.753	0.594
IDW_MMD	0.383	0.470	0.736	0.511
SE_CDT	0.383	0.470	0.736	0.511
ShapeDD_IDW	0.383	0.470	0.736	0.511
MMD	0.291	0.462	0.740	0.566
ShapeDD	0.267	0.451	0.700	0.529
D3	0.213	0.029	0.998	0.012
KS	0.263	0.315	0.356	0.335

Bảng 5.5 F1-Score theo dataset (Phần 2: Drift mạnh và thay đổi mặt phẳng)

Method	Random Moderate	Random Severe	Random Ultra Severe	Hyperplane
DAWIDD	0.728	0.739	0.738	0.136
IDW_MMD	0.688	0.708	0.712	0.179
SE_CDT	0.688	0.708	0.712	0.179
ShapeDD_IDW	0.688	0.708	0.712	0.179
MMD	0.685	0.707	0.709	0.169
ShapeDD	0.619	0.636	0.638	0.197
D3	0.244	0.931	0.933	0.000
KS	0.407	0.434	0.434	0.240

Bảng 5.6 F1-Score theo dataset (Phần 3: RBF Blips, GCS và GRS)

Method	Led Abrupt	Rbfbliips	Stagger	Stagger Recurrent Explicit
DAWIDD	0.130	0.749	0.750	0.749
IDW_MMD	0.148	0.744	0.755	0.753
SE_CDT	0.148	0.744	0.755	0.753
ShapeDD_IDW	0.148	0.744	0.755	0.753
MMD	0.127	0.747	0.742	0.739
ShapeDD	0.163	0.670	0.681	0.679
D3	0.000	1.000	0.998	1.000
KS	0.071	0.371	0.459	0.458

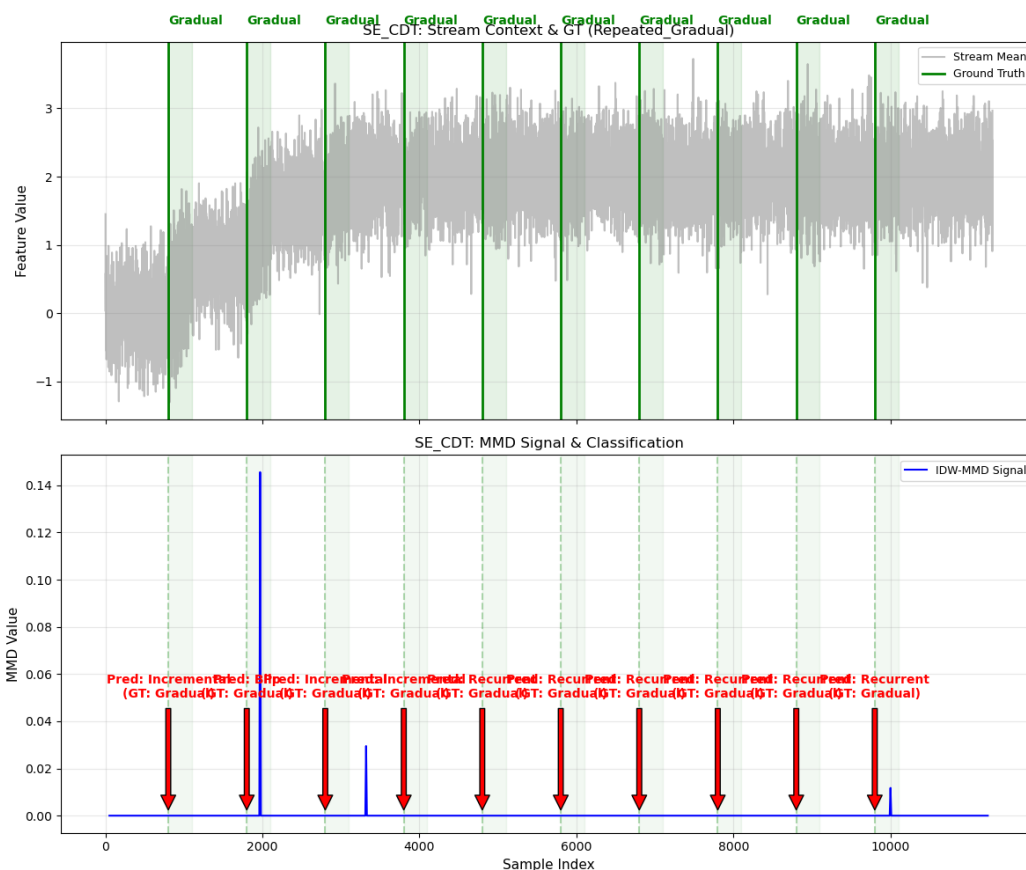
Bảng 5.7 F1-Score theo dataset (Phần 4: Standard SEA và trung bình)

Method	Standard Sea	Mean
DAWIDD	0.129	0.532
IDW_MMD	<u>0.118</u>	<u>0.531</u>
SE_CDT	<u>0.118</u>	<u>0.531</u>
ShapeDD_IDW	<u>0.118</u>	<u>0.531</u>
MMD	0.163	0.527
ShapeDD	0.151	0.491
D3	0.000	0.489
KS	0.206	0.335

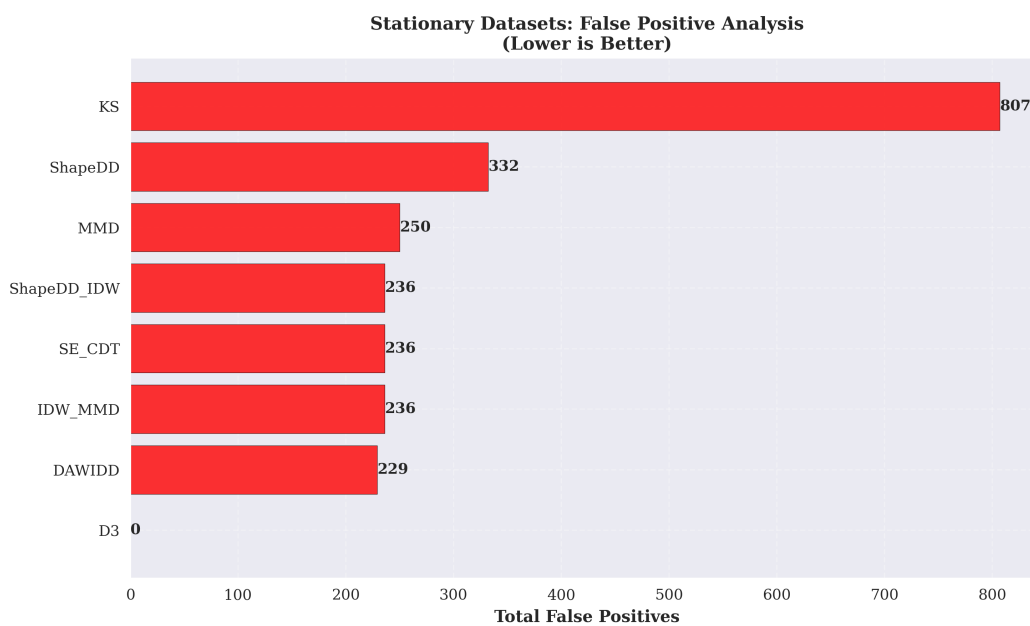
1. Trên các tập có thay đổi phân phối rõ như Random Moderate, Random Severe, Random Ultra Severe, GCS và GRS (Gaussian Recurrent Stream), ShapeDD-IDW đạt F1 khoảng 0.69–0.76, ổn định hơn ShapeDD gốc ở phần lớn các trường hợp và giữ được tín hiệu Standard MMD cần cho bước phân loại. Hình 5.2 minh họa dạng tín hiệu ShapeDD tại một kịch bản sudden drift.
2. Đối với các tập dữ liệu drift nhẹ và drift dần dần, trên các tập như Random Mild, ba phương pháp đề xuất đạt $F1 = 0.511$, thấp hơn MMD (0.566) nhưng không bị mất hoàn toàn tín hiệu. Với Gaussian Gradual Synthetic, cả ba phương pháp đề xuất đạt $F1 = 0.470$, cao hơn MMD (0.462) một chút (Hình 5.3 minh họa tín hiệu trên kịch bản gradual). Bước kiểm định IDW-MMD với p-value Gamma vẫn đủ nhạy để phát hiện drift dần dần khi cửa sổ tham chiếu và cửa sổ kiểm tra đủ lớn ($l_1 = 50, l_2 = 150$).
3. Đối với các tập dữ liệu chỉ thay đổi về nhãn như Hyperplane, LED Abrupt, và Standard SEA, điểm F1 của module phát hiện SE-CDT tương đối thấp (0.179, 0.148, và 0.118). Đây là hành vi phù hợp với phạm vi của phương pháp không giám sát: hệ thống chủ yếu quan sát $P(X)$, nên các thay đổi nằm ở $P(Y|X)$ không tạo tín hiệu rõ. Tuy nhiên, các giá trị khác 0 cũng cho thấy vẫn còn một mức báo động dư thừa do nhiễu mẫu và do cách đánh giá theo cửa sổ.
4. Đối với blip drift, trên tập RBF Blips, ba phương pháp đề xuất đạt $F1 = 0.744$ ở bài toán phát hiện, gần với MMD (0.747), trong khi D3 đạt cao hơn (1.000) nhờ classifier phân biệt cửa sổ trước và sau blip. Các thay đổi ngắn hạn vẫn có thể được phát hiện hiệu quả; việc phân loại đúng Blip vẫn còn khó hơn và được phân tích riêng ở Mục 5.3.3.
5. Báo động giả trong detection: Bảng 5.2 báo cáo FP trung bình theo mỗi run trên toàn bộ bộ đánh giá: KS-Test cao nhất (27.2 FP/run), trong khi ShapeDD-IDW (và cấu hình IDW-MMD đứng riêng) ở mức ~ 9.6 FP/run, thấp hơn ShapeDD

[illegible]

Hình 5.2 Minh họa tín hiệu phát hiện trên dữ liệu Sudden Drift (kịch bản Mixed A). ShapeDD tạo ra hình dạng “tam giác cân” đặc trưng tại điểm drift, đúng như dự báo lý thuyết.



Hình 5.3 Minh họa tín hiệu trên dữ liệu Gradual Drift. Tín hiệu (màu đỏ - Subplot 03) không tạo thành đỉnh nhọn rõ ràng mà bị san phẳng.



Hình 5.4 Tổng số báo động giả trên tập dữ liệu tĩnh (Stationary), gộp qua các lần chạy. Chỉ số này bổ sung cho FP/run ở Bảng 5.2: KS tạo ra nhiều báo động sai nhất, trong khi các phương pháp MMD-based thấp hơn rõ rệt.

5.2.5 Kiểm tra tỉ lệ báo động sai trên dữ liệu không có drift

Số False Positive quan sát được trên bộ đánh giá phát hiện chỉ phản ánh một phần hành vi của kiểm định, vì các mốc drift thật tương đối thưa và bước lọc theo hình dạng đỉnh đã loại bớt ứng viên trước khi kiểm định. Để đánh giá trực tiếp liệu kiểm định có thực sự gần mức $\alpha = 0.05$ hay không, luận văn tiến hành thêm một thí nghiệm phụ trên dữ liệu **hoàn toàn không có drift** và đo trực tiếp **Type-I error** (xác suất báo động sai khi giả thuyết H_0 đúng) của ba bộ phát hiện chính:

- **MMD** với p-value tính bằng permutation test ($N_{\text{perm}} = 2,500$).
- **IDW-MMD** với p-value tính theo xấp xỉ Gamma.
- **SE-CDT** (kết hợp ShapeDD-IDW cho phát hiện và luật phân loại theo ngưỡng).

Mỗi phương pháp chạy trên ba phân phối tính để bao quát các tình huống thường gặp: (i) Gauss-iid $d = 5$, (ii) Gauss-iid $d = 10$ (kiểm tra ảnh hưởng số chiều), và (iii) Gauss-AR(1) $d = 5$ với $\rho = 0.6$ (kiểm tra dữ liệu có tương quan thời gian). Mỗi cấu hình chạy độc lập 200 lần với seed khác nhau, dùng cửa sổ tổng kích thước $W = 300$ mẫu. Standard MMD và IDW-MMD chia W thành hai nửa bằng nhau (150 tham chiếu vs 150 kiểm tra); SE-CDT (composite) áp dụng giao thức ShapeDD với $l_1 = 50$, $l_2 = 150$ trên cùng cửa sổ. Mức ý nghĩa danh nghĩa $\alpha = 0.05$.

Bảng 5.8 Tỉ lệ Type-I error trên dữ liệu tính (200 lần chạy mỗi cell, $\alpha = 0.05$). Số sau dấu $\pm$ là nửa khoảng tin cậy 95% theo Wilson.

Distribution	Standard MMD (perm)	IDW-MMD (Gamma null)	SE-CDT (composite)
Gauss-iid d=5	0.025 $\pm$ 0.022	0.040 $\pm$ 0.027	0.115 $\pm$ 0.044
Gauss-iid d=10	0.055 $\pm$ 0.032	0.050 $\pm$ 0.030	0.080 $\pm$ 0.038
Gauss-AR1 d=5	0.045 $\pm$ 0.029	0.075 $\pm$ 0.037	0.140 $\pm$ 0.048

Kết quả ở Bảng 5.8 cho thấy:

- MMD (permutation test, $N_{\text{perm}} = 2,500$) có Type-I error nằm trong khoảng 0.025–0.055, sát với mức danh nghĩa $\alpha = 0.05$. Đây là hành vi đúng với lý thuyết, cho thấy MMD dùng làm baseline trong luận văn không bị thiên lệch.
- IDW-MMD với xấp xỉ Gamma đạt Type-I error trong khoảng 0.040–0.075, gần với mức danh nghĩa $\alpha = 0.05$. Kiểm định do đó không quá bảo thủ như xấp xỉ Gaussian dưới giả thuyết không drift.
- SE-CDT (kết hợp bước lọc hình dạng và IDW-MMD ở bước kiểm định) đạt Type-I error 0.080–0.140, cao hơn α rõ rệt — anti-conservative trên cả hai phân phối $d = 5$ iid (0.115) và AR(1) (0.140, $\approx 2.8\times$ mức danh nghĩa). Mô hình đã áp

dụng hiệu chỉnh Bonferroni khi một trace có nhiều đỉnh ứng viên, nhưng Type-I error tổng hợp của toàn pipeline vẫn chịu ảnh hưởng từ bước chọn đỉnh và tính chất phụ thuộc giữa các cửa sổ trượt. Vì vậy kết quả H_0 được diễn giải thận trọng như một kiểm tra calibration thực nghiệm, không phải bảo đảm lý thuyết đầy đủ cho toàn bộ pipeline.

Việc chọn xấp xỉ Gamma theo Gretton et al. [45] thay cho xấp xỉ Gaussian xuất phát từ một nhận xét lý thuyết: dưới giả thuyết không drift, MMD_u^2 tiệm cận một χ^2 có trọng chú không phải Gaussian. Nếu áp dụng Gaussian dưới giả thuyết không drift, kiểm định có xu hướng quá bảo thủ. Xấp xỉ Gamma khắc phục điểm này tốt hơn và cho kết quả gần mức $\alpha = 0.05$ hơn trong thí nghiệm trên dữ liệu tĩnh.

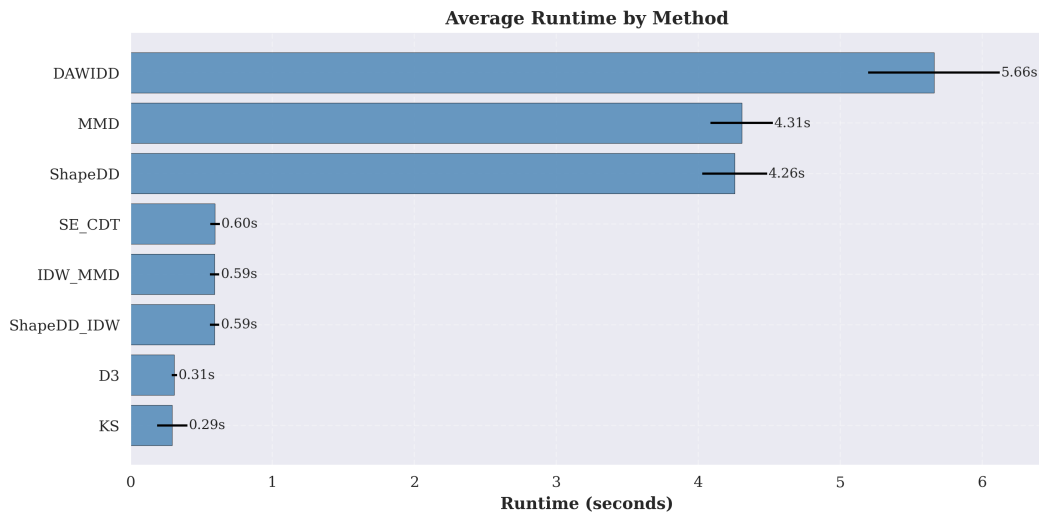
5.2.6 Đánh giá hiệu suất tính toán

Bảng 5.9 so sánh thời gian và thông lượng.

Bảng 5.9 So sánh hiệu suất tính toán của các phương pháp

Method	Runtime (s/stream) [†]	Throughput (samples/s)	Speedup vs ShapeDD
KS	0.29	33,990	14.47×
D3	0.31	32,300	13.75×
ShapeDD-IDW	0.59	16,858	7.18×
IDW-MMD (standalone)	0.59	16,849	7.17×
SE-CDT	0.60	16,764	7.14×
ShapeDD (gốc)	4.26	2,349	1.00×
MMD	4.31	2,322	0.99×
DAWIDD	5.66	1,767	0.75×

[†]Runtime đo bằng `time.process_time()` cho toàn bộ một stream 10,000 mẫu, thực thi tuần tự để loại bỏ tranh chấp tài nguyên CPU, lấy trung bình trên tất cả dataset và toàn bộ các lần chạy. Throughput = số mẫu/Runtime. Speedup được tính tương đối với ShapeDD gốc.



Hình 5.5 Thời gian chạy trung bình của các phương pháp trên đánh giá phát hiện (10,000 mẫu, 30 lần chạy).

ShapeDD-IDW nhanh hơn rõ rệt so với ShapeDD gốc trên luồng 10,000 mẫu (Bảng 5.9, Hình 5.5), khoảng $7\times$, nhờ thay permutation test bằng xấp xỉ Gamma; MMD và DAWIDD chậm hơn do không có cơ chế kiểm định gần đúng. Điểm quan trọng hơn là bước kiểm định dựa trên xấp xỉ Gamma có diễn giải thống kê rõ hơn so với việc chỉ đặt ngưỡng thủ công trên trị MMD.

5.3 Đánh giá module phân loại của SE-CDT

Phần này đánh giá khả năng phân loại của SE-CDT: khả năng nhận diện loại drift (Sudden, Gradual, Incremental, Recurrent, Blip) dựa trên hình dạng tín hiệu drift magnitude $\sigma(t)$ do module phát hiện ShapeDD-IDW (Mục 5.2) sản xuất, hoàn toàn không giám sát.

5.3.1 Thiết lập thực nghiệm phân loại drift

Thực nghiệm phân loại dùng song song hai chế độ đo lường, ứng với hai câu hỏi khác nhau:

- **Oracle mode** (đo CAT, SUB): SE-CDT được cho biết trước vị trí drift ground-truth và chỉ thực hiện bước phân loại. Chế độ này tách biệt năng lực phân biệt hình dạng tín hiệu khỏi năng lực tự phát hiện.
- **Real detection mode** (đo EDR, MDR): SE-CDT tự tìm điểm drift từ tín hiệu MMD rồi phân loại, đánh giá pipeline đầu-cuối. EDR (Event Detection Rate) là tỉ lệ sự kiện drift được nhận diện; MDR (Miss Detection Rate = $1 - \text{EDR}$) là tỉ lệ bỏ sót.

Vì CAT và SUB được đo ở oracle mode, hai cấu hình SE-CDT (Std) và SE-CDT (IDW) cho cùng kết quả CAT/SUB. Khác biệt giữa hai cấu hình chỉ xuất hiện ở EDR/MDR. Đánh giá được chạy với 6 kịch bản tổng hợp (30 seeds mỗi kịch bản = 180 luồng, mỗi luồng chứa 10 sự kiện drift = 1800 sự kiện phân loại):

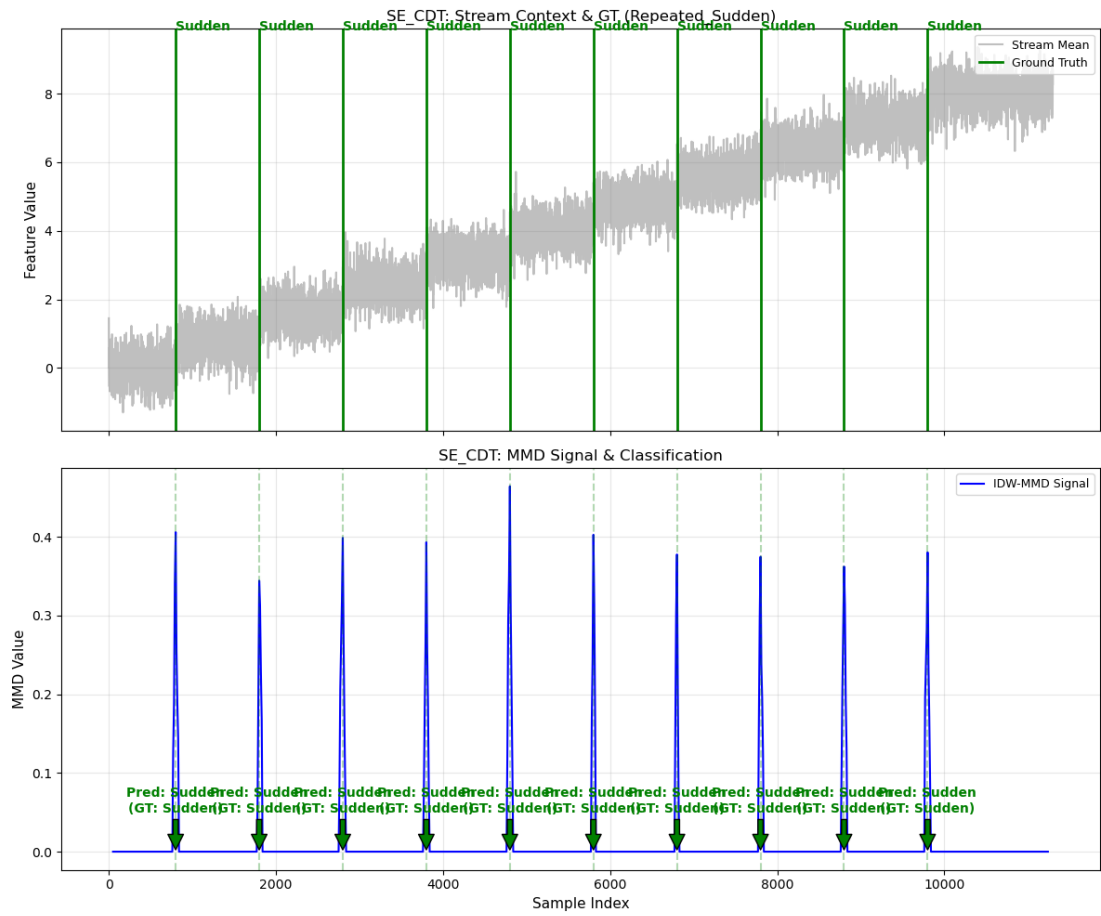
Bảng 5.10 Sáu kịch bản benchmark module phân loại của SE-CDT

Kịch bản	Thành phần drift (xoay vòng)	Độ dài luồng
Mixed A	Sudden $\rightarrow$ Gradual $\rightarrow$ Recurrent	$\sim 11,800$ mẫu
Mixed B	Blip $\rightarrow$ Incremental $\rightarrow$ Sudden	$\sim 13,800$ mẫu
Repeated Sudden	$10 \times$ Sudden	$\sim 9,600$ mẫu
Repeated Gradual	$10 \times$ Gradual	$\sim 11,800$ mẫu
Repeated Incremental	$10 \times$ Incremental	$\sim 13,800$ mẫu
Repeated Recurrent	$10 \times$ Recurrent (A $\rightarrow$ B $\rightarrow$ A)	$\sim 11,800$ mẫu

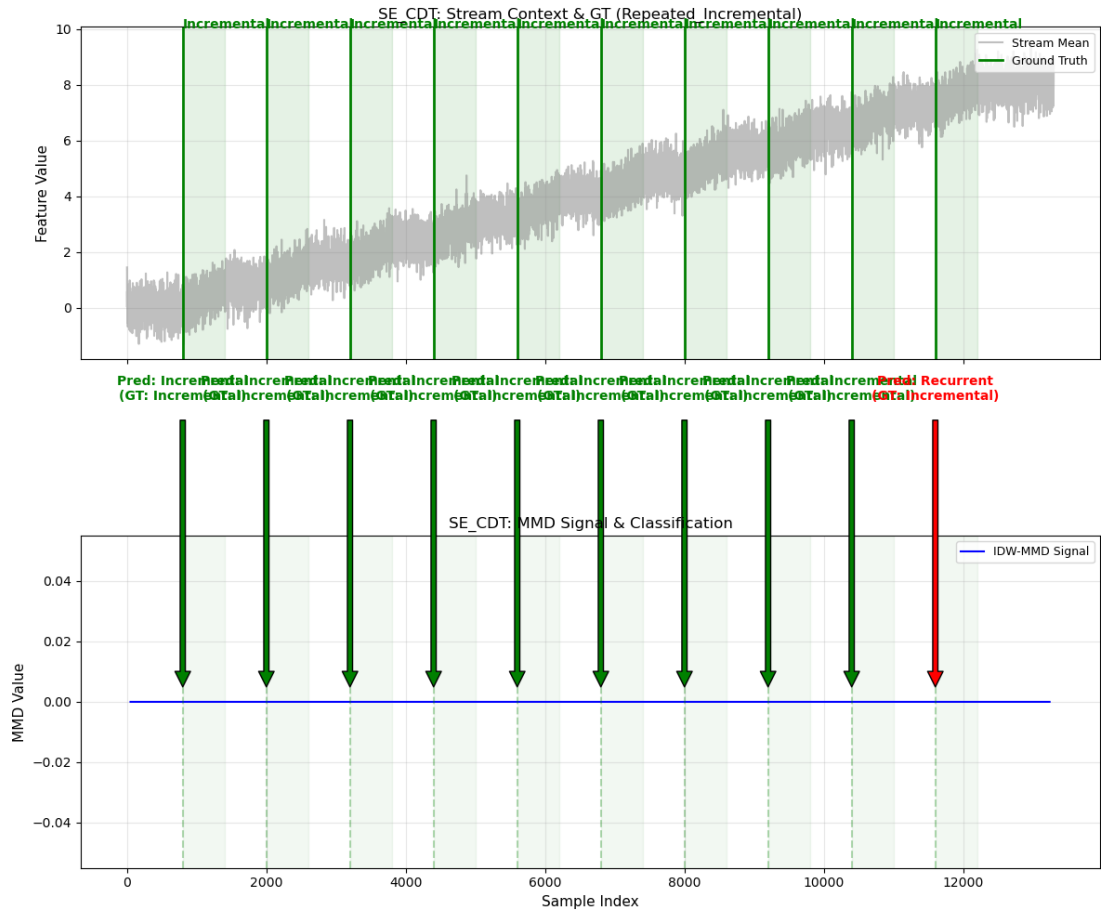
Trong đó, mỗi sự kiện Sudden, Gradual và Incremental dịch chuyển một tập đặc trưng khác nhau theo cơ chế xoay vòng, để mỗi concept mới là độc lập với các concept đã thấy, đúng theo định nghĩa của Lu và cộng sự [11]. Riêng Recurrent giữ logic luân phiên $A \leftrightarrow B$ vì đây mới là định nghĩa đặc trưng của loại drift này.

- **Cửa sổ ShapeDD:** $W = 50$, stride $s = 10$.
- **Tín hiệu phân loại:** Standard MMD. Các thử nghiệm so sánh cho thấy IDW-MMD không phù hợp làm tín hiệu phân loại vì dễ làm mờ các drift chậm như Gradual/Incremental. Vì vậy hệ thống dùng mô hình kết hợp: bước kiểm định ứng viên dùng IDW-MMD, còn bước phân loại dùng Standard MMD để giữ lại thông tin hình dạng toàn cục.
- **Ngưỡng phát hiện:** hiệu chỉnh trên một tập validation riêng (xem Bảng A.1 và A.2 ở Phụ lục). Ngưỡng mặc định của thư viện gốc quá bảo thủ trong điều kiện streaming nên đã được nới xuống cho phù hợp.

Trước khi đọc kết quả số, Hình 5.6 và Hình 5.7 minh họa trực quan lý do Sudden drift dễ phân loại hơn Incremental: SE-CDT dựa vào sự khác biệt hình dạng của tín hiệu $\sigma(t)$ để phân biệt các loại drift.



Hình 5.6 Tín hiệu SE-CDT trên kịch bản Repeated Sudden: $\sigma(t)$ tạo đỉnh nhọn, đối xứng tại mỗi điểm drift — đặc trưng để nhận diện bằng height threshold.



Hình 5.7 Tín hiệu SE-CDT trên kịch bản Repeated Incremental: $\sigma(t)$ tăng dần theo dạng ramp, không có đỉnh nhọn. Hình dạng này khó phân biệt bằng đặc trưng MMD đơn thuần; đặc trưng Variance Ratio trên dữ liệu thô được dùng để tách Incremental khỏi Gradual, song hai loại drift chậm này vẫn chồng lẫn đáng kể về phương sai của số.

5.3.2 Kết quả phân loại tổng hợp của SE-CDT

Bảng 5.11 trình bày kết quả tổng hợp trên 1800 sự kiện phân loại ở trên. Hai cấu hình đều dùng Standard MMD cho tín hiệu phân loại nên CAT/SUB bằng nhau, khác biệt chỉ nằm ở tín hiệu detection. SE-CDT (Std) dùng Standard MMD, nhạy với cả 5 loại drift (EDR $\approx 92.6\%$) nhưng FP cao hơn. SE-CDT (IDW) dùng IDW-MMD, chặt hơn về FP nhưng bỏ sót Gradual/Incremental. Trong ngữ cảnh phân loại, ưu tiên là không bỏ sót sự kiện, vì vậy **SE-CDT (Std)** được chọn làm cấu hình chính.

Bảng 5.11 Kết quả phân loại drift type của SE-CDT (tổng hợp 1800 sự kiện, 6 kịch bản $\times$ 30 seeds)

Method	CAT Acc	SUB Acc	EDR $\uparrow$	MDR $\downarrow$	FP	Supervised
SE-CDT (Std)	80.1%	55.3%	0.926	0.074	4846	No
SE-CDT (IDW)	80.1%	55.3%	0.567	0.433	657	No

Kết quả ở Bảng 5.11 cho thấy một sự chênh lệch giữa độ chính xác phân nhóm mức

lớn (CAT Acc = 80.1%, phân biệt giữa drift nhanh TCD và drift chậm PCD) và độ chính xác phân loại chi tiết tiểu loại (SUB Acc = 55.3% micro, 54.4% macro). Đặc trưng Variance Ratio (Eq. (4.6)) bổ sung thông tin phương sai dữ liệu thô cho nhánh PCD bên cạnh các đặc trưng hình học. Tuy vậy, độ chênh CAT-SUB vẫn còn, tập trung gần như hoàn toàn ở hai tiểu loại drift chậm, và xuất phát từ một số lý do kỹ thuật cùng giới hạn lý thuyết cốt lõi của phương pháp tiếp cận không giám sát:

- **Hai tiểu loại drift chậm là khó nhất (Gradual 27.2%, Incremental 30.3%):** Cả Gradual và Incremental đều thuộc nhóm PCD và để lại tín hiệu MMD kéo dài, ít đỉnh nhọn (Hình 5.7), nên chúng nhầm lẫn lẫn nhau qua trục Variance Ratio. Về lý thuyết, chuyển tiếp hỗn hợp của Gradual làm phương sai của số thành phần ($VR > \tau_{VR}^+$) còn dịch chuyển đơn của Incremental giữ phương sai ổn định ($VR < \tau_{VR}^-$); nhưng trên các luồng mô phỏng, phân bố VR của hai loại chồng lẫn đáng kể, dẫn tới 47% sự kiện Incremental bị đọc thành Gradual và một phần Gradual bị gán ngược lại (Hình 5.8). Variance Ratio nâng khả năng tách tiểu loại PCD so với chỉ dùng đặc trưng hình học, song chưa đủ để phân biệt dứt khoát hai dạng drift chậm.
- **Drift chậm lặp lại bị hút vào nhánh Recurrent:** Trong các kịch bản nhiều sự kiện Gradual/Incremental liên tiếp, các dịch chuyển tích lũy dần đến khi $P(X)$ ổn định; cửa sổ sau drift khi đó trùng khớp với một concept đã lưu và bị Concept Memory gán nhãn Recurrent. Có tới 57% sự kiện Gradual rơi vào trường hợp này, khiến Gradual chỉ đạt 27.2% trên toàn benchmark dù vẫn nhận diện được khi đứng độc lập. Đây là hệ quả của việc Concept Memory ưu tiên bản chất lặp lại của phân phối hơn hình dạng chuyển tiếp danh nghĩa.
- **Sự đánh đổi nghiêm trọng về báo động giả (FP):** Cấu hình chính SE-CDT (Std) ghi nhận lượng báo động giả cực kỳ lớn (4846 FP trên toàn bộ các lần chạy thử nghiệm). Điều này là do Standard MMD rất nhạy với các biến động nhiễu tần số cao của luồng dữ liệu. Khi chuyển sang cấu hình SE-CDT (IDW) để siết chặt FP xuống còn 657, bộ phát hiện lại bỏ sót hầu hết các sự kiện drift chậm (EDR giảm mạnh từ 92.6% xuống còn 56.7%). Đây là một giới hạn lớn: hệ thống hoặc phải chấp nhận alert fatigue (quá nhiều báo động giả) để phân loại đầy đủ các lớp drift, hoặc phải hy sinh khả năng nhận diện các loại drift chậm để giữ hệ thống ổn định.
- **Tính heuristic của các ngưỡng quyết định:** Việc phân loại hoàn toàn không giám sát dựa trên một cây quyết định tĩnh với các ngưỡng cứng ($WR < 0.15$, $SNR > 2.0$, v.v.). Các ngưỡng này chưa có cơ sở toán học chặt chẽ giống như

bước phát hiện (vốn được chuẩn hóa bằng p-value thông qua xấp xỉ Gamma), dẫn đến mô hình kém linh hoạt khi xử lý các kịch bản dữ liệu luồng đa dạng.

SE-CDT (Std) đạt $CAT = 80.1\%$ và $SUB \text{ micro-average} = 55.3\%$ ở oracle mode. Bảng 5.12 cũng báo cáo macro-average per-class = 54.4% để tránh việc kết quả bị chi phối bởi lớp xuất hiện nhiều hơn trong benchmark. Nhóm TCD (Sudden, Blip, Recurrent) ổn định hơn hẳn nhóm PCD, phản ánh việc tín hiệu chuyển tiếp chậm khó tách tiểu loại hơn khi chỉ dựa vào hình dạng tín hiệu không giám sát.

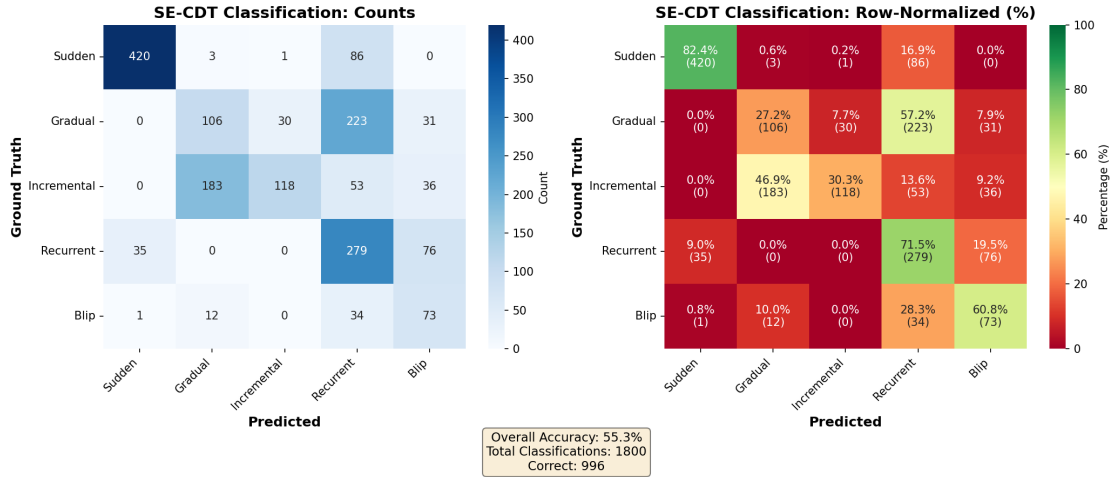
Concept Memory giúp Recurrent đạt 71.5% , thấp hơn kết quả recurrent mà CDT-MSW công bố trên benchmark UCI khác (83.9%) nhưng vẫn cho thấy cơ chế so khớp khái niệm có tác dụng. Tuy nhiên điều này cũng làm lộ ra sự đánh đổi: một phần Gradual và Incremental bị hút vào nhánh Blip/Recurrent khi tín hiệu có nhiều đỉnh phụ.

5.3.3 Phân tích chi tiết theo loại drift

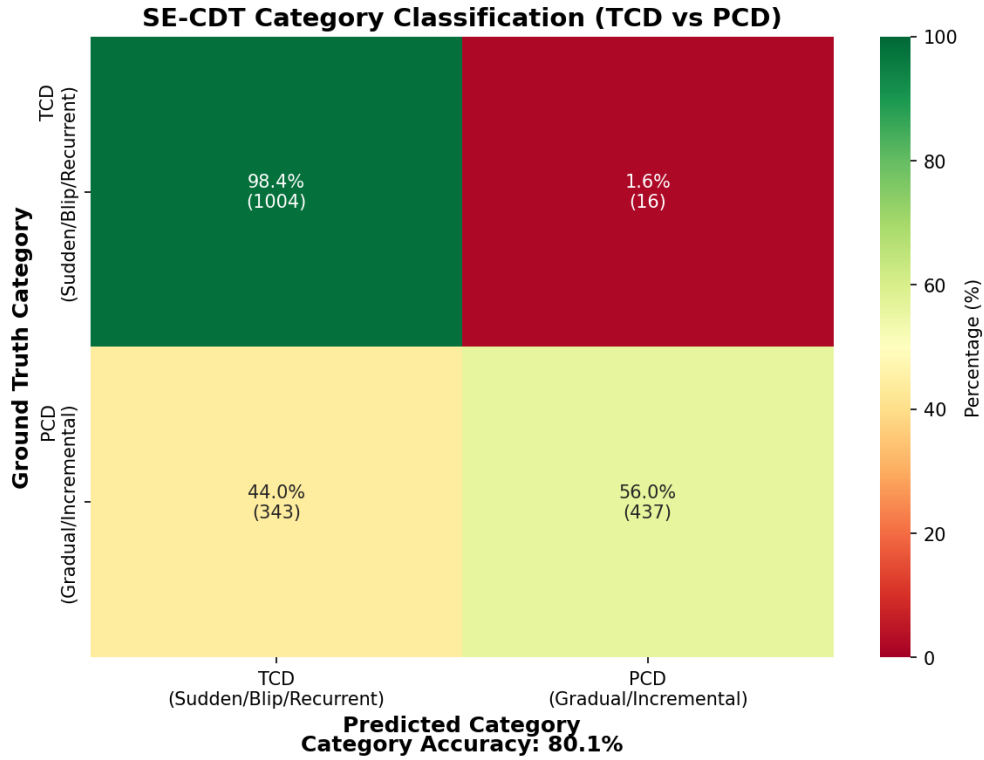
Bảng 5.12 và Hình 5.8, 5.9 phân tích nguồn gốc lỗi phân loại.

Bảng 5.12 Độ chính xác phân loại theo từng loại drift (oracle mode, SE-CDT Std)

Drift Type	Accuracy [%]
Sudden	82.4%
Blip	60.8%
Gradual	27.2%
Incremental	30.3%
Recurrent	71.5% [†]
Macro avg (per-class)	54.4%
Micro avg (over events)	55.3%
[†] Recurrent drift identified via Concept Memory (Standard MMD comparison with stored snapshots; $\tau_{\text{match}} = 0.15$). Macro avg averages the five per-class accuracies (each weighted equally); micro avg pools all events and counts correct predictions, so it is biased toward the most frequent class (here Sudden).	



Hình 5.8 Confusion Matrix cho 5 loại drift (oracle mode, cấu hình SE-CDT đầy đủ với Concept Memory và Online Self-Calibration). Chi tiết độ chính xác từng lớp xem Bảng 5.12.



Hình 5.9 Confusion Matrix ở mức nhóm (TCD vs PCD, oracle mode). Độ chính xác nhóm đạt 80.1%; drift chậm (Gradual, Incremental) tiếp tục chồng lấn tín hiệu nhiều hơn drift nhanh (Sudden, Blip).

Sudden đạt 82.4%, vẫn là nhóm ổn định nhất vì tín hiệu MMD tạo đỉnh nhọn và SNR cao. Blip đạt 60.8% sau khi luật phân loại cho phép hai hoặc ba peak và kiểm tra profile nhiễu ngắn hạn; kết quả này tốt hơn phiên bản cứng trước đó nhưng chưa ngang mức supervised trong CDT-MSW. Recurrent đạt 71.5% nhờ Concept Memory, song vẫn còn nhầm với Blip/Sudden khi mẫu sau drift giống một concept đã lưu.

Hai nhóm drift chậm (PCD) là nơi tập trung phần lớn lỗi phân loại. Incremental đạt 30.3% và Gradual đạt 27.2% — thấp hơn rõ rệt so với các tiểu loại TCD. Confusion Matrix (Hình 5.8) cho thấy lỗi không phân tán ngẫu nhiên mà tập trung vào hai cơ chế nhầm lẫn có hệ thống.

Thứ nhất, Gradual và Incremental nhầm lẫn lẫn nhau qua trục Variance Ratio. Về lý thuyết, chuyển tiếp hỗn hợp của Gradual làm phương sai cửa sổ phình to còn dịch chuyển đơn điệu của Incremental giữ phương sai gần như không đổi (Eq. (4.6)); nhưng trên dữ liệu mô phỏng, cả hai cơ chế đều tạo một mức phình phương sai trung gian trong cửa sổ chuyển tiếp, làm phân bố VR của hai loại chồng lấn. Hệ quả là 47% sự kiện Incremental bị gán thành Gradual, và một phần Gradual bị gán ngược lại thành Incremental. Thứ hai, với chuỗi nhiều sự kiện drift chậm liên tiếp, các dịch chuyển trên những tập đặc trưng bị thay đổi tích lũy đến khi $P(X)$ bão hòa; cửa sổ sau drift khi đó trùng với một concept đã lưu nên Concept Memory gán nhãn Recurrent — chiếm tới 57% số sự kiện Gradual. Hai cơ chế này giải thích vì sao Gradual và Incremental tuy vẫn nằm đúng nhóm PCD (giữ CAT ở mức 80.1%) nhưng độ chính xác ở mức tiểu loại lại thấp.

Đây là một giới hạn cố hữu của việc phân loại tiểu loại drift chậm hoàn toàn không giám sát từ hình dạng tín hiệu: khi không có nhãn y , ranh giới giữa Gradual và Incremental — cũng như giữa một chuỗi drift chậm lặp lại với Recurrent — trở nên nhập nhằng về mặt thống kê. CDT-MSW, nhờ sử dụng tín hiệu độ chính xác có giám sát, tách các lớp này tốt hơn nhiều (Gradual 96.9%, Incremental 74.0%; Bảng 5.13). Việc thu hẹp khoảng cách này — chẳng hạn bằng một bộ phân loại học từ dữ liệu thay cho cây quyết định ngưỡng cứng, hoặc tách bước so khớp Concept Memory khỏi bước phân loại hình dạng — được để lại như một hướng phát triển.

5.3.4 So sánh với CDT-MSW

Bảng 5.13 so sánh với CDT-MSW [5], một phương pháp supervised sử dụng label và dữ liệu UCI thực tế.

Bảng 5.13 So sánh độ chính xác phân loại theo từng loại drift giữa CDT-MSW (Guo et al. 2022) và SE-CDT. Đây là đối chiếu tham khảo, không phải so sánh đồng điều kiện: CDT-MSW chạy có giám sát trên 10 tập UCI thực tế với ACC_{subcat} tính có điều kiện trên các drift được phát hiện và phân nhóm đúng; SE-CDT chạy không giám sát trên 6 kịch bản Gaussian 5-D (Mục 5.3.1) với SUB tính trên toàn bộ ground truth. Cần lưu ý thêm rằng “EDR” trong [5] viết tắt cho *Error Detection Rate* (càng thấp càng tốt), khác hẳn với “EDR” được dùng trong luận văn (*Event Detection Rate*, tương đương Recall), nên hai chỉ số trùng tên này không so sánh trực tiếp được với nhau.

Drift Type	CDT-MSW (Paper) [%]	SE-CDT (Ours) [%]
Sudden	90.2%	82.4%
Blip	88.5%	60.8%
Gradual	96.9%	27.2%
Incremental	74.0%	30.3%
Recurrent	83.9%	71.5% [†]

[†] Recurrent drift identified via Concept Memory (Standard MMD comparison with stored snapshots; threshold $\mu = 0.15$).

Vì hai cột vận hành trên hai phân phối dữ liệu và hai chế độ giám sát khác nhau, bảng chỉ có ý nghĩa tham khảo. SE-CDT chạy không cần nhãn và đạt recall cao hơn trong thiết lập này, nhưng độ chính xác phân loại tiểu loại vẫn thấp hơn CDT-MSW, đặc biệt ở Gradual và Incremental. Luận văn không khẳng định SE-CDT vượt CDT-MSW trên cùng thiết lập; so sánh trực tiếp cần triển khai SE-CDT trên các tập UCI gốc, được liệt kê là hướng phát triển ở Chương 6.

5.4 Đánh giá khả năng thích ứng mô hình

Sau khi SE-CDT phát hiện và phân loại drift, khung thích ứng dùng nhãn loại drift để chọn chiến lược cập nhật mô hình tương ứng. Phần này đánh giá hiệu quả của chiến lược thích ứng theo loại drift (Type-Specific Adaptation) bằng chỉ số **Prequential Accuracy**.

5.4.1 Thiết lập thực nghiệm thích ứng mô hình

- **Môi trường:** Prequential Evaluation trên luồng dữ liệu mô phỏng.
- **Dữ liệu:** $T = 5,000$ mẫu, $N = 30$ lần chạy độc lập, mật độ drift cao (5 sự kiện/luồng).
- **Kịch bản:** ba loại: Stepping (drift tích lũy), Sudden, và Mixed.
- **Bốn chiến lược thích ứng:**
 - *No Adaptation*: giữ nguyên mô hình ban đầu (cận dưới).
 - *Periodic Retrain*: huấn luyện lại định kỳ mỗi 500 mẫu, không dùng tín hiệu detector.

- *Simple Retrain*: huấn luyện lại trên cửa sổ gần nhất mỗi khi SE-CDT báo drift, bất kể loại nào.
- *Type-Specific (đề xuất)*: chọn một trong năm nhánh thích ứng tùy theo nhãn drift do SE-CDT phân loại (Full Reset cho Sudden, Incremental Fine-tune cho Incremental, Recency-weighted cho Gradual, Model Caching cho Recurrent, và bỏ qua thích ứng cho Blip; xem Mục 4.4).
- **Mô hình dự đoán**: pipeline gồm chuẩn hoá đặc trưng bằng Standard Scaler và phân loại bằng Logistic Regression, với số vòng lặp tối đa đặt là 1 000. Đây là mô hình phân loại được bảo vệ trong thí nghiệm thích ứng; SE-CDT chỉ là tăng giám sát drift, không phải mô hình dự đoán nhãn.
- **Tham số**: initial training = 500 mẫu, adaptation delay = 50 mẫu, periodic interval = 500 mẫu, mức ý nghĩa phát hiện $\alpha = 0.05$ (ngưỡng p-value của kiểm định IDW-MMD), ShapeDD $l_1 = 50$, $l_2 = 150$.
- **Chỉ số**: Prequential Accuracy (độ chính xác cuộn chiếu) đo khả năng phục hồi sau drift; số False Positives (FP) đo chi phí cảnh báo dư thừa.

Luồng dữ liệu mô phỏng được tạo từ hỗn hợp dữ liệu Gaussian năm chiều với năm điểm drift đều nhau. Theo định nghĩa của Lu và cộng sự [11], mỗi sự kiện Sudden, Gradual hoặc Incremental phải dẫn đến một concept mới, còn Recurrent là sự lặp lại có chu kỳ giữa hai concept. Để đáp ứng điều này, bộ sinh dùng cơ chế xoay vòng đặc trưng: mỗi sự kiện non-recurrent dịch chuyển một tập đặc trưng khác nhau, tránh quay về phân phối đã thấy. Riêng Recurrent giữ pattern luân phiên $A \leftrightarrow B$ đúng theo định nghĩa. Kịch bản Stepping bổ sung thêm thành phần dịch chuyển tích lũy theo cùng hướng để mô phỏng môi trường tiến hoá đơn điệu (ví dụ trôi cảm biến công nghiệp).

5.4.2 Phạm vi phát hiện trong thích ứng

Theo giả định joint drift (Mục 2.1.2), các kịch bản thích ứng dưới đây tập trung vào kịch bản có thay đổi rõ ràng ở $P(X)$, nơi tín hiệu của SE-CDT có thể được dùng để kích hoạt cập nhật mô hình.

5.4.3 Kết quả phục hồi (Recovery)

Các bộ tạo dữ liệu Sudden Drift truyền thống thường luân phiên giữa hai concept $A \rightarrow B \rightarrow A$, dẫn đến mô hình tĩnh có độ chính xác ảo $\approx 75\%$. Để khắc phục giới hạn này, luận văn bổ sung kịch bản **Stepping Drift**. Đây là dạng trôi dạt tích lũy

(Cumulative Sudden Drift), nơi phân phối dữ liệu liên tục dịch chuyển ra xa trạng thái ban đầu ($P_0 \rightarrow P_1 \rightarrow P_2 \dots$), buộc mô hình phải học lại liên tục để duy trì hiệu suất.

Ngoài ra, hệ thống tích hợp cơ chế **reset bộ đệm theo mẫu ADWIN** (Adaptive Windowing): sau mỗi lần thích ứng, bộ đệm được xóa và chỉ giữ lại 200 mẫu gần nhất làm tham chiếu mới. Cơ chế này giảm đáng kể hiện tượng phát hiện lại cùng một drift event: Type-Specific và Simple Retrain (cả hai đều dùng tín hiệu detector để kích hoạt) có FP trung bình khoảng 1.5–2.2/run, trong khi Periodic Retrain và No Adaptation (không thay đổi bộ đệm theo detector) có FP trung bình 13.0–17.0/run.

Bảng 5.14 tổng hợp kết quả Prequential Accuracy trên $N = 30$ lần chạy độc lập, so sánh 4 chiến lược: Type-Specific (đề xuất), Simple Retrain, Periodic Retrain và No Adaptation. Số $N = 30$ đủ để kiểm định Friedman có lực thống kê cao và đảm bảo khoảng tin cậy 95% chặt với độ lệch chuẩn quan sát được trên các kịch bản.

Bảng 5.14 Kết quả Prequential Accuracy theo loại drift ($N = 30$ runs, $T = 5000$ mẫu). Cột FP: số lần báo động sai trung bình mỗi run, theo thứ tự chiến lược (Type-Specific / Simple Retrain / Periodic Retrain / No Adaptation).

Loại Drift	Type-Specific	Simple Retrain	Periodic Retrain	No Adaptation	FP
Stepping (Cumulative)	73.96% ± 2.42	62.72% ± 2.60	64.03% ± 2.74	54.80% ± 3.60	1.5/1.5/17.0/17.0
Sudden (Joint Drift)	73.34% ± 2.72	61.87% ± 2.25	63.65% ± 2.37	53.68% ± 3.46	1.5/1.5/17.0/17.0
Mixed (Combined)	86.36% ± 1.25	83.80% ± 0.96	86.09% ± 0.69	82.69% ± 0.94	2.2/2.2/13.0/13.0

Áp dụng kiểm định không tham số Friedman (paired theo seed) trên 4 chiến lược $\times 30$ runs cho mỗi kịch bản:

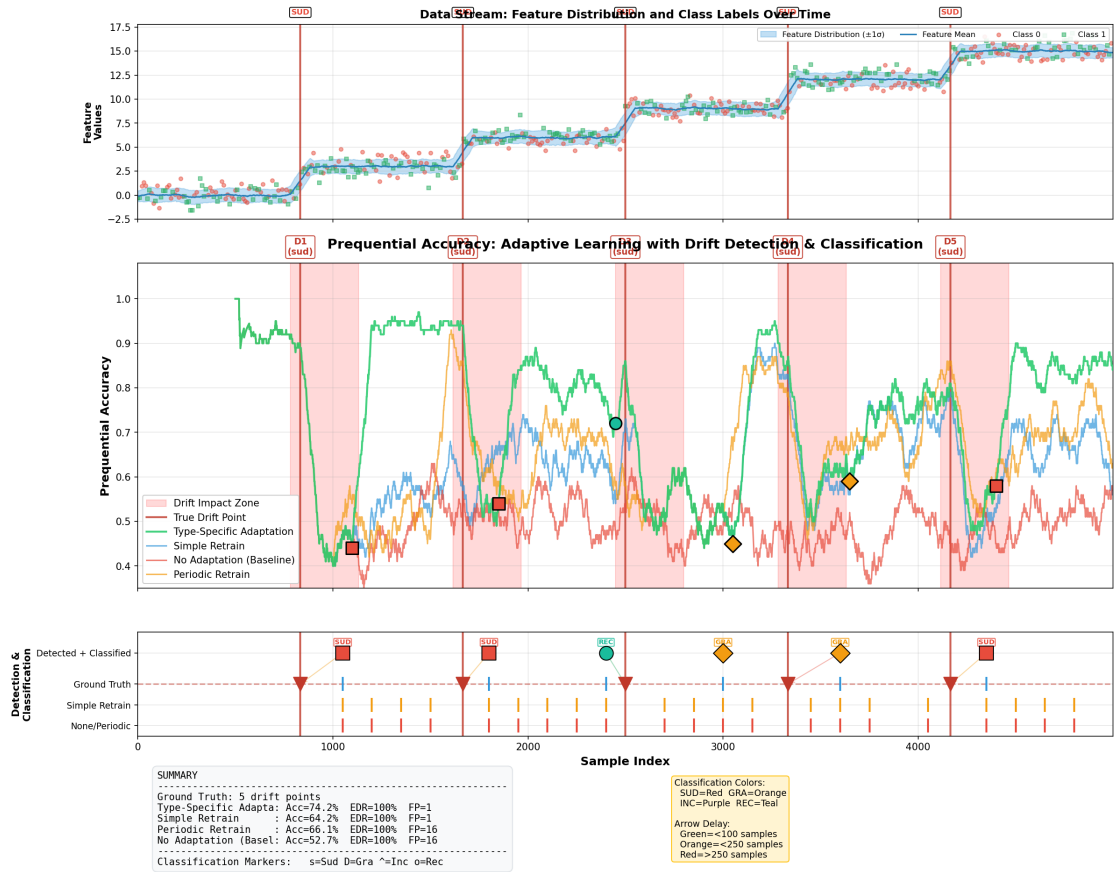
- Stepping: $\chi^2 = 78.76$, $p = 5.66 \times 10^{-17}$
- Sudden: $\chi^2 = 85.48$, $p = 2.05 \times 10^{-18}$
- Mixed: $\chi^2 = 75.16$, $p = 3.35 \times 10^{-16}$

Trên cả ba kịch bản, $p \ll 0.05$ nên có đủ bằng chứng thống kê để bác bỏ giả thuyết H_0 “bốn chiến lược cho cùng phân phối độ chính xác”. Kết quả này xác nhận khác biệt tổng thể giữa các chiến lược; các chênh lệch theo từng cặp bên dưới được diễn giải theo độ lớn quan sát được, không thay cho một kiểm định hậu nghiệm riêng cho từng cặp.

Trên Stepping, Type-Specific đạt 73.96%, hơn No Adaptation +19.16pp, Simple Retrain +11.24pp và Periodic Retrain +9.93pp. Trên Sudden, kết quả tương tự: Type-Specific đạt 73.34%, hơn No Adaptation +19.66pp, Simple Retrain +11.47pp và Periodic Retrain +9.69pp. Hai kịch bản này cho thấy lợi ích rõ của việc chọn chiến lược cập nhật theo loại drift thay vì chỉ huấn luyện lại theo một quy tắc cố định.

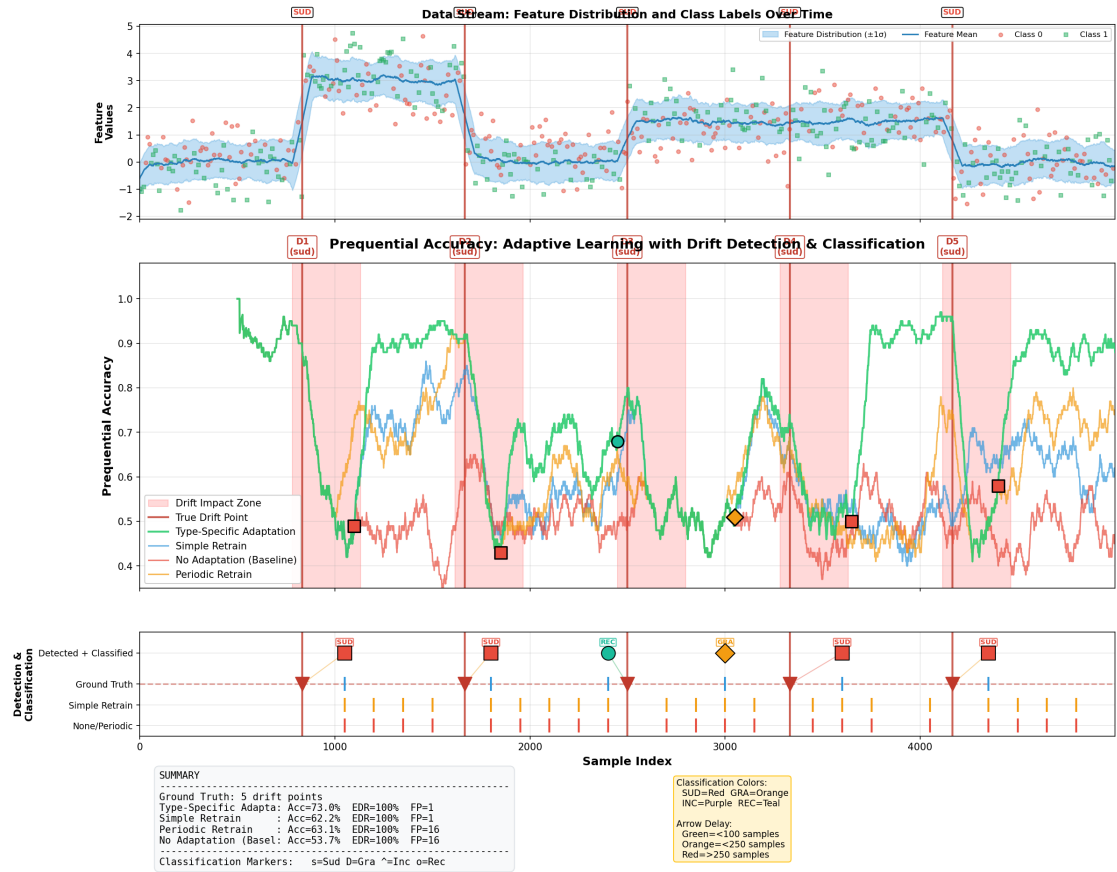
Trên Mixed, Type-Specific đạt 86.36%, rất gần Periodic Retrain (86.09%) và cao hơn Simple Retrain/No Adaptation. Chênh lệch +0.27pp so với Periodic Retrain nhỏ

so với độ lệch chuẩn, nên không nên đọc như một chiến thắng chắc chắn theo từng cặp; điểm đáng chú ý hơn là FP của Periodic Retrain cao hơn rõ rệt (13.0 so với 2.2/run). Do đó, lợi thế chính của Type-Specific trên Mixed là đạt mức độ chính xác gần Periodic Retrain với ít cảnh báo dư hơn.



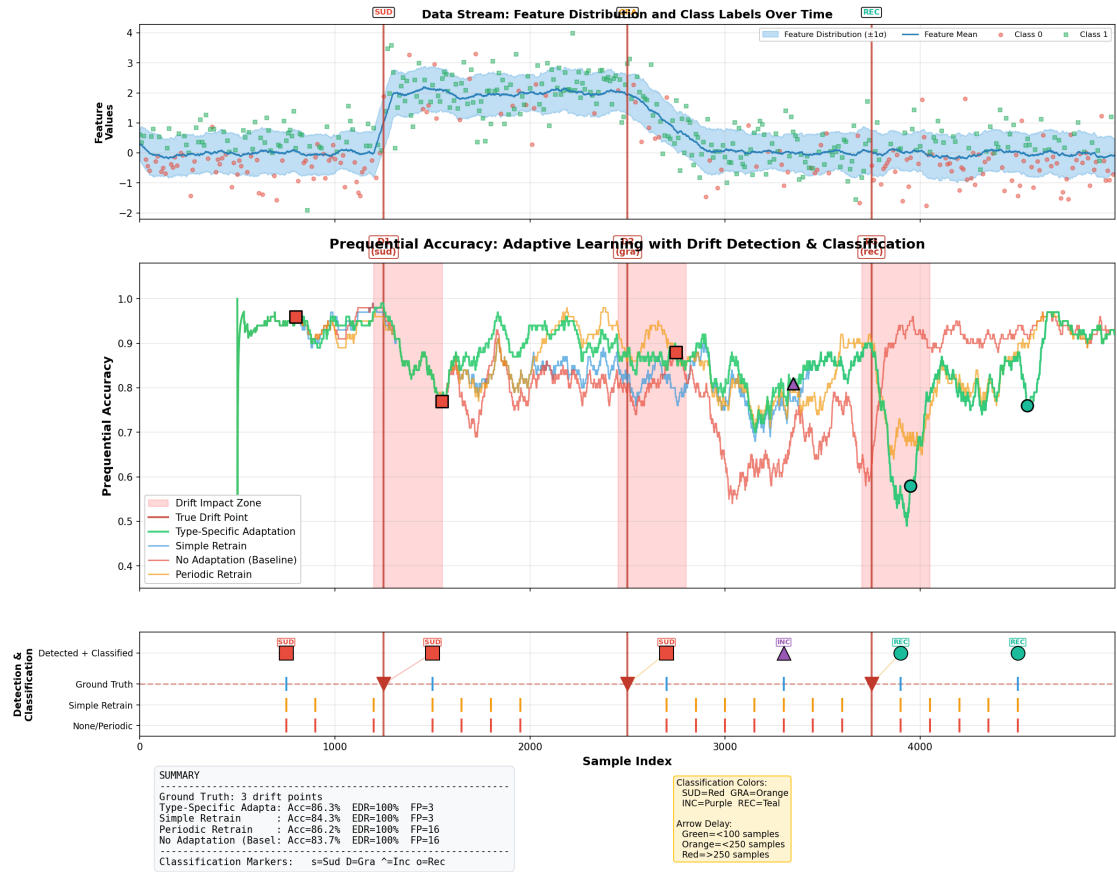
Hình 5.10 Prequential Accuracy trên Stepping Drift ($N = 30$). No Adaptation (đỏ) tụt đều về $\sim 55\%$ khi phân phối tiếp tục dịch chuyển. Type-Specific (xanh) giữ khoảng 74% nhờ chọn chiến lược thích ứng theo loại drift.

Trên Sudden Drift (Hình 5.11), No Adaptation giảm về 53.68% do toàn bộ phân phối thay đổi sau mỗi sự kiện. Type-Specific đạt 73.34%, cách Simple Retrain +11.47pp và Periodic Retrain +9.69pp. Cả hai kịch bản Stepping (Hình 5.10) và Sudden đều cho thấy cải thiện lớn khi chiến lược cập nhật được chọn theo loại drift.



Hình 5.11 Prequential Accuracy trên Sudden Drift ($N = 30$). No Adaptation giảm về $\sim 54\%$; Type-Specific duy trì khoảng 74% nhờ chọn Full Reset cho từng sự kiện Sudden mới.

Trên Mixed (Hình 5.12), bốn chiến lược đều đạt kết quả cao hơn hai kịch bản trên do luồng dữ liệu chứa hỗn hợp các loại drift có mức khó khác nhau. Type-Specific đạt 86.36%, gần với Periodic Retrain (86.09%) nhưng với FP thấp hơn (2.2 so với 13.0/run). Đây là kết quả gần hòa về độ chính xác theo nghĩa thực nghiệm, nhưng Type-Specific ít kích hoạt thừa hơn.



Hình 5.12 Prequential Accuracy trên Mixed Drift ($N = 30$). Type-Specific và Periodic Retrain gần tương đương về độ chính xác trong artifact hiện tại, nhưng Type-Specific có số cảnh báo dư thấp hơn.

5.5 Kiểm thử nguyên mẫu trên kiến trúc tương thích Kafka

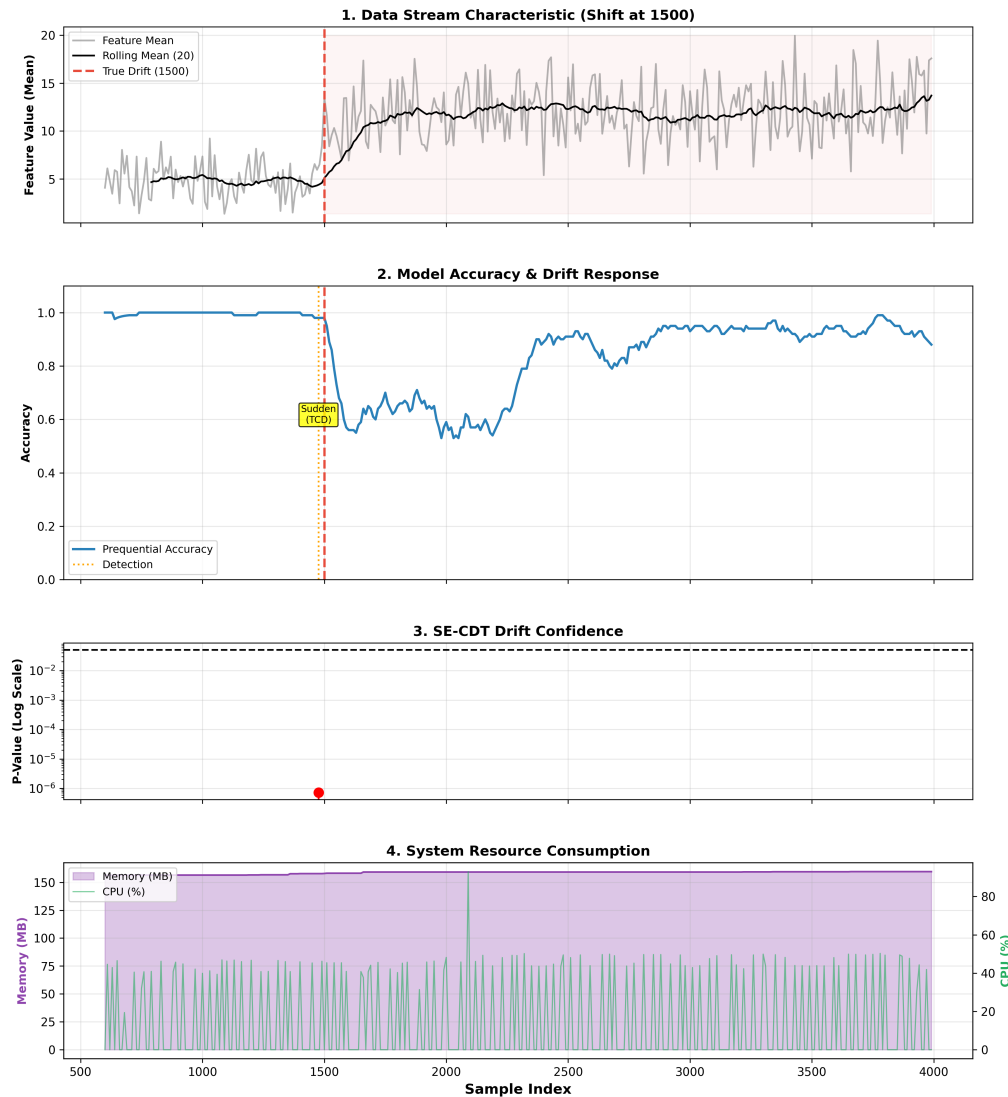
Phạm vi của nguyên mẫu Kafka đã mô tả ở Mục 4.5 (lưu ý về phạm vi triển khai); phần này chỉ báo cáo kết quả kiểm thử khép kín giữa các thành phần Redpanda, SE-CDT Consumer và Adaptation Manager trên Docker.

5.5.1 Kịch bản triển khai

- **Dữ liệu:** Luồng dữ liệu mô phỏng từ SEA dataset với Sudden Drift tại mẫu thứ 1500.
- **Cấu hình:** Cửa sổ phát hiện $W = 250$ mẫu, độ trễ thích ứng $\Delta = 50$ mẫu (để thu thập đủ dữ liệu sau điểm drift trước khi cập nhật).
- **Mục tiêu:** Xác nhận quy trình khép kín: Phát hiện Drift → chờ thu thập dữ liệu mới → cập nhật mô hình → nạp lại mô hình → phục hồi độ chính xác.

5.5.2 Kết quả thực tế

Hình 5.13 trình bày dashboard giám sát từ lần chạy nguyên mẫu:



Hình 5.13 Dashboard giám sát nguyên mẫu tương thích Kafka. (1) Dữ liệu luồng với điểm drift tại mẫu 1500. (2) Độ chính xác giảm xuống 0.59 tại điểm drift, sau đó phục hồi lên 0.93 sau khi mô hình được cập nhật tự động. (3) Tín hiệu SE-CDT phát hiện Sudden Drift. (4) Tài nguyên hệ thống (CPU/RAM) duy trì ổn định trong lần chạy thử.

- **Độ trễ phát hiện:** Drift được đặt tại mẫu 1500 trong dữ liệu. ShapeDD xác định điểm drift tại mẫu 1476. Đây là vị trí đỉnh tam giác ước lượng (triangle peak

position) từ tín hiệu tích chập ShapeDD, không phải thời điểm xử lý thực. Do cơ chế cửa sổ trượt và lọc hình dạng, vị trí drift báo cáo có thể lệch so với nhãn danh nghĩa $\pm l_1$ (với $l_1 = 50$ là kích thước nửa cửa sổ reference), nằm trong giới hạn chấp nhận của ShapeDD.

- **Quy trình khép kín:** Log hệ thống xác nhận Consumer đã nhận tín hiệu cập nhật từ bộ thích ứng và tải lại mô hình thành công tại mẫu 2277.
- **Hiệu quả phục hồi:** Độ chính xác tăng từ mức 59% (gần ngẫu nhiên) lên 93% ngay sau khi mô hình mới được tải, khớp với hành vi mong đợi của quy trình thích ứng.

5.6 Kết luận chương

Các kết quả thực nghiệm cho thấy ShapeDD-IDW đạt $F_1 = 0.531$ với khoảng 9.6 FP/run trên benchmark phát hiện, cao hơn ShapeDD gốc ($F_1 = 0.491$) và không khác biệt thống kê so với DAWIDD dưới kiểm định Nemenyi. Về tốc độ, benchmark cho thấy ShapeDD-IDW nhanh hơn $\approx 7.2\times$ so với ShapeDD gốc nhờ xấp xỉ Gamma thay thế permutation test tốn kém, trong khi kiểm định IDW-MMD giữ Type-I error đơn lẻ gần mức $\alpha = 0.05$; pipeline SE-CDT composite vẫn cần được đọc thận trọng vì Type-I error có thể lên đến 0.140 trên dữ liệu AR(1).

Ở bài toán phân loại, SE-CDT đạt CAT = 80.1%, SUB micro = 55.3% và SUB macro = 54.4% ở oracle mode. Kết quả tốt nhất nằm ở Sudden, Blip và Recurrent; hai tiểu loại drift chậm Gradual và Incremental là khó nhất do tín hiệu chuyển tiếp chồng lấn về phương sai và một phần bị Concept Memory hút sang nhánh Recurrent. Trong đánh giá thích ứng, Type-Specific cải thiện rõ trên Stepping và Sudden, còn trên Mixed thì gần ngang Periodic Retrain về độ chính xác nhưng ít FP hơn. Nhìn chung, kiến trúc đề xuất khả thi, nhưng các kết quả cũng chỉ ra rõ những phần cần cải thiện trước khi xem đây là một bộ phân loại drift hoàn chỉnh.

Benchmark detection tập trung chủ yếu vào sudden/abrupt drift và blip, đúng với trọng tâm thiết kế của ShapeDD gốc [4] là phát hiện thay đổi phân phối đột ngột. Các loại drift chậm hơn (gradual, incremental) và drift lặp lại (recurrent) được đưa vào benchmark phân loại như mở rộng năng lực, không phải trọng tâm tối ưu. Kết quả phân loại theo từng loại drift (Bảng 5.12) phản ánh đúng độ khó vốn có và mở ra hướng cải tiến cho các công trình tiếp theo.

CHƯƠNG 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Tổng kết các đóng góp chính

Luận văn giải quyết bài toán phát hiện, phân loại và thích ứng với concept drift trong môi trường dữ liệu luồng. Bảng 6.1 tóm tắt các số liệu chính; phần sau diễn giải ý nghĩa và giới hạn của các kết quả này.

Bảng 6.1 Tóm tắt số liệu chính của luận văn theo ba trục đóng góp. Chi tiết và phân tích thống kê trong Chương 5.

Trục	Kết quả chính	Đối chứng
Module phát hiện của SE-CDT (Mục 5.2)	ShapeDD-IDW đạt $F_1 = 0.531$, không khác biệt thống kê với DAWIDD; runtime nhanh hơn ShapeDD gốc khoảng $7\times$	ShapeDD gốc dùng permutation; KS-Test có Recall cao nhưng FP nhiều
Module phân loại của SE-CDT (Mục 5.3)	Đạt CAT = 80.1%, SUB micro = 55.3%, SUB macro = 54.4%; nhóm TCD ổn định, hai tiểu loại drift chậm Gradual/Incremental là khó nhất	CDT-MSW (Guo et al.) dùng nhãn y và không cùng thiết lập dữ liệu
Thích ứng (Mục 5.4)	Type-Specific Adaptation cải thiện rõ trên Sudden/Stepping; trên Mixed gần Periodic Retrain về độ chính xác nhưng ít FP hơn	No Adaptation, Periodic Retrain, Simple Retrain

Hệ thống SE-CDT gồm hai module nối tiếp: module phát hiện ShapeDD-IDW thay permutation test đầy đủ bằng xấp xỉ Gamma cho IDW-MMD và áp dụng hiệu chỉnh Bonferroni khi một trace có nhiều đỉnh ứng viên; module phân loại đọc hình dạng tín hiệu MMD do bước phát hiện sản xuất để xuất nhãn loại drift mà không cần nhãn y . Kiểm tra trên dữ liệu tĩnh cho thấy IDW-MMD đơn lẻ gần mức danh nghĩa $\alpha = 0.05$, dù pipeline SE-CDT composite vẫn có thể cao hơn trên một số phân phối do bước chọn đỉnh và cửa sổ trượt. Kết quả phân loại đủ dùng cho một số quyết định thích ứng, nhưng chưa đạt mức ổn định của phương pháp có giám sát.

Benchmark được mở rộng với dữ liệu recurrent đối xứng $A \rightarrow B \rightarrow A \rightarrow B$, gradual drift có khoảng chuyển tiếp xác định, và dữ liệu stationary để đo báo động giả. Nhờ 30

lần chạy độc lập và kiểm định Friedman/Nemenyi, các kết luận về thứ hạng detection được đọc thận trọng hơn: một số chênh lệch số học không đủ để kết luận khác biệt thống kê. Ở phần hệ thống, nguyên mẫu Kafka/Redpanda cho thấy các module phát hiện, phân loại và thích ứng có thể tách rời và phối hợp được trong một pipeline streaming khép kín.

6.2 Hạn chế và Phạm vi nghiên cứu

Bên cạnh kết quả đạt được, nghiên cứu vẫn còn một số hạn chế:

6.2.1 Trade-off giữa Recall và False Positives

SE-CDT (Std) ưu tiên EDR cao và chấp nhận FP nhiều hơn; SE-CDT (IDW) ngược lại, FP thấp hơn nhưng cũng giảm EDR (số liệu cụ thể ở Mục 5.2 và Mục 5.3). Hai cấu hình phục vụ hai mục tiêu sử dụng khác nhau: nếu ưu tiên không bỏ sót drift trong giám sát vận hành thì dùng SE-CDT (Std), còn nếu chi phí phản ứng cao và cần giảm báo động giả thì dùng SE-CDT (IDW) với ngưỡng chặt hơn. Muốn giảm FP thêm có thể siết α hoặc hiệu chỉnh đa kiểm định theo số đỉnh ứng viên, đổi lại recall có thể giảm.

6.2.2 Subcategory Accuracy còn hạn chế

Category Accuracy (TCD vs PCD) đạt 80.1%, còn Subcategory Accuracy đạt 55.3% theo micro-average và 54.4% theo macro-average. Hai tiểu loại drift chậm là khó nhất: Gradual đạt 27.2% và Incremental đạt 30.3%. Đặc trưng Variance Ratio (dựa trên Định luật Tổng Phương Sai) bổ sung thông tin phương sai dữ liệu thô để tách hai loại này, nhưng phân bố VR của chúng chồng lấn đáng kể nên gần một nửa sự kiện Incremental bị gán thành Gradual; đồng thời với chuỗi drift chậm lặp lại, cửa sổ sau drift bão hoà và bị Concept Memory gán sang Recurrent. Cả hai vẫn nằm đúng nhóm PCD nên Category Accuracy không bị ảnh hưởng. Recurrent đạt 71.5% nhờ Concept Memory nhưng vẫn phụ thuộc vào ngưỡng so khớp τ_{match} và kích thước snapshot n_s . Blip đạt 60.8% sau khi nối luật, song cần kiểm chứng thêm trên dữ liệu thực tế.

Nguyên nhân gốc là CDT-MSW [5] dùng Growth process dựa trên độ chính xác có nhãn để phân biệt TCD và PCD, trong khi SE-CDT chỉ quan sát tín hiệu MMD của $P(X)$. Vì MMD được đo trên cửa sổ trượt khá rộng, quá trình chuyển đổi có thể bị nén thành các đoạn tín hiệu tương tự nhau. Khi không có nhãn, khả năng phân biệt

tiểu loại theo tốc độ drift thấp hơn so với phương pháp có giám sát là điều cần chấp nhận.

6.2.3 Ngưỡng phân loại vẫn còn yếu tố heuristic

Các ngưỡng cứng trong luật phân loại SE-CDT (Algorithm 3) như $WR < 0.15$, $SNR > 2.0$, $CV < 0.3$, $LTS > 0.5$, v.v. vẫn được chọn dựa trên quan sát thực nghiệm chứ chưa suy ra từ phân tích thống kê lý thuyết. So với bước phát hiện (dùng p-value dựa trên xấp xỉ Gamma có cơ sở từ [45]), bước phân loại chưa có công cụ lý thuyết tương đương ở mức từng đặc trưng đơn lẻ. Các biện pháp đã áp dụng để giảm phụ thuộc heuristic.

1. **Tự hiệu chỉnh ngưỡng SNR/WR/CV** (Mục 4.3.6): nối ngưỡng cứng theo phân vị quan sát trên cửa sổ không drift, giúp SE-CDT chịu được nền nhiễu cao hơn mà không cần tinh chỉnh thủ công cho từng dataset.
2. **Kiểm tra Type-I error** (Mục 5.2.5): kiểm chứng IDW-MMD với xấp xỉ Gamma có Type-I error gần mức danh nghĩa trên ba phân phối tĩnh; đồng thời chỉ ra SE-CDT composite có thể cao hơn nhẹ do kiểm định nhiều đỉnh ứng viên.
3. **Loại bỏ các hằng số tuyệt đối trong Blip và Growth process** (Mục 4.3): điều kiện Blip chỉ dựa trên các đặc trưng tỉ lệ, không gán với ngưỡng khoảng cách peak cố định; tham số trong Growth process được biểu diễn theo tỉ lệ với cửa sổ tham chiếu l_1 thay cho hằng số dạng phần ba độ rộng FWHM.

Tuy nhiên, vẫn còn tồn đọng những vấn đề hạn chế như sau.

1. Self-calibration chỉ có thể nối ngưỡng, chưa thay được luật phân loại bằng một mô hình xác suất, vì vậy vẫn không xuất confidence/p-value cho mỗi nhãn drift.
2. Ngưỡng cho LTS, MS, SDS vẫn cố định, chưa được tự hiệu chỉnh. Ba đặc trưng này có miền giá trị nhỏ và nhạy với kích thước cửa sổ, nên việc rút phân vị từ dữ liệu nền không drift dễ tạo ra giá trị kém ổn định.
3. Concept Memory vẫn cần ngưỡng so khớp $\tau_{\text{match}} = 0.15$ chọn theo thực nghiệm (đặt ở mức khoảng $2 \times$ nhiễu MMD nền). Khi snapshot tích lũy đủ gần với một sự kiện Sudden mới, ngưỡng này có thể dán nhãn nhầm thành Recurrent. Hướng cho ngưỡng tự thích nghi theo phân phối được trình bày ở Mục 6.3.2.

Phân phân loại hiện tại đã có khả năng tự nối một số ngưỡng theo dữ liệu quan sát khi vận hành, nhưng chưa xuất được độ tin cậy cho mỗi nhãn. Để bước phân loại đạt mức xác suất đầy đủ cần các nghiên cứu tiếp theo, được trình bày ở phần Hướng phát triển.

6.2.4 Giả định joint drift trong phân loại

SE-CDT phân loại hình dạng thay đổi của $P(X)$ dưới giả định joint drift (Mục 2.1.2); các tình huống chỉ có $P(Y|X)$ thay đổi hoặc chỉ có $P(X)$ thay đổi mà ranh giới quyết định giữ nguyên đều nằm ngoài phạm vi thiết kế. Đây là giới hạn chung của mọi phương pháp unsupervised drift detection, nên trong triển khai thực tế cần kết hợp SE-CDT với giám sát hiệu suất mô hình khi nhận có sẵn, dù có độ trễ.

6.2.5 Dữ liệu tổng hợp (Synthetic Data)

Đánh giá chủ yếu dựa trên các tập dữ liệu tổng hợp. Dữ liệu tổng hợp cho phép kiểm soát chính xác ground truth (thời điểm drift, loại drift) để đo F1-score và độ trễ, nhưng có thể chưa phản ánh hết độ phức tạp, nhiễu và tính bất định của dữ liệu thực tế trong các hệ thống sản xuất quy mô lớn.

6.2.6 Phạm vi đánh giá thích ứng

Đánh giá thích ứng ở Chương 5 dùng 4 phương án so sánh (Type-Specific, Simple Retrain, Periodic Retrain, No Adaptation) trên 3 kịch bản (Stepping, Sudden, Mixed) với $N = 30$ lần chạy, $T = 5000$ mẫu, initial training = 500, adaptation delay = 50, periodic interval = 500, và mô hình nền gồm Standard Scaler kết hợp Logistic Regression với số vòng lặp tối đa 1 000. Hướng mở rộng tự nhiên: (1) bổ sung kịch bản Gradual và Incremental drift, (2) đo chi phí tính toán của từng chiến lược trong môi trường triển khai thực tế, và (3) đánh giá trên dữ liệu thực tế có drift ground truth được kiểm chứng.

6.2.7 Nguyên mẫu Kafka chưa phải benchmark production

Nguyên mẫu Kafka/Redpanda trong luận văn chứng minh được luồng xử lý khép kín giữa producer, SE-CDT Consumer và Adaptation Manager, nhưng chưa phải benchmark hạ tầng sản xuất. Các chỉ số cần có trước khi triển khai thực tế gồm độ trễ đầu-cuối dưới tải cao, thông lượng khi tăng số partition, thời gian phục hồi khi broker/consumer lỗi, độ ổn định khi có backpressure và quy trình shadow deployment cho mô hình mới. Vì vậy kết quả nguyên mẫu chỉ chứng minh tính khả thi kiến trúc, không chứng minh năng lực vận hành production-grade.

6.3 Hướng nghiên cứu tương lai

Các hướng tiếp theo tập trung vào phần phân loại, vì đây là mắt xích còn yếu nhất trong pipeline hiện tại.

6.3.1 Mở rộng sang các loại Drift khác nhau

Gradual và Incremental cần một tín hiệu tốt hơn cửa sổ trượt hiện tại. Một hướng hợp lý là dùng cửa sổ tham chiếu đa thang đo: giữ một cửa sổ dài làm baseline ổn định, đồng thời vẫn dùng cửa sổ ngắn để bắt thay đổi nhanh. Khi đó tín hiệu MMD có thể duy trì ở mức cao với drift bền vững và trở về baseline với drift tạm thời, giúp tách Incremental khỏi Gradual rõ hơn. Với Incremental, cũng nên thử nghiệm các chiến lược cập nhật liên tục bằng thư viện River để cân bằng giữa tốc độ thích ứng và độ ổn định mô hình.

Concept Memory có thể được mở rộng bằng cách học biểu diễn snapshot thay vì so sánh trực tiếp cửa sổ thô bằng Standard MMD. Ngưỡng τ_{match} cũng nên được học từ phân phối khoảng cách giữa các snapshot không drift, thay vì đặt cố định. Nếu liên kết được Concept Memory với Model Caching, hệ thống có thể tái sử dụng cả mô hình đã học chứ không chỉ nhận diện rằng một concept cũ đã quay lại.

6.3.2 Cải tiến phân loại SE-CDT

Như đã phân tích trong Mục 6.2.3, bước phân loại của SE-CDT vẫn là phần có cơ sở lý thuyết yếu nhất trong chuỗi xử lý, dù đã được củng cố bằng tự hiệu chỉnh ngưỡng và kiểm tra Type-I error. Các hướng cải tiến tiếp theo: Về bản thân SE-CDT, hướng quan trọng nhất là thay luật ngưỡng cứng bằng mô hình có thể ước lượng độ tin cậy của nhãn phân loại. Các đặc trưng thời gian như LTS, MS và SDS cũng cần cơ chế tự hiệu chỉnh riêng, vì hiện vẫn dùng ngưỡng cố định. Khi miền ứng dụng có một lượng nhỏ nhãn, có thể dùng chúng để tinh chỉnh ngưỡng hoặc huấn luyện một mô hình nhỏ trên chuỗi MMD, thay vì phụ thuộc hoàn toàn vào đặc trưng thủ công.

6.3.3 Hướng dài hạn

Về dài hạn, hệ thống có thể tiến tới meta-learning trực tuyến để tự chọn chiến lược thích ứng và tham số theo đặc điểm dữ liệu quan sát. Một benchmark chuẩn hóa với dữ liệu thực tế được gán nhãn drift kỹ lưỡng cũng rất cần thiết, vì dữ liệu tổng hợp giúp kiểm soát ground truth nhưng chưa phản ánh đầy đủ độ nhiễu và tính bất định của môi trường sản xuất.

6.4 Kết luận

Luận văn đề xuất hệ thống SE-CDT — một pipeline không giám sát tích hợp phát hiện và phân loại drift, kết hợp với khung thích ứng theo loại drift. Module phát hiện (ShapeDD-IDW) đạt F1 cạnh tranh với DAWIDD và tốt hơn ShapeDD gốc trong benchmark hiện tại, dù chênh lệch thứ hạng không đủ để kết luận khác biệt thống kê với DAWIDD. Module phân loại đạt kết quả chấp nhận được ở mức TCD/PCD nhưng còn yếu ở tiểu loại, đặc biệt với Incremental. Đây là giới hạn lớn nhất trong phạm vi nghiên cứu của luận văn, đòi hỏi sự nhìn nhận khách quan để làm tiền đề cho các hướng tiếp cận tiếp theo.

Kết quả thích ứng cho thấy thông tin loại drift có ích trên Stepping và Sudden, nơi Type-Specific vượt rõ No Adaptation và các baseline retrain đơn giản. Trên Mixed, Type-Specific và Periodic Retrain gần tương đương về độ chính xác trong artifact hiện tại, còn lợi thế rõ hơn của Type-Specific là giảm cảnh báo dư. Vì vậy, đóng góp của luận văn nằm ở việc đặt nền cho một hệ thống drift-aware có thể vận hành không cần nhân tức thời; để tiến tới áp dụng trong môi trường vận hành thực tế, phần phân loại cần được củng cố bằng mô hình xác suất, tự hiệu chỉnh tốt hơn và đánh giá trên dữ liệu thực tế có ground truth đáng tin cậy.

DANH MỤC CÔNG BỐ KHOA HỌC

Thông tin về các công bố khoa học của tác giả có liên quan đến nội dung luận văn:

Hội nghị quốc tế / Tạp chí:

Học viên chưa có công bố khoa học.

TÀI LIỆU THAM KHẢO

- [1] N. Jourdan, T. Bayer, T. Biegel, and J. Metternich, “Handling concept drift in deep learning applications for process monitoring,” *Procedia CIRP*, vol. 120, pp. 33–38, 2023.
- [2] L. Yuan, H. Li, B. Xia, C. Gao, M. Liu, W. Yuan, and X. You, “Recent advances in concept drift adaptation methods for deep learning,” in *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI-22)*. IJCAI, 2022, pp. 5654–5661.
- [3] F. Hinder, V. Vaquet, and B. Hammer, “One or two things we know about concept drift—a survey on monitoring in evolving environments. part a: detecting concept drift,” *Frontiers in Artificial Intelligence*, vol. 7, p. 1330257, 2024.
- [4] F. Hinder, J. Brinkrolf, V. Vaquet, and B. Hammer, “A shape-based method for concept drift detection and signal denoising,” in *2021 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE, 2021, pp. 1–8.
- [5] H. Guo, H. Li, Q. Ren, and W. Wang, “Concept drift type identification based on multi-sliding windows,” *Information Sciences*, vol. 585, pp. 1–23, 2022.
- [6] J. C. Schlimmer and R. H. Granger, “Incremental learning from noisy data,” *Machine learning*, vol. 1, no. 3, pp. 317–354, 1986.
- [7] K. Ramakrishnan, D. Preuveneers, and Y. Berbers, “Enabling self-learning in dynamic and open iot environments,” *Procedia Computer Science*, vol. 32, pp. 207–214, 2014.
- [8] G. Hovakimyan and J. Bravo, “Evolving strategies in machine learning: A systematic review of concept drift detection,” *Information*, vol. 15, no. 12, p. 786, 2024.
- [9] J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla, and F. Herrera, “A unifying view on dataset shift in classification,” *Pattern recognition*, vol. 45, no. 1, pp. 521–530, 2012.
- [10] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey

- on concept drift adaptation,” *ACM computing surveys*, vol. 46, no. 4, pp. 1–37, 2014.
- [11] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, “Learning under concept drift: A review,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, 2019.
 - [12] M. Basseville and I. V. Nikiforov, *Detection of abrupt changes: theory and application*. Prentice Hall Englewood Cliffs, 1993, vol. 104.
 - [13] B. Celik and J. Vanschoren, “Adaptation strategies for automated machine learning on evolving data,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 9, pp. 3067–3078, 2021.
 - [14] N. Jourdan, L. Longard, T. Biegel, and J. Metternich, “Machine learning for intelligent maintenance and quality control: A review of existing datasets and corresponding use cases,” in *Proceedings of the Conference on Production Systems and Logistics (CPSL 2021)*. publish-Ing., 2021, pp. 473–483.
 - [15] S. Tripathi, D. Muhr, M. Brunner, H. Jodlbauer, M. Dehmer, and F. Emmert-Streib, “Ensuring the robustness and reliability of data-driven knowledge discovery models in production and manufacturing,” *Frontiers in Artificial Intelligence*, vol. 4, p. 576892, 2021.
 - [16] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, “A kernel two-sample test,” *Journal of Machine Learning Research*, vol. 13, no. 25, pp. 723–773, 2012.
 - [17] B. Schölkopf and A. J. Smola, *Learning with kernels: support vector machines, regularization, optimization, and beyond*. Cambridge, MA: MIT Press, 2002.
 - [18] A. Bifet and R. Gavaldà, “Learning from time-changing data with adaptive windowing,” in *Proceedings of the 2007 SIAM International Conference on Data Mining (SDM)*. SIAM, 2007, pp. 443–448.
 - [19] P. Good, *Permutation Tests: A Practical Guide to Resampling Methods for Testing Hypotheses*, 2nd ed. New York: Springer, 2000.
 - [20] A. Bharti, M. Naslidnyk, O. Key, S. Kaski, and F.-X. Briol, “Optimally-weighted estimators of the maximum mean discrepancy for likelihood-free inference,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 2289–2312. [Online]. Available: <https://proceedings.mlr.press/v202/bharti23a.html>
 - [21] E. S. Page, “Continuous inspection schemes,” *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

- [22] D. V. Hinkley, “Inference about the change-point from cumulative sum tests,” *Biometrika*, vol. 58, no. 3, pp. 509–523, 1971.
- [23] C. Raab, M. Heusinger, and F.-M. Schleif, “Reactive soft prototype computing for concept drift streams,” *Neurocomputing*, vol. 416, pp. 340–351, 2020.
- [24] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, “Learning with drift detection,” in *Advances in Artificial Intelligence – SBIA 2004*, ser. Lecture Notes in Computer Science, vol. 3171. Springer Berlin Heidelberg, 2004, pp. 286–295.
- [25] M. Baena-García, J. del Campo-Ávila, R. Fidalgo-Merino, A. Bifet, R. Gavaldà, and R. Morales-Bueno, “Early drift detection method,” in *Proceedings of the 4th ECML PKDD International Workshop on Knowledge Discovery from Data Streams (IWKDDs 2006)*, Berlin, Germany, 2006, pp. 77–86.
- [26] I. Frías-Blanco, J. del Campo-Ávila, G. Ramos-Jiménez, R. Morales-Bueno, A. Ortiz-Díaz, and Y. Caballero-Mota, “Online and non-parametric drift detection methods based on hoeffding’s bounds,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 27, no. 3, pp. 810–823, 2015.
- [27] A. Pesaranhader and H. L. Viktor, “Fast hoeffding drift detection method for evolving data streams,” in *Joint European Conference on Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*. Springer, Cham, 2016, pp. 96–111.
- [28] A. Pesaranhader, H. Viktor, and E. Paquet, “Mcdiarmid drift detection methods for evolving data streams,” in *2018 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2018, pp. 1–9.
- [29] R. S. Barros, D. R. Cabral, P. M. Gonçalves Junior, and S. G. Santos, “Rddm: Reactive drift detection method,” *Expert Systems with Applications*, vol. 90, pp. 344–355, 2017.
- [30] M. M. W. Yan, “Accurate detecting concept drift in evolving data streams,” *ICT Express*, vol. 6, no. 4, pp. 332–338, 2020.
- [31] O. A. Mahdi, E. Pardede, N. Ali, and J. Cao, “Diversity measure as a new drift detection method in data streaming,” *Knowledge-Based Systems*, vol. 191, p. 105227, 2020.
- [32] P. Wang, N. Jin, and G. Fehringer, “Concept drift detection with false positive rate for multi-label classification in IoT data stream,” in *2020 International Conference on UK-China Emerging Technologies (UCET)*. IEEE, 2020, pp. 1–4.
- [33] S. Bach and M. Maloof, “Paired learners for concept drift,” in *Proceedings - IEEE International Conference on Data Mining, ICDM*, 12 2008, pp. 23–32.

- [34] K. Nishida and K. Yamauchi, “Adaptive classifiers-ensemble system for tracking concept drift,” in *Sixth International Conference on Machine Learning and Cybernetics (ICMLC)*. IEEE, 2007, pp. 3607–3612.
- [35] R. Elwell and R. Polikar, “Incremental learning of concept drift in nonstationary environments,” *IEEE Transactions on Neural Networks*, vol. 22, no. 10, pp. 1517–1531, 2011.
- [36] Ö. Gözüaık and F. Can, “Concept learning using one-class classifiers for implicit drift detection in evolving data streams,” *Artificial Intelligence Review*, vol. 54, no. 5, pp. 3725–3747, 2021.
- [37] M. Pratama, C. Za’in, A. Ashfahani, Y. S. Ong, and W. Ding, “Automatic construction of multi-layer perceptron network from streaming examples,” in *Proceedings of the 28th ACM international conference on information and knowledge management*, 2019, pp. 1171–1180.
- [38] M. Jaworski, L. Rutkowski, and P. Angelov, “Concept drift detection using autoencoders in data streams processing,” in *Artificial Intelligence and Soft Computing (ICAISC 2020)*, ser. Lecture Notes in Computer Science, vol. 12415. Springer, Cham, 2020, pp. 124–133.
- [39] G. Widmer and M. Kubat, “Learning in the presence of concept drift and hidden contexts,” *Machine Learning*, vol. 23, no. 1, pp. 69–101, 1996.
- [40] F. Hinder, A. Artelt, and B. Hammer, “Towards non-parametric drift detection via dynamic adapting window independence drift detection (dawidd),” in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 4249–4259. [Online]. Available: <https://proceedings.mlr.press/v119/hinder20a.html>
- [41] A. Ashfahani and M. Pratama, “Autonomous deep learning: Continual learning approach for dynamic environments,” in *Proceedings of the 2019 SIAM International Conference on Data Mining (SDM)*. Society for Industrial and Applied Mathematics, 2019, pp. 666–674.
- [42] A. Ashfahani, M. Pratama, E. Lughofer, and Y.-S. Ong, “DEV DAN: Deep evolving denoising autoencoder,” *Neurocomputing*, vol. 390, pp. 297–314, 2020.
- [43] Ö. Gözüaık, A. Büyükakir, H. Bonab, and F. Can, “Unsupervised concept drift detection with a discriminative classifier,” in *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*. ACM, 2019, pp. 2365–2368.
- [44] J. Yoon, E. Yang, J. Lee, and S. J. Hwang, “Lifelong learning with dynamically expandable networks,” in *International Conference on Learning*

- Representations (ICLR)*, 2018. [Online]. Available: <https://openreview.net/forum?id=Sk7KsfW0->
- [45] A. Gretton, K. Fukumizu, Z. Harchaoui, and B. K. Sriperumbudur, “A fast, consistent kernel two-sample test,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 22, 2009, pp. 673–681.
 - [46] J. Kreps, N. Narkhede, and J. Rao, “Kafka: a distributed messaging system for log processing,” in *Proceedings of the NetDB workshop*, 2011.
 - [47] Apache Software Foundation, “Apache kafka documentation,” <https://kafka.apache.org/documentation/>, 2024.
 - [48] G. Wang, J. Koshy, S. Subramanian, K. Paramasivam, M. Zadeh, N. Narkhede, J. Rao, J. Kreps, and J. Stein, “Building a replicated logging system with apache kafka,” *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1654–1655, 2015.
 - [49] M. Kleppmann and J. Kreps, “Kafka, samza and the unix philosophy of distributed data,” *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, vol. 38, no. 4, pp. 4–14, 2015.
 - [50] K. Goodhope, J. Koshy, J. Kreps, N. Narkhede, R. Park, J. Rao, and V. Y. Ye, “Building linkedin’s real-time activity data pipeline,” *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, vol. 35, no. 2, pp. 33–45, 2012.
 - [51] B. R. Hiranman, A. M. Viresh, S. A. Patil, and A. B. Kolekar, “A study of apache kafka in big data stream processing,” in *2018 International Conference on Information, Communication, Engineering and Technology (ICICET)*. IEEE, 2018, pp. 1–3.
 - [52] J. Montiel, M. Halford, S. M. Mastelini, G. Bolmier, R. Sourty, R. Vaysse, A. Zouitine, H. M. Gomes, J. Read, T. Abdessalem, and A. Bifet, “River: machine learning for streaming data in python,” *Journal of Machine Learning Research*, vol. 22, no. 110, pp. 1–8, 2021. [Online]. Available: <https://jmlr.org/papers/v22/20-1380.html>
 - [53] W. N. Street and Y. Kim, “A streaming ensemble algorithm (sea) for large-scale classification,” in *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining*, 2001, pp. 377–382.
 - [54] G. Hulten, L. Spencer, and P. Domingos, “Mining time-changing data streams,” in *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining*, 2001, pp. 97–106.
 - [55] M. Harries, “SPLICE-2 comparative evaluation: Electricity pricing,” School of

Computer Science and Engineering, University of New South Wales, Technical Report, 1999.

- [56] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.

PHỤ LỤC

A. Bảng cấu hình tham số thực nghiệm

Bảng dưới đây tổng hợp các tham số chính được sử dụng trong toàn bộ thực nghiệm. Các giá trị được lấy từ mã nguồn triển khai và giữ cố định trong suốt quá trình đánh giá, trừ khi được ghi chú riêng.

Bảng A.1: Cấu hình tham số Detection Module (ShapeDD-IDW)

Tham số	Giá trị	Mô tả
l_1	50	Kích thước cửa sổ tham chiếu (reference window)
l_2	150	Kích thước cửa sổ kiểm tra (test window)
α	0.05	Mức ý nghĩa thống kê cho kiểm định drift
N_{perm}	2 500	Số vòng hoán vị cho ShapeDD gốc (permutation test)
Stride (s)	10	Bước trượt cửa sổ
Kernel	RBF	Hạt nhân Gaussian $k(x, y) = \exp(-\gamma \ x - y\ ^2)$
γ	median heuristic	Ước lượng một lần trên đoạn đầu của luồng
ϵ	0.5	Hằng số ổn định số học trong IDW weighting
Weighting scheme	Inverse density	Phương pháp tính trọng số IDW-MMD dựa trên mật độ cục bộ
Gaussian filter σ_g	4	Độ lệch chuẩn bộ lọc Gauss làm mượt tín hiệu
Peak threshold	$\bar{\sigma}_s + 0.3\sigma_{\text{std}}$	Ngưỡng phát hiện đỉnh ứng viên

Bảng A.2: Cấu hình tham số Classification Module (SE-CDT)

Tham số	Giá trị	Mô tả
<i>Ngưỡng phân loại (mặc định / trước self-calibration)</i>		
WR threshold (τ_{WR})	0.15	Ngưỡng Peak Width Ratio cho Sudden
WR threshold (TCD/PCD)	0.12	Ngưỡng WR trong EstimateDriftDuration: $\leq 0.12 \Rightarrow$ TCD, $> 0.12 \Rightarrow$ PCD
SNR threshold (τ_{SNR})	2.0	Ngưỡng Signal-to-Noise cho TCD
CV threshold (τ_{CV})	0.3	Ngưỡng Periodicity CV cho Recurrent
LTS threshold	0.5	Ngưỡng Linear Trend Strength cho Incremental
MS threshold	0.6	Ngưỡng Monotonicity Score (kết hợp với LTS)
SDS threshold	0.12	Ngưỡng Step Detection Score (kết hợp với LTS)
PPR threshold	0.20	Ngưỡng Peak Proximity Ratio cho Blip
DPAR threshold	0.60	Ngưỡng Dual-Peak Amplitude Ratio cho Blip
<i>BlipProfile — biến thể noisy (kết hợp với compact_pair)</i>		
WR (noisy blip)	< 0.17	Peak hẹp nhưng rộng hơn compact
SNR (noisy blip)	(1.45, 2.60)	Tín hiệu trung bình, không quá mạnh
DPAR (noisy blip)	(0.45, 0.85)	Hai đỉnh gần đối xứng
LTS (noisy blip)	< 0.12	Không có xu hướng tuyến tính
drift_length	> 1	FWHM vượt ngưỡng TCD/PCD do nhiễu
<i>Concept Memory</i>		
M	8	Số lượng snapshot tối đa trong bộ nhớ vòng
n_s	150	Số điểm dữ liệu mỗi snapshot
τ_{match}	0.15	Ngưỡng Standard-MMD để so khớp concept ($\approx 2 \times$ nhiễu nền)
<i>Self-Calibration</i>		
baseline_size	200	Kích thước bộ đệm vòng rolling baseline
baseline_min_history	20	Số mẫu tối thiểu trước khi kích hoạt tự hiệu chỉnh

Bảng A.3: Cấu hình Benchmark đánh giá

Tham số	Giá trị	Mô tả
<i>Detection Benchmark</i>		
Stream size (T)	10 000	Số mẫu mỗi luồng
Số drift events	10	Đặt cách đều mỗi 1 000 mẫu
N runs	30	Số lần chạy độc lập (seed spacing: $42 + i \times 137$)
δ (tolerance)	75	Độ trễ chấp nhận cho TP (cửa sổ đối xứng)
Cooldown	150	Khoảng cách tối thiểu giữa hai lần phát hiện
<i>Classification Benchmark</i>		
Số kịch bản	6	Mixed A, Mixed B, Repeated $\times 4$ loại
Seeds/kịch bản	30	Tổng cộng 180 luồng, 1 800 sự kiện
δ (tolerance)	250	Phù hợp với drift kéo dài
δ_{early}	50	Cảnh báo sớm
<i>Adaptation Benchmark</i>		
Stream size	5 000	Mẫu/luồng
Drift events	5	Sự kiện/luồng
N runs	30	Số lần chạy
δ (tolerance)	400	Đánh giá khả năng phục hồi
Mô hình nền	Logistic Regression	Tối đa 1 000 vòng lặp
Prequential window	100	Cửa sổ tính Prequential Accuracy

Bảng A.4: Cấu hình nguyên mẫu Kafka

Tham số	Giá trị	Mô tả
Broker	localhost:19092	Redpanda (tương thích Kafka API)
Topic (input)	sensor.stream	Luồng dữ liệu đầu vào
Topic (results)	drift.results	Kết quả phát hiện drift
Topic (accuracy)	model.accuracy	Chỉ số độ chính xác real-time
Group ID	shapedd-detector	Consumer group
Buffer size	750	Kích thước bộ đệm phát hiện
Chunk size	150	Tần suất kiểm tra drift
Adaptation delay	50	Mẫu chờ sau phát hiện trước khi thích ứng
Adaptation window	800	Mẫu dùng cho huấn luyện lại

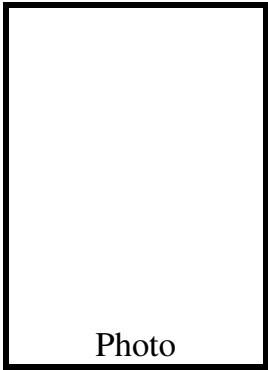
PHẦN LÝ LỊCH TRÍCH NGANG

Họ và tên: Lê Phúc Đức

Ngày, tháng, năm sinh: 29/09/2000

Nơi sinh: Thành phố Hồ Chí Minh

Địa chỉ liên lạc: 75/54 Nguyễn Tư Giản, phường
An Hội Tây, Tp.HCM



Email:

QUÁ TRÌNH ĐÀO TẠO (Bắt đầu từ Đại học đến nay)

Thời gian	Bậc đào tạo	Trường
.....		
.....		
.....		

QUÁ TRÌNH CÔNG TÁC (Bắt đầu từ khi đi làm đến nay)

Thời gian	Đơn vị công tác	Chức vụ
.....		
.....		
.....		